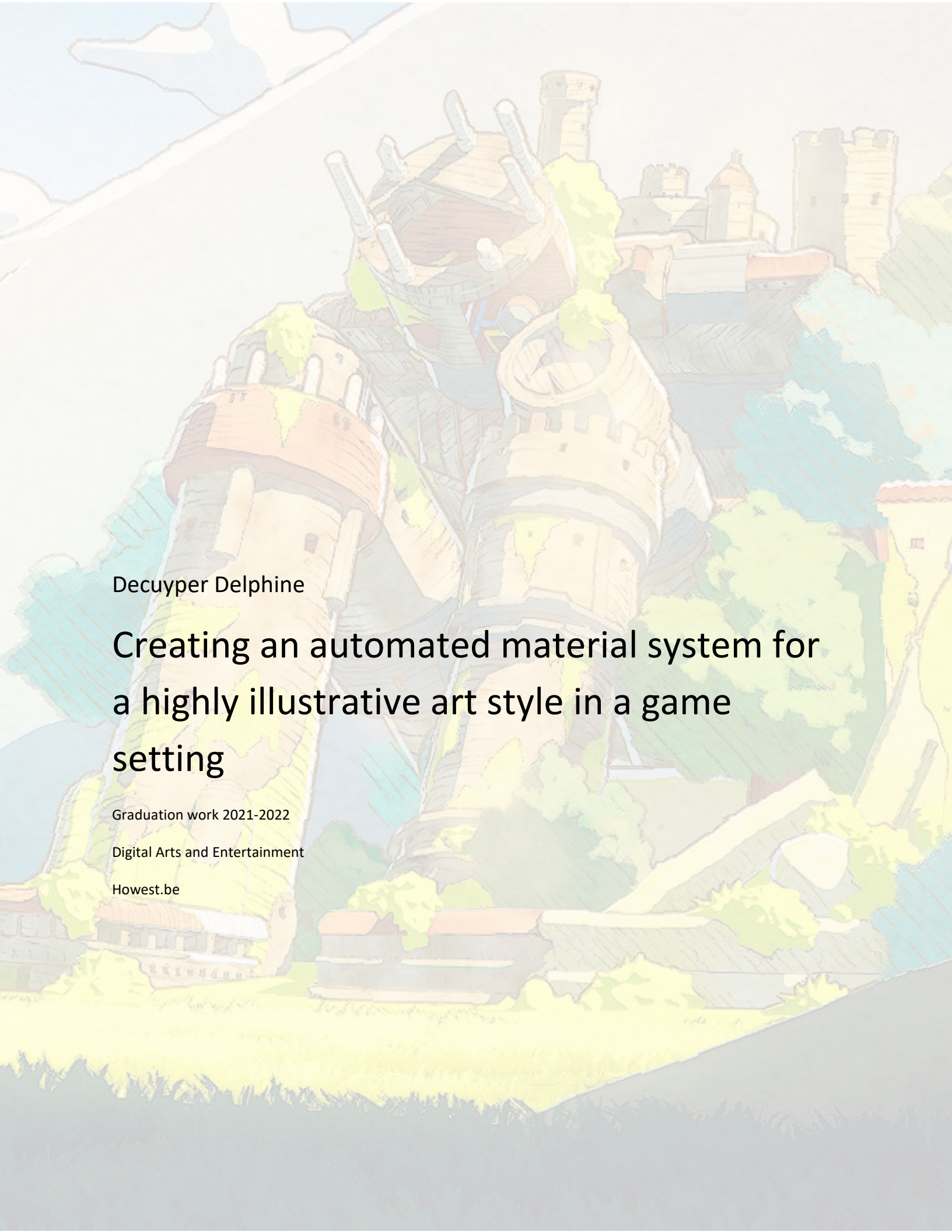




Decuyper Delphine

Creating an automated material system for a highly illustrative art style in a game setting

Graduation work 2021-2022

Digital Arts and Entertainment

Howest.be

CONTENTS

Introduction.....	3
Research and related work.....	4
1. PBR vs NPR.....	4
2. Unity vs Unreal Engine	4
3. Post-processing materials in Unreal Engine	4
4. The layered material system in Unreal Engine	7
5. Making 3D look 2D	9
Case study.....	9
1. Breaking down the artstyle	9
Automating the main look with Post-Processing materials	11
2.1 Setting up a post-processing material	11
2.3 Outlines shader	12
2.5 Cel shader	24
2.6 Hatching shader	36
2.7 Fog and distance fade	41
2.8 Paper overlay	42
2.9 Masking objects with custom depth and custom stencils	42
2. Automating texturing using Layered materials	44
2.1 Creating a layered material.....	44
2.2 Creating material layers and blends	45
2.3 Scene layers and blends.....	46
3. Creating procedural textures in Substance Designer	53
5.1 Hatching texture	54
5.2 Watercolor texture	57
5.3 Moss patches decal.....	59
4. Modeling and scene composition.....	60
Results and evaluation.....	61
Results	61
Evaluation.....	69
Conclusion	75
Bibliography.....	76

Decuyper Delphine

Concept art used during this research by Oswaldo KATO:

<https://www.pixiv.net/en/users/550944>

<https://twitter.com/ozoztv/status/1488089680172269569>

Special thanks to my supervisor for the support, valuable insights, and kindness:

<https://jordylakiere.net/WeWhoAreAboutToDie-Game>

INTRODUCTION

Hey! I'm Delphine Decuyper, a 3D generalist based in Belgium with a love for stylized art. I'm currently in my last year of University, wrapping up my bachelor's degree in Digital Arts & Entertainment: Game Graphics Production. Over the past semester I've become particularly interested in the use of Post-Processing materials to alter the visuals of a scene/game. I've always been fond of 2D hand-drawn games like Hollow Knight, Cuphead, GRIS, Child of Light,... However, I found that styles like these are a bit harder to find in the 3D world and so I decided to do my graduation work on this topic.

Translating the effect and feel of a 2D style onto a functioning real-time 3D environment is a challenge that sparked my interest tremendously. This topic was approved and granted to me on the condition that I would research a way to automate a big part of this process. Because of that, I will explore how a 2D style can be achieved in 3D, what is key in selling the look and how several automated systems can be implemented to speed up the workflow.

In this article, I will explore in depth how post-processing materials can be used to alter the general look of a scene to fit a 2D style. However, altering the general look of a game is not enough to achieve an art style. Textures and materials need to be tailored to it. For this, I will explore how the layered material system in Unreal Engine can be used for the texturing of a 2D style. These aspects will be put to the test by recreating a specific concept. In doing so, I will attempt to formulate an answer to the following questions:

- How effective are these systems for an artist?
For this I will review how well the set-up went and how time efficient, user-friendly and reusable it is.
- How well does the style run in a game setting?
I will review shader complexity and performance in-game.
- What could make or break this auto-generated look for a player?
I will review what the shaders do well and where there is room for improvement.

The goal of this project is to get as close to the concept as possible with these systems. Another goal is to expand these systems so they can become customizable for a user to the point where it's possible to quickly create a variety of 2D hand drawn looks, applicable to any kind of 3D scene in Unreal Engine.

I do want to mention this project was only possible because other artists explored it and were willing to share their workflows. They are credited throughout this paper and in the bibliography section.

I hope this paper and the artists mentioned can motivate and help artists dive into this topic. It certainly is an interesting part of the field and so many styles are still untouched and waiting to be translated.

RESEARCH AND RELATED WORK

1. PBR VS NPR

The translation of 2D art styles in a 3D environment has been a hot topic in the gaming industry for quite some time. Because of its aesthetic, certain styles have become very popular. For example, *Guilty Gear*, which fits into the anime style, and *The Walking Dead*, which is more a western comic. These games provide evidence that 2D styles can be achieved in 3D and can succeed in selling the look. However, I found that more experimental art styles are usually the path less traveled in the industry and it's harder to find examples of those in the game industry.

In the 3D industry a distinction is made between physically based or photo-realistic rendering (PBR) and non-photorealistic rendering (NPR). PBR is a computer graphics approach that renders out light in the same way as the real world. Many PBR techniques thus aim to achieve realism. However, since some of the previous examples of games don't want to achieve realism, but more the recreation of a certain art style, an alternative approach to rendering light and shading is needed. Then we come into the field of NPR. NPR is an area of computer graphics that aims to shade in a way that a variety of expressive art styles become possible. NPR has appeared in many video games and movies under its most common and popular example: the cel shader. The cel shader is designed to make 3D graphics appear to be flat by using less shading.

2. UNITY VS UNREAL ENGINE

Learning about the distinction between PBR and NPR made it clear that, first and foremost, an alternative way of rendering light would need to be implemented to flatten out a 3D scene. While exploring the workflow of existing games ([Guilty Gear](#)) and personal work of artists online (definitely check them out - [Nick Carver](#), [Dermott Burns](#), [Kamil Hepner](#),...), I discovered this was mostly approached by making custom shaders. In Unity through Shadergraph and in Unreal Engine through Post-Processing Materials. It's important to note it's also possible to script effects into these engines. Because the focus of this paper is Unreal Engine and Post-Processing Materials, I won't discuss Shadergraph.

3. POST-PROCESSING MATERIALS IN UNREAL ENGINE

As the word suggests, post-processing is the work that happens after the initial frame is fully rendered. Another pass is then added to that image, applying all the settings from your post-processing volumes. Settings like these could include color adjustments, screen-space ambient occlusion, bloom,...

Post-processing materials are materials you can make and use within a post-processing volume to alter the look of a scene beyond the typical settings of the volume. With these materials, you can add in custom alterations like screen effects, area type effects, or adjust the general visuals of your game without the implementation of scripts. These materials can be stacked in the volume. Each material you add to the stack adds another pass on top of the final render and thus also adds to the performance cost. The performance cost depends on the complexity of the material made. It's good practice to combine passes and only activate the ones needed. It's also good practice to use the existing image adjustments settings within the post-process volume as much as possible for what you want to achieve before creating any custom materials as these are most optimized.

Post-processing is thus part of the render pipeline. The default renderer in Unreal Engine is the deferred renderer. This means that all lights are applied *deferred*. This is achieved by first rendering the scene using shaders designed to output basic attributes of geometry such as position, normal, and diffuse color. Other attributes might include per-pixel specular values and other material properties. These are stored in full screen render targets, collectively

known as the G-buffer. Once the G-buffer has been generated, it's possible to compute a fully lit result for any pixel on the screen (StackExchange, 2010). With post-processing materials, you can use the SceneTexture node to access the different render targets in the G-Buffer used to compose the ultimate image. These render targets can be used in your calculation steps for your post-processing effect. A typical example is using the render target Scene Depth to create outlines around an object, which is something we will tackle later for the style.

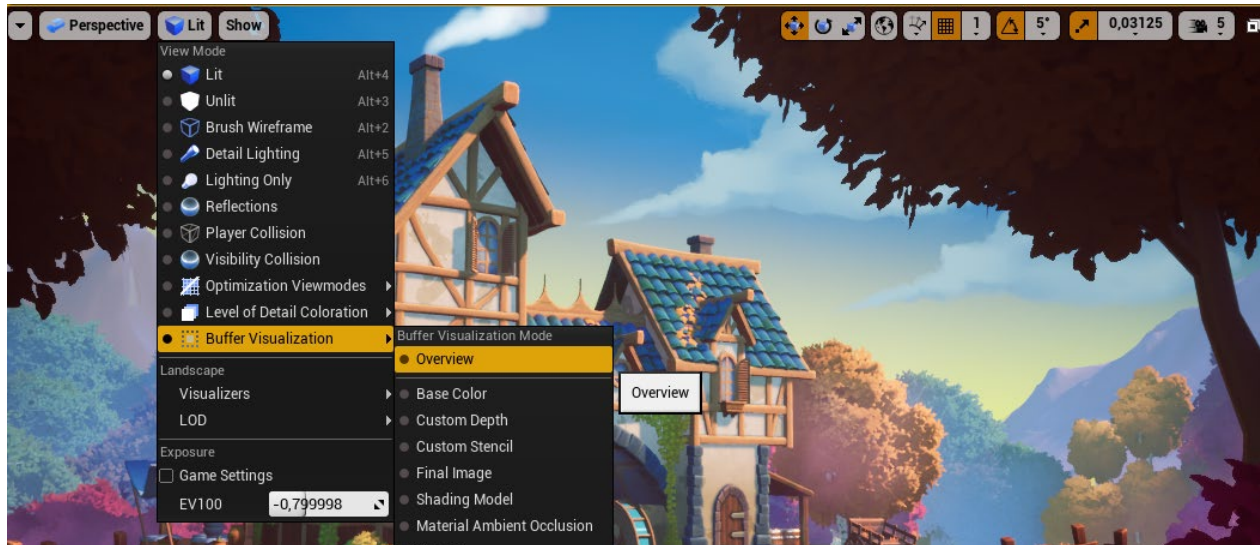


Figure 1: How to access the G-buffer in UE

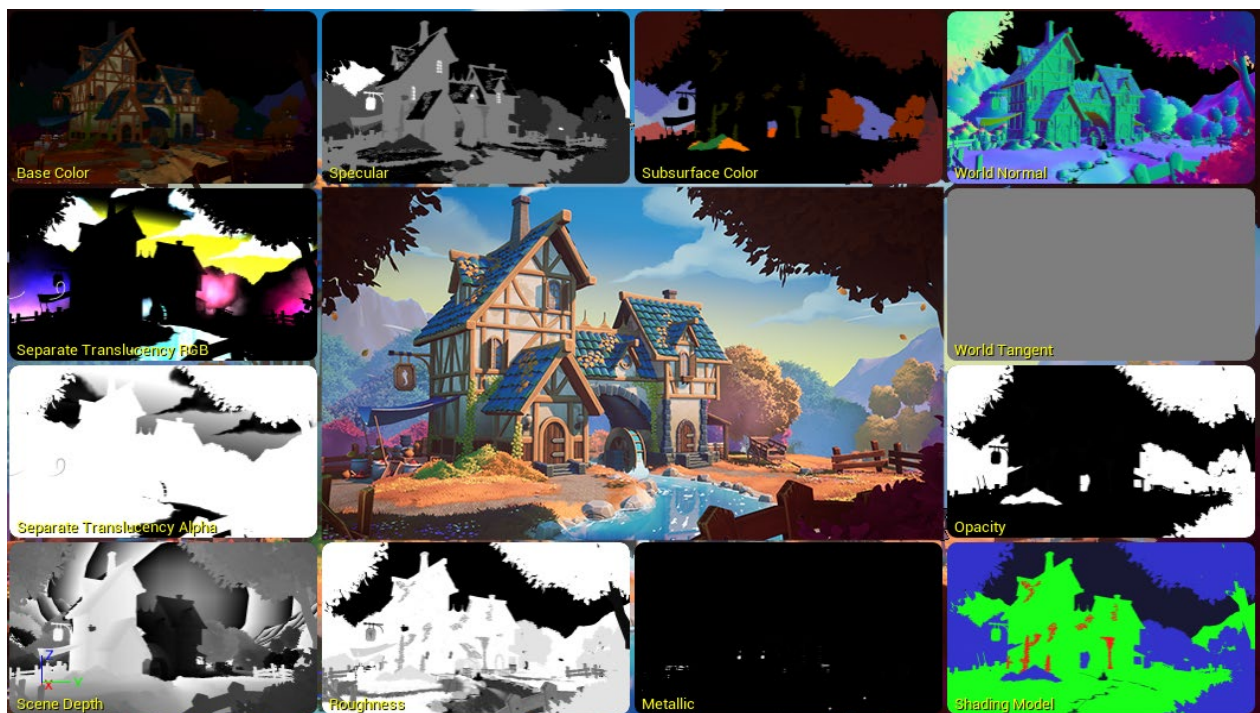


Figure 2: Example of the visualization of the G-buffer in UE

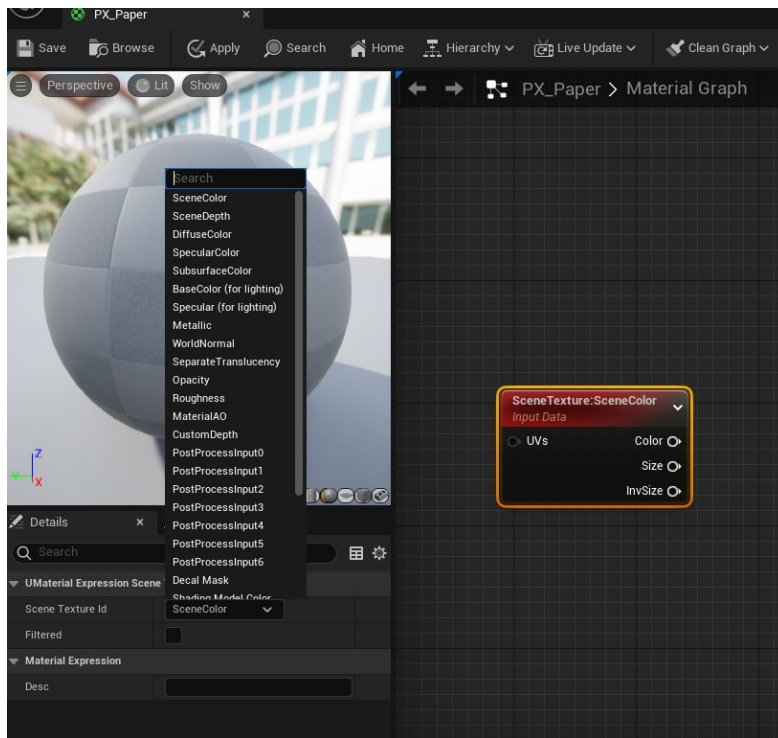


Figure 3: The SceneTexture node in a Post-Processing material and the IDs you can set it to/the render targets you can access

4. THE LAYERED MATERIAL SYSTEM IN UNREAL ENGINE

Altering the general look of a game is not enough to achieve an art style. Textures and materials need to be tailored to it and will also play a significant role in selling the look. While looking for a way to automate and speed up this part of the pipeline, I came across the game [“The Cycle” by developer YAGER](#) and their benefits of using the layered material system in Unreal Engine. This workflow was also researched and explored by [Silke van der Smissen](#) to completely texture a stylized scene in-engine.

Layered materials are an extension of the material functions in Unreal Engine. As mentioned before, a material function is a self-contained node network that lets you package a part of a material into a reusable asset. You can then insert this asset into the node network of any master material you want. Any adjustments you make to these functions then gets automatically propagated to the master materials and instances that are using it. The goal of these functions is mostly to create a material library of which you can easily pick functions you need, like glass, moss, dirt, metals,... and insert wherever you want.

The layered material system allows you to create a single material in which you can stack multiple sub-materials of your choosing. The different sub-materials, called material layers – a form of material functions that you can layer, then get blended on top of each other per pixel using a blend asset – a form of a mask.

The difference between material functions and the layered material system is that layered materials provide a user interface in which you can easily add, swap or remove layers from their stack. In contrast, with material functions you need to get into your node network and adjust it to add or remove what you want. This also results in a lot more master materials if you need variation.

Underneath, we can see an example of the layered materials stack from the game “The Cycle” by developer YAGER. They used the layered material system to make assets fit in more with their environments. The workflow was as follows: they cleanly textured assets in Substance Designer and then used the system to build up wear and tear to adapt the assets to their respective environments. Because they had a material library consisting of different wear and grime based on environment, they could easily swap out certain weathering.

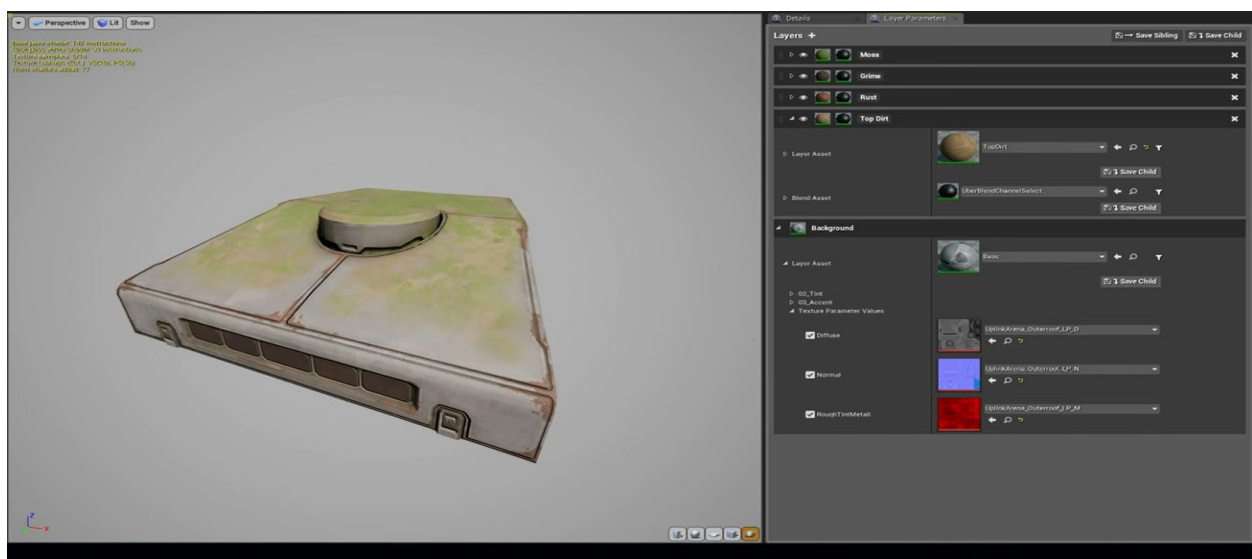


Figure 4: Layered materials for environments in The Cycle – Matthew Brady from YAGER (2019)

Working with this system proved very beneficial for them. Some of its key benefits were:

- **Reduced complexity:**
The system has the power of material functions but without the complexity of the traditional workflow (complex graph of material functions in a master material)
- **Manageability:**
The system offers more control and is much more manageable. Especially with the UI
- **Flexibility:**
The stack, layers and blends are much more editable in the future in comparison with the traditional workflow: It's easy to create a new layer and add it to the stack vs creating a new function and adding it to the existing graph of a master material
- **Clean base materials:**
There's less work to be done in the base material. Most of the work goes into the assets (layers and blends)
- **Instancing:**
You can instance material layers and blends. You can stack the same layer with different inputs multiple times without overriding their inputs
- **Consistency:**
Since the system works with decals as well, the same wear and tear can be used for both assets and decals. This creates a more coherent and consistent look of a level.

Epic Games also created their own version similar to the system and experienced certain benefits by using this for Gears of War. For example, artist no longer had to "guess" how a fully externally textured asset would look in-engine. By being able to build up certain effects in-engine, they could receive that feedback immediately.

Seeing these workflows and the benefits experienced made me wonder how I could apply this research to my topic.

When looking at the concept for the case study, I noticed that the objects were all build- up of different layers of effects. Take the robot: it has a base watercolor layer, on top of that another watercolor layer, then hatching, a white rim light and moss patches. I also noticed that some objects only had a few of these layers, meaning that not every object needed the same amount of layers.

This warmed me up to the idea of implementing a similar system and creating a small material library to texture assets for a highly illustrative art style in-engine.



5. MAKING 3D LOOK 2D

When looking at how [Guilty Gear](#) approached their style, they often stressed that the key to achieving 2D in 3D is the artists intentionality. Since nothing in the world is perfect, the eye immediately senses something as off if it is. Adding intentional imperfections will be key to making or breaking the look you're going for. Certain examples could be adding inconsistent line weight, thickness and distortion to make it feel hand drawn. These are just some examples to illustrate the importance of further alterations of math calculations so they don't seem perfect and thus extremely CGI.

CASE STUDY

1. BREAKING DOWN THE ARTSTYLE

While achieving 2D in 3D is a popular subject in the industry, recreating more experimental styles seems to be the path less traveled. That is exactly the reason I chose this specific [concept by Oswaldo KATO](#). Its watercolors, subtlety and soft look will prove to be an interesting challenge.

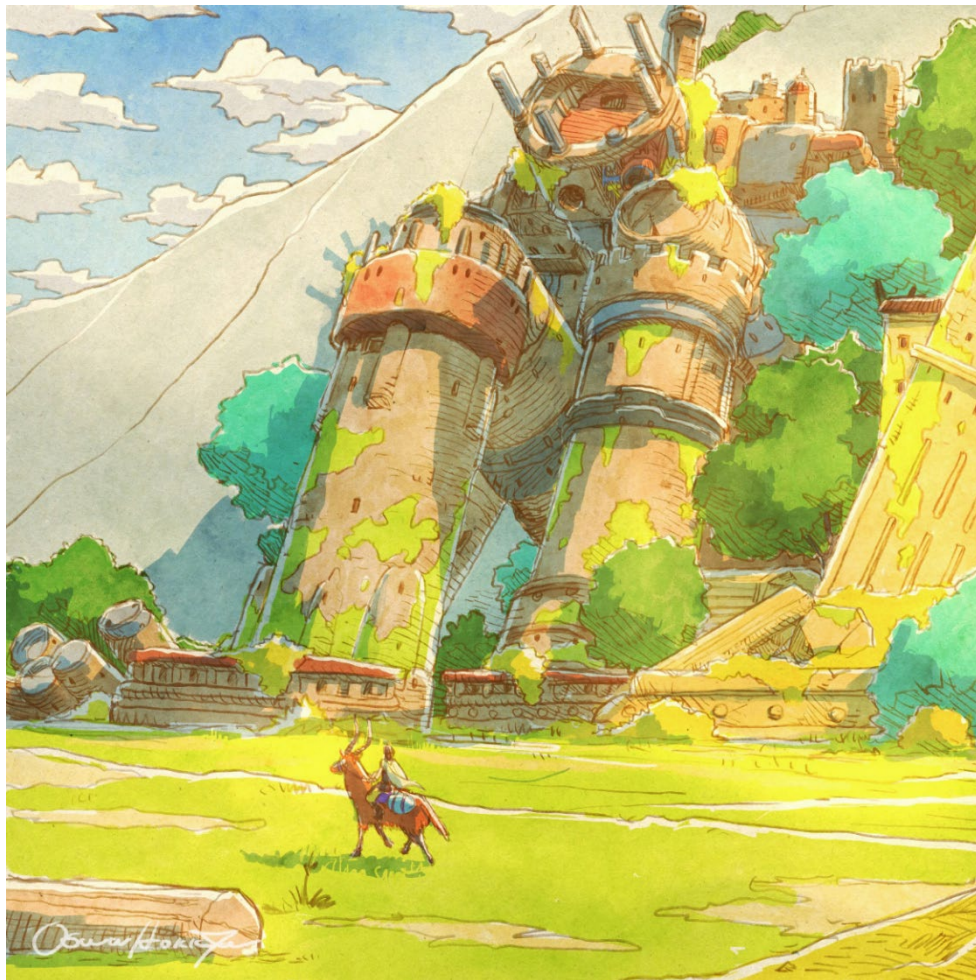


Figure 5: Concept by Oswaldo KATO

Before any other actions, it's important to first and foremost analyze the art style and pinpoint the key characteristics that will have to be recreated.

The key elements for implementation are:

- **The usage of watercolors**
 - There's a "spottiness", which could also be because of the paper being used.
 - Certain areas are subtly bleeding into each other.
 - While the pigment intensity is quite consistent, now and then there's a slight fluctuation.
- **There's a paper texture**
 - There's a graininess to the drawing.
- **All silhouettes have an outline**
 - The outlines are wavy because it's hand drawn.
 - There's a slight inconsistency in the line weight.
 - The lines have a subtle a pencil texture to them.
- **Interaction between light and shadow**
 - There's a clear division between light and shadow, there are no gradients.
 - The shadow is a darker color of the main base color of an object.
- **Hatching**
 - Hatching is used to accentuate volumes.
 - Hatching is also used for shadow areas and the ambient occlusion.
- **There's a white rimlight**
 - This reacts with light: it's brighter in lit areas and darker in shadow areas.
- **Not a lot of detail in the geometry, only focus on the silhouette of shapes**
 - Most of the detail comes from the texturing process.

AUTOMATING THE MAIN LOOK WITH POST-PROCESSING MATERIALS

2.1 SETTING UP A POST-PROCESSING MATERIAL

Creating a post-processing material is done by first creating a normal material. You can then transform a regular material in UE to a post-process material by changing its material domain to “Post Process”. It’s also important to set its blendable location from the default “After Tonemapping” to “Before Tonemapping”. In essence, we’re telling the material to apply this pass earlier (before tonemapping) then it normally does (after tonemapping). This setting avoids the jittering of the silhouette because of temporal anti-aliasing, which often occurs when working with the scene depth. There’s also an option to set the blendable priority of the material. When having multiple post-processing materials, in here you can manually define in which order the passes should be processed. I will leave it at 0 for my shader. An instance is then made to be used within the post-processing volume.

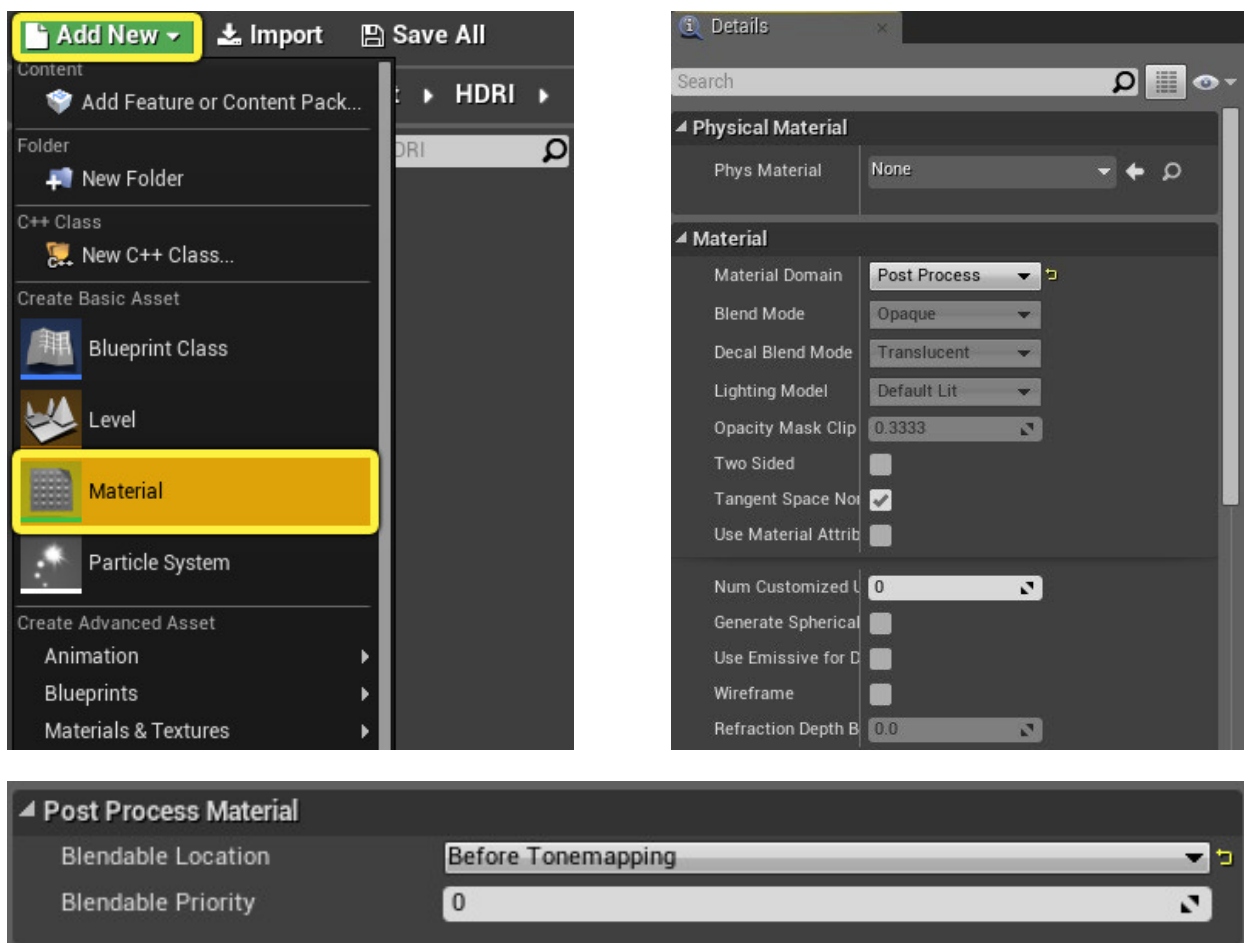


Figure 6: Creating a post-processing material

Post Process volumes have boundaries, but can be set to boundless. Within the volume, under the tab “Rendering Features” post-processing materials can be added. The tab holds an array in which you can drop your post-process materials. This is the post-processing stack I was referring to. Notice that, when inserting a material, the value “1” appears next to it. This signifies a slider that goes from 0 (fully disabled) to 1 (fully enabled). You can think of it like the opacity slider on a layer in Photoshop, though it’s best to stick to a value of either 0 or 1 since any value in between doesn’t mean the pass becomes more transparent, instead it can give an unpredictable result depending

on the nodes. As mentioned previously, you can manually define the priority of a post-process material. If you don't do that, it will follow the order of the materials in the stack. You can add in as many post-processing materials to the stack as you want, but it's best to combine post-processes into one whenever possible.

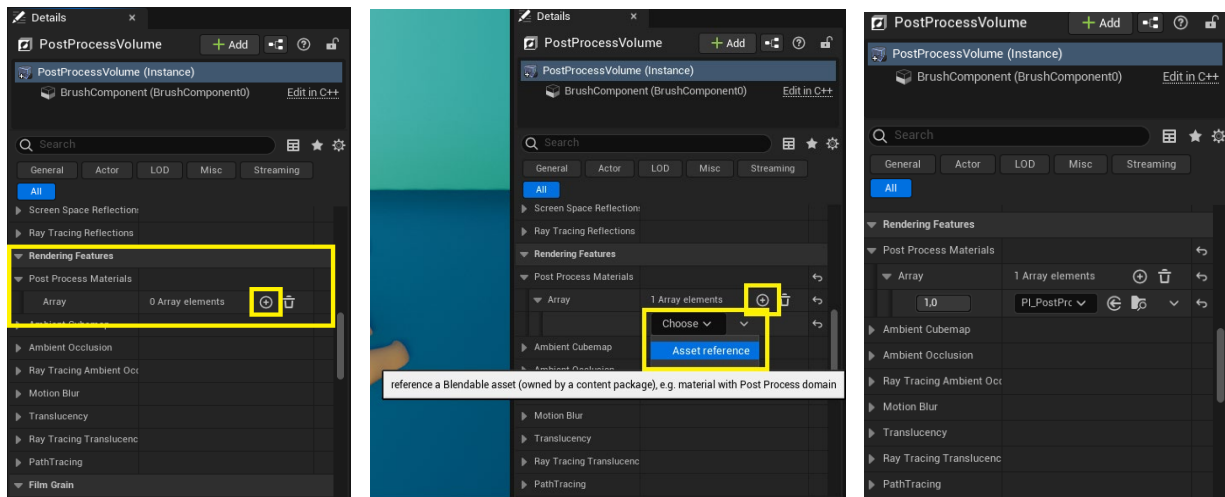


Figure 7: Adding post-processing materials into the post-processing volume

2.3 OUTLINES SHADER

2.2.1 SETTING UP

The first key characteristic of the art style are the outlines. While there are a lot of different ways to approach outlines in 3D (inverted hull, fresnel, dot product,...), I decided to do them in a post-processing material since I need the effect globally and want to support normal detail of textures. To approach this, a Sobel operation will be performed on the Scene Depth, which is extracted from the G-buffer ([Kris Kaufman, 2022](#) and [Kamil Hepner, 2022](#)).

A Sobel operation, also known as the Sobel-Feldman operator, is an edge detection algorithm used in image-processing and computer vision. This operation takes a render and shifts it by one pixel in multiple directions: up and down vertically and left and right horizontally. Each of these results get subtracted from the original render. This subtraction detects discrepancies, which represent the lines. When all of these are then added together, they create the fully outlined image. The effect can be recreated in Photoshop to better illustrate how it works. By duplicating an image, offsetting it by one pixel in any direction and then setting the blend mode to "Subtract", lines will start to show up.

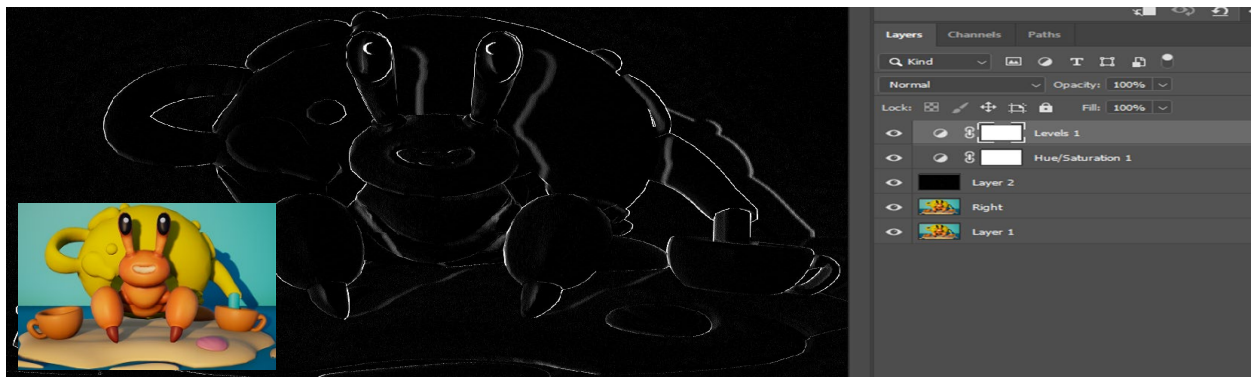


Figure 8: Example render in Photoshop to showcase the effect

To recreate the effect in Unreal Engine, a SceneTexture node is used in the previous created Post-Process material. This SceneTexture is then set to Scene Depth. When the color output of this node is connected with the emissive slot of the material, you can see in the viewport a black and white version of the scene. The Scene Depth node returns a value between 0 and 1 which tells us how close or far away an object is from the camera. When the pixels get darker (closer to 0), it means the object is near. When the pixels get brighter (closer to 1), it means the object is further away. This information can be used to generate outlines even on objects that are far away.



Figure 9: The scene depth in action: the closer objects are, the darker – the further, the brighter

Since the scene depth is in grayscale, we only need to work with one channel from the color output. A component mask is used to disable the G, B and A channel. This will be the render to offset.

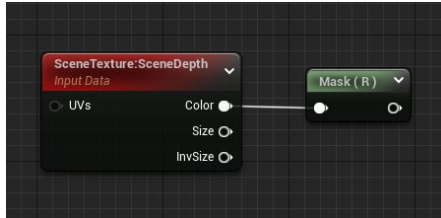


Figure 10: Creating the render of the scene depth we will offset

To offset the render, another Scene Depth is necessary. To get each pixel, we need to use the InvSize output which divides the U (1) and V (1) of the viewport by the width and the height of the window. To create the shift, we multiply the InvSize, which is a 2D vector for the UVs, by the direction we want to offset it. This is also a 2D vector so we can manipulate the U and V individually. The first direction we will offset it in will be the negative U or -1,0.

Now that we have created the shift, we need to add it to the picture. To offset the picture, a Texture Coordinates node is needed. The multiplication can then be added to this node. This result is then used to offset the UV's of the initially created scene depth.

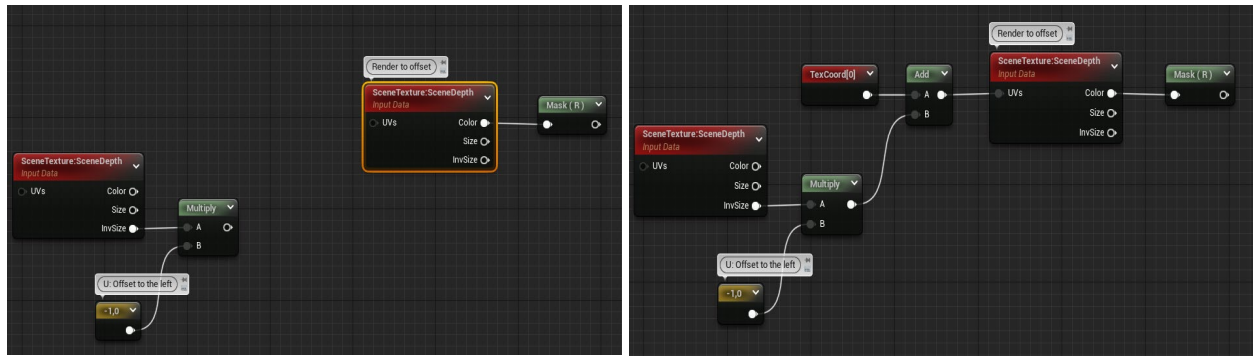


Figure 11: Adding the offset to the image

Last, we need to subtract this offset render from the original image to detect lines in that direction. To create the original image, another scene depth node is added.

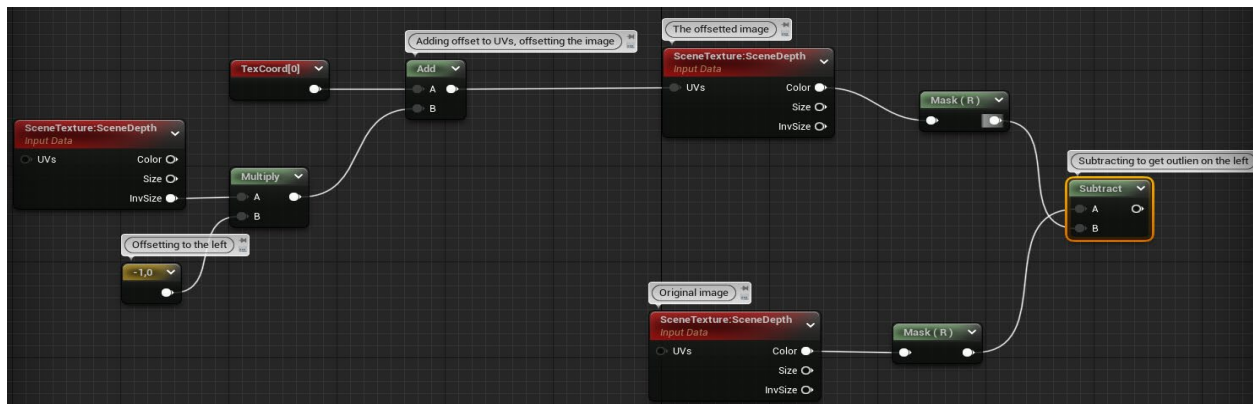


Figure 12 Nodes equivalent of the operation in Photoshop

Outputting this will result in outlines mostly on one side of the objects. That's because edges were detected in that specific direction.



Figure 13: The result of the first offset

The next step is to do this for all other directions: U right [1,0], V down [0,1] and V up [0,-1]. By adding all of these results together, outlines will show up on all sides of the objects. Since we are reusing the same function for every

direction, later in this paper I will optimize this by putting the main calculation in a material function and then reusing the material function instead.

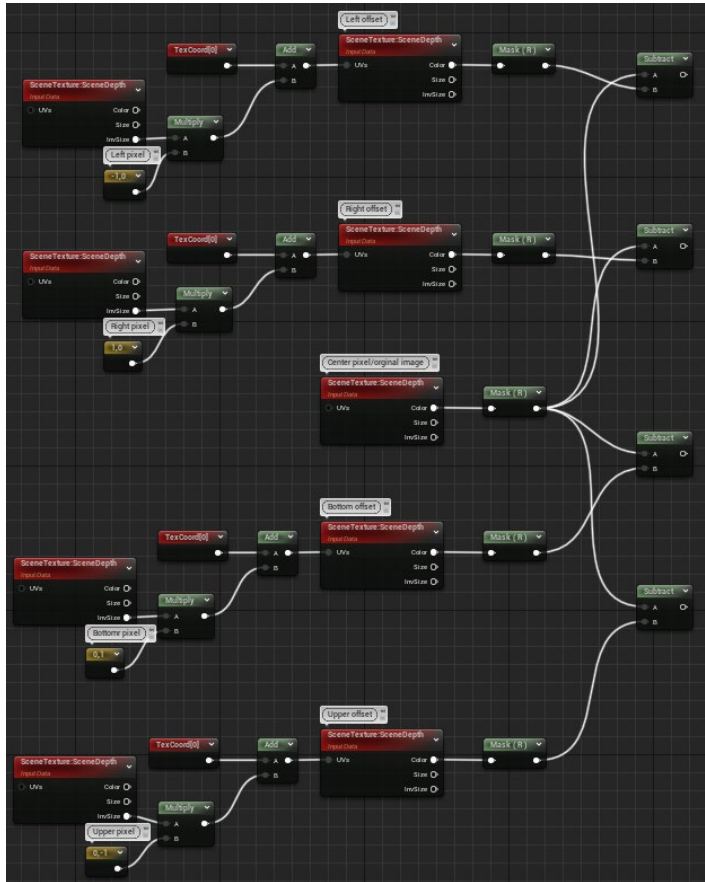


Figure 14: Subtracting each direction from the original image

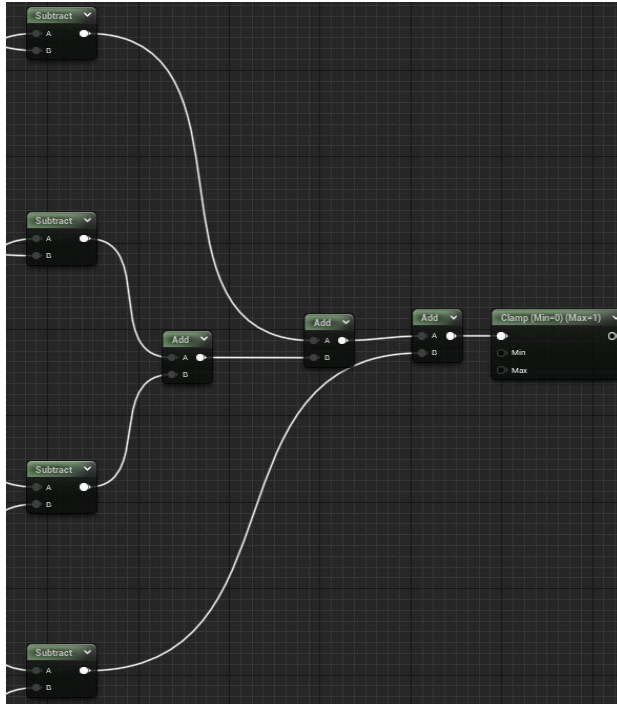


Figure 15: Adding all the subtractions back together to create the fully outlined image

The range of the outlines goes too far and even draws outlines on polygons that shouldn't have them. This can be fixed by multiplying our additions by a scalar. Then, a power node can be added to make the outlines stronger. It's best to clamp this result so all values stay between 0 and 1 and the values don't become burned by further calculations.

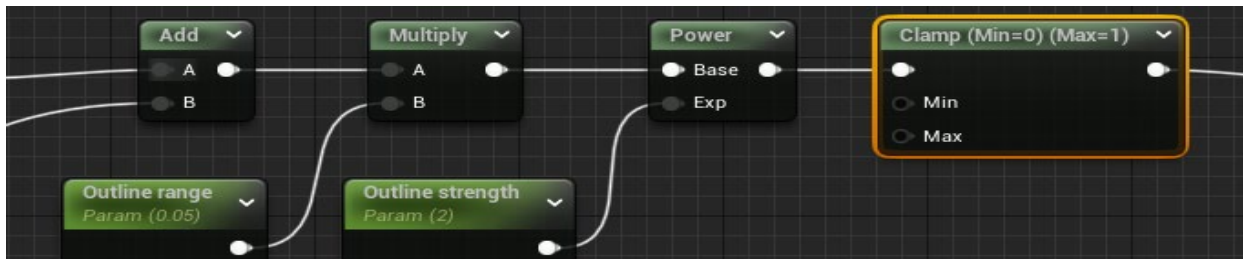


Figure 16: Adjusting the outlines so they only outline the silhouette of a mesh and not inner polygons



Figure 17: Outlines vs the adjusted outlines

Overlaying the outlines on the original scene can be done with a lerp (linear interpolate) node. This node allows you to transition from one component to another based on an alpha/mask. The mask will be our outlines. We can then go from our initial render to a flat color (for the lines).

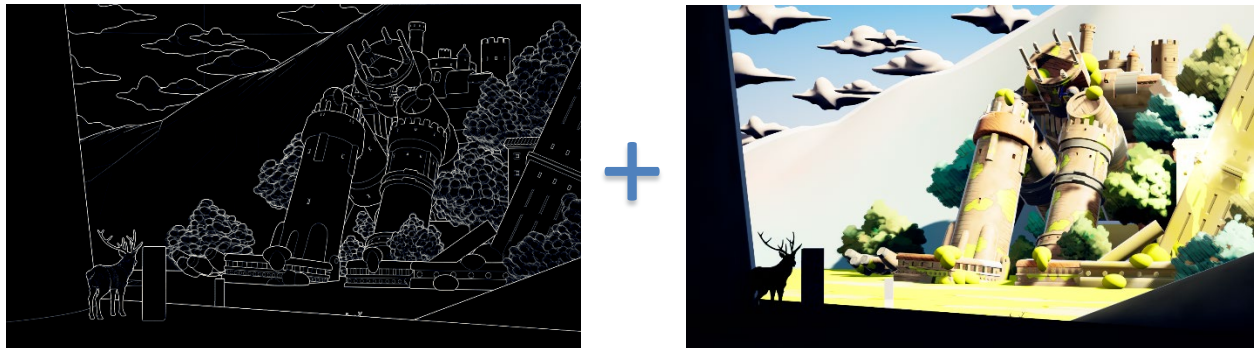


Figure 18: The outlines and the unaltered image of the scene

To get the original scene back, another Scene Texture node is needed. This needs to be set to Post-ProcessingInput0. PostProcessingInput0 is the original scene after the image has been fully rendered and before any post-processing materials are applied, except the build-in settings from the post-processing volume if you've adjusted those. It's worth to note that when you blend something on top of the Post-ProcessingInput0 and output that, in the next material, Post-ProcessingInput0 will have been overridden with what you previously created.

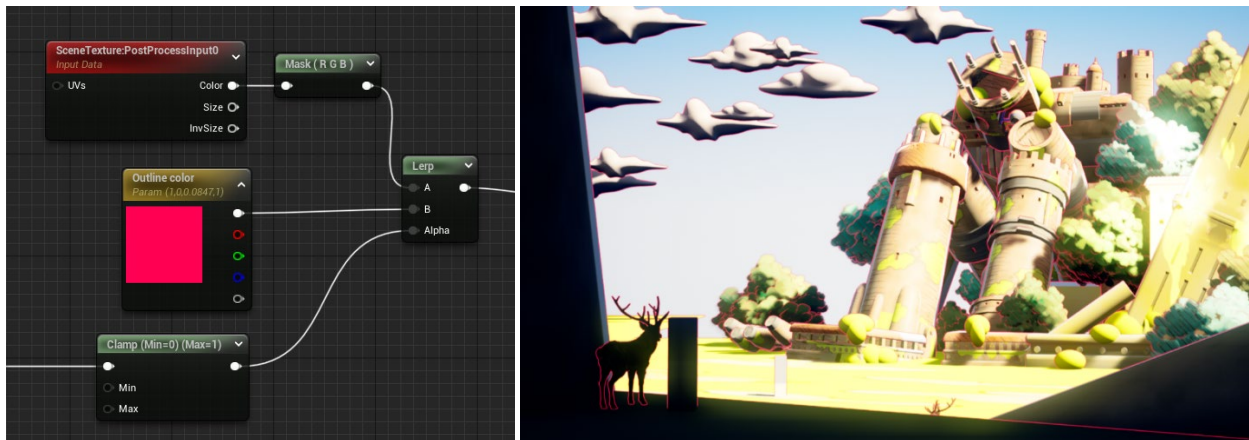


Figure 19: Blending the lines on top of the original scene

2.2.2 ADDING NORMAL LINES

To add in normal detail, we will need to perform the same Sobel operation as before, but this time on the world normal instead of the scene depth. We can copy and paste the same nodes until the addition from the previous operation and change the SceneTexture from Scene Depth to World Normal. Since anything normal-related uses R G and B values, we need to adjust the component mask and enable those channels again.

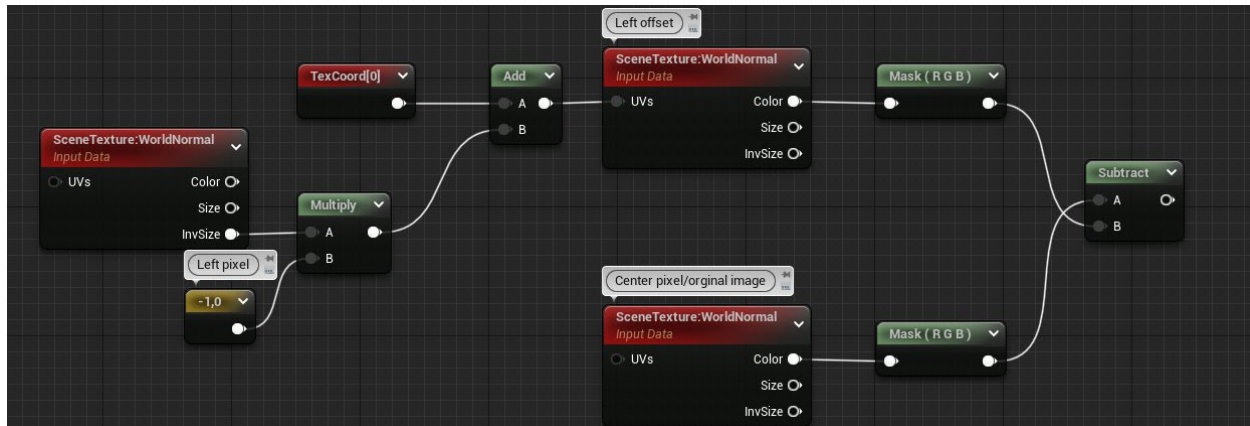


Figure 20: Example of the World Normal version of the Sobel operation

After all the versions are added together, we will put the result in an AddComponents node. It will add all channels into one channel, outputting one value and thus making a grayscale result. Then the same multiplication and power node can be used to further adjust the lines.

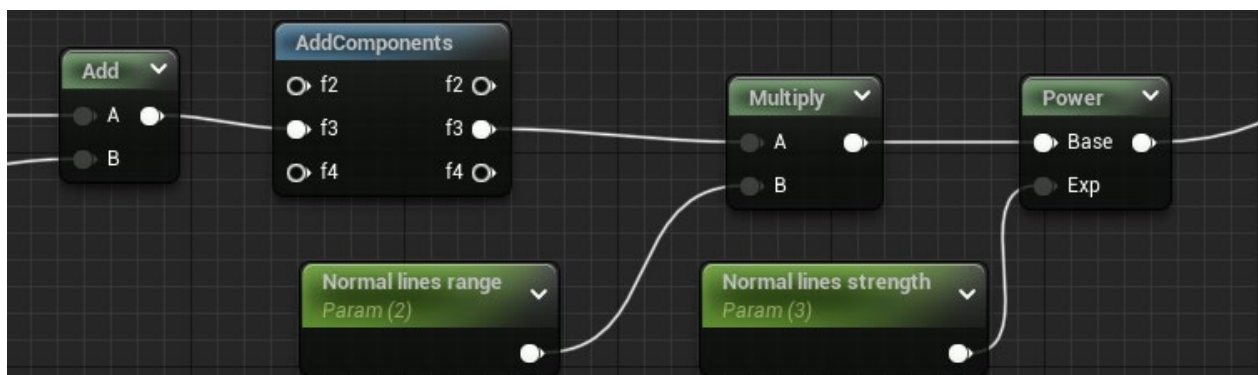


Figure 21: Adjustments for the Sobel operation for normal lines

To combine the normal lines with the outlines, a max node was used. The max node will always output the highest value out of the two inputs it gets. This will make sure that, out of the two masks we just created (the outlines and the normal lines), it will always pick the most prominent value to draw a line on screen. For example, where the outlines don't register anything but the normal lines do, the normal lines value will be used. Where the normal lines don't register anything but the outlines do, the outlines value will be used. This result needs to be clamped again.

If you want to apply the same color to both the normal lines and the outlines, you can use the blend we previously created after this max node. Otherwise, you can use a different lerp for outlines and normal lines and blend them together afterwards.

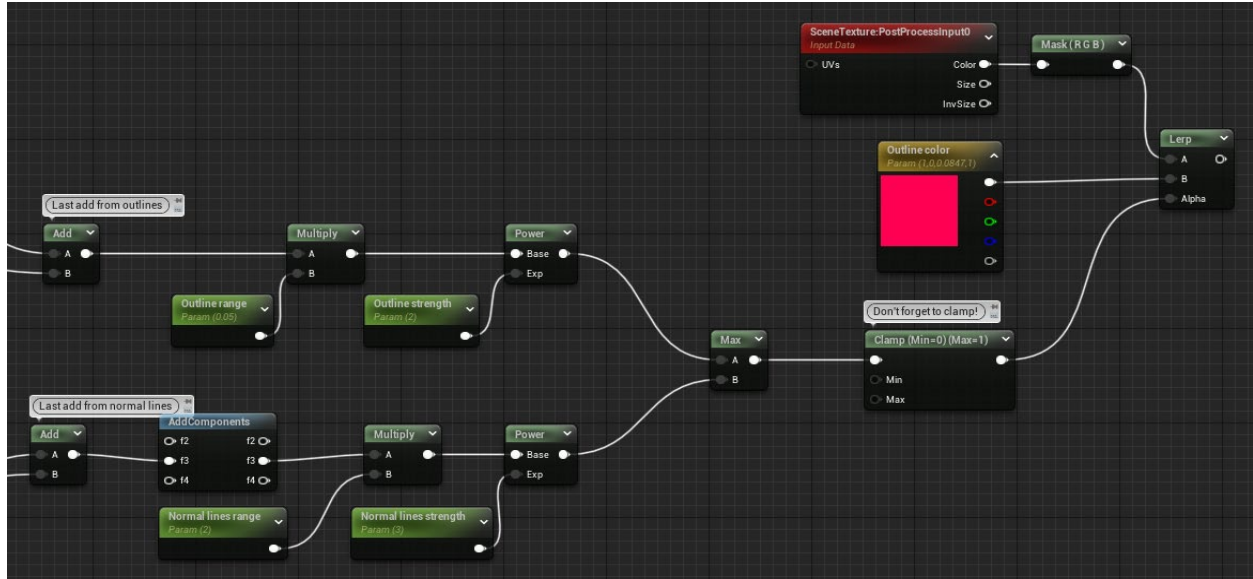


Figure 22: result of adding outlines and normal lines together and coloring them

2.2.3 ADDING LINE THICKNESS

To add in line thickness, the directional offset needs to be multiplied with a scalar value that represents the line thickness. This way, we're telling our shader how many extra pixels it needs to take to do the offset and draw the outline. This multiplication is done for every direction and uses the same scalar. It's the same operation for the normal outlines, though here I wanted to control the thickness of the normal outlines separately, so I created a new scalar for that.

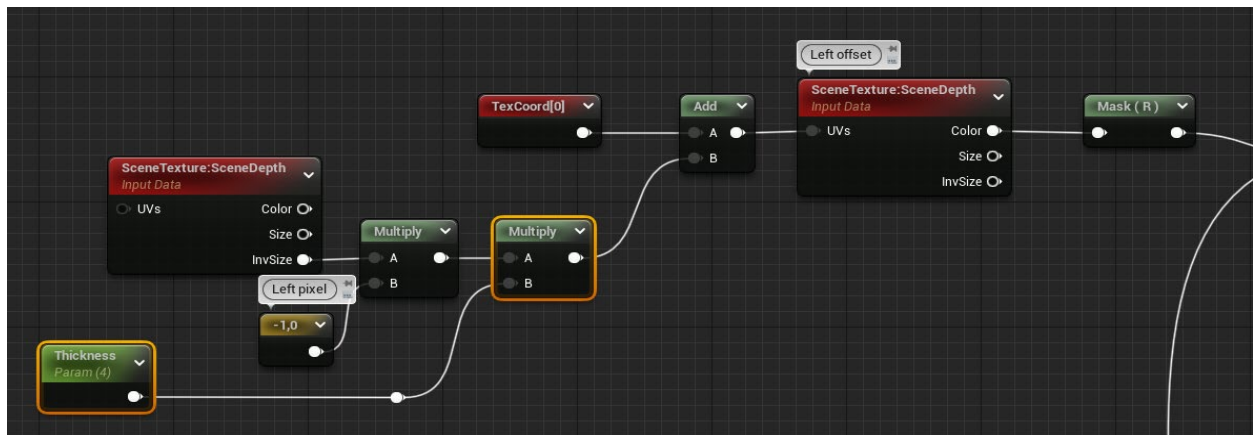


Figure 23: Example of adding in line thickness to a direction.

2.2.4 CHANGING THICKNESS OVER DISTANCE

You might notice that the further away from the scene you are, the more the lines become dense and cluttered because they stay the same pixel size as up close.



Figure 24: Pixel size up close vs from afar

We can adjust our thickness based on the distance of an object with the scene depth and a RemapValueRange node. This method was discovered in [Kamil Hepner](#)'s stylized lines and outlines. We could say:

- For every object **within 600 units** range of the camera, we want the thickness to be 4.
The closer, the thicker we want the lines.
- For every object **between 600-1000 units** range, we want the objects to gradually transition from thickness 4 to 1. So, **the further away, the less thick** the lines become.
- Every object that's **beyond 1000 units'** distance can be capped at 1 pixel line thickness.

If we cap the value that the scene depth node returns to a lower threshold of 600 and an upper threshold of 1000, we will already have completed the first and last condition. That means any value below 600 will be capped at its lower threshold and everything above 1000 will be capped at its higher threshold. This capped value can then be used as the main input for the RemapValueRange node to take care of all the values in between. The RemapValueRange remaps a value from its two original thresholds (600-1000) to two new thresholds (4-1). We can use this remapped value as the scalar to multiply all the directional offsets with. You can use the same remapped value for the normal lines or create a new one if you want different thresholds.

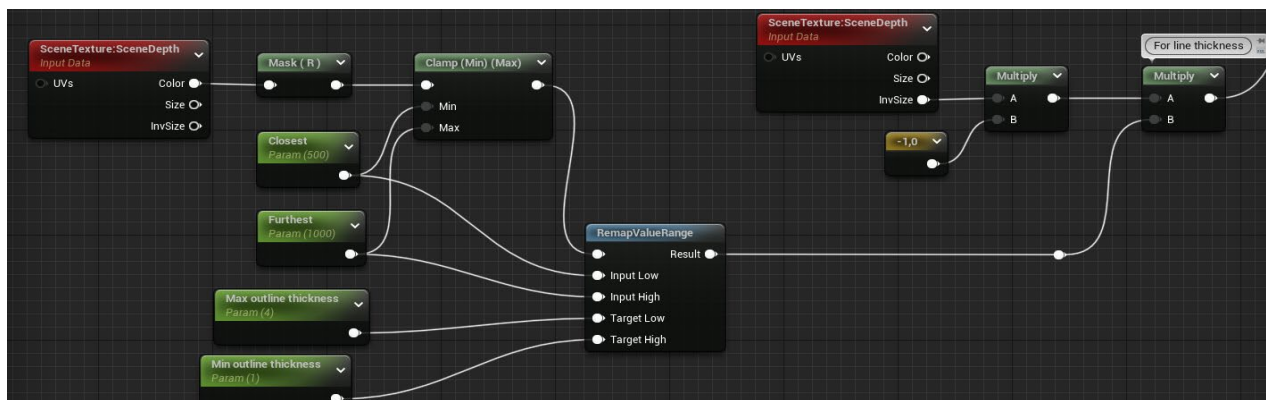


Figure 25: Adjusting line thickness over distance

2.2.5 DISTORTING THE LINES

In traditional, hand drawn art, we never really see any straight or perfect lines. That gives a lot of character to a drawing. It's important to mimic traits like these to make 3D feel like 2D. In that sense, distortion can be added to the outlines to make it feel as if it's hand drawn.

This can be achieved by adding a texture used to distort the thickness of the lines and is a method I saw [Dermott Burns](#) use in their 2D stylized shader. Another option is to make a custom node which holds a few lines of code generating a random number between certain thresholds. It's important to note that it's best practice to use a texture as opposed to lines of code since this greatly increases shader complexity and compilation times.

A grayscale noise texture returns a certain value per pixel. By adding these values to the value of our line thickness, we can make some lines thicker and thus create some irregularity. Depending on the texture size and the strength of the black and white values, this effect can be even more exaggerated. To have that control, a scalar is added to manipulate the tiling of the node and a multiplication to strengthen the values of the texture. This operation gets added to the line thickness, determined by the RemapValueRange, and gets used as the new input for the offset multiplication. If you want the normal lines to be distorted as well, they will need the same operation.

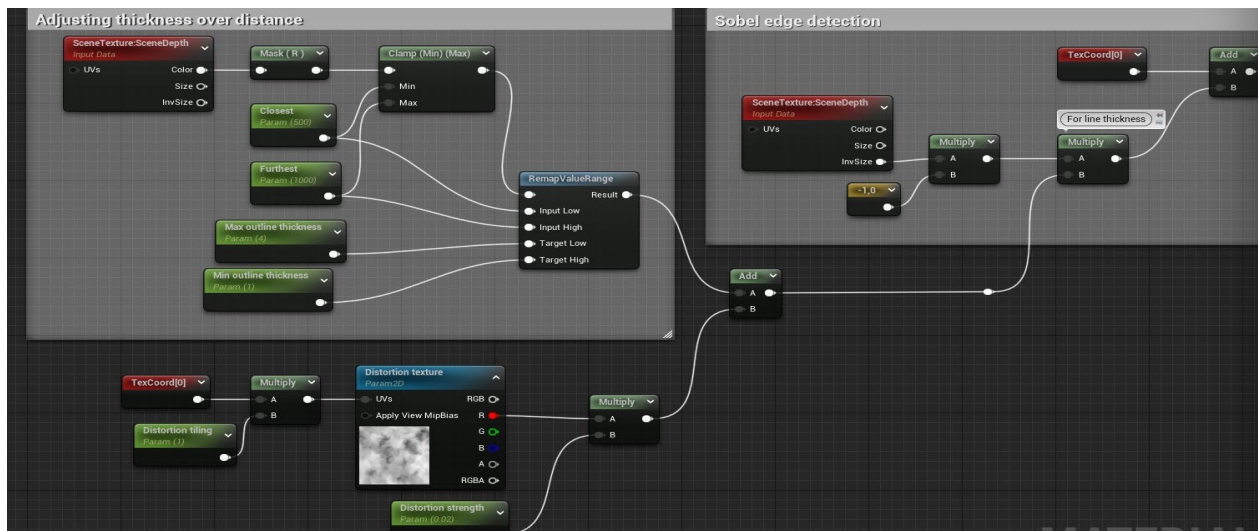


Figure 26: Node equivalent of adding distortion to the outlines.

To give the user a choice whether to apply distortion or not, a Static Switch Parameter was added. This functions as a Boolean does. Then, in the true input, you plug in what it needs to output if it's true and in the false output what it needs to output if it's false.

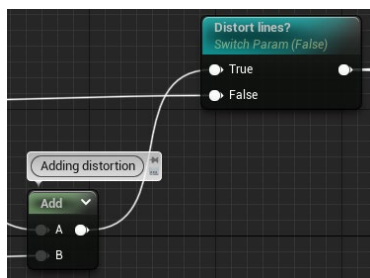


Figure 27: Adding in Booleans in a material

2.2.7 CLEANING UP WITH MATERIAL FUNCTIONS

A material function is a self-contained node network that lets you package a part of a material into a reusable asset. You can then insert this asset into the node network of any master material you want. Any adjustments you make to these functions then gets automatically propagated to the master materials and instances that are using it.

Since it's possible to use material functions in a post-processing material, and since we are using the same parts of code multiple times, we can create a new material function which does this part of the code for us so we can call it whenever we need it later.

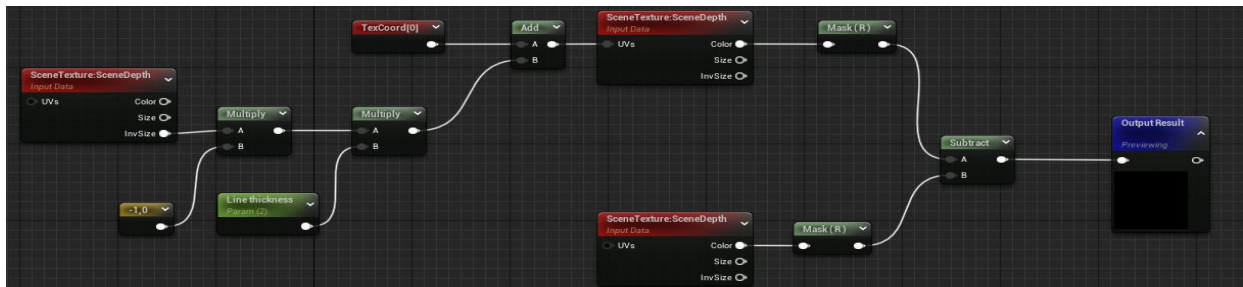


Figure 28: Creating a function for the Sobel operation on one direction

We want this function to have an input for the directional offset and the line thickness so we can pass along the information from the main material to the function to do its calculations. To do that, we need to swap out our vector 2 and scalar with a FunctionInput vector 2 and FunctionInput scalar.

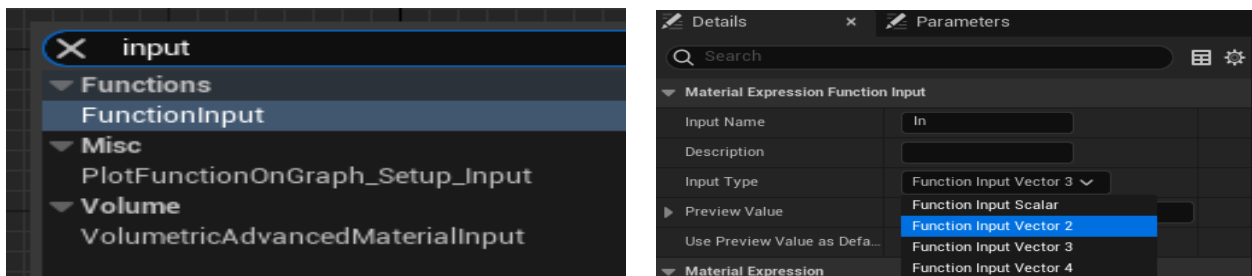


Figure 29: Adding an input slot into our function

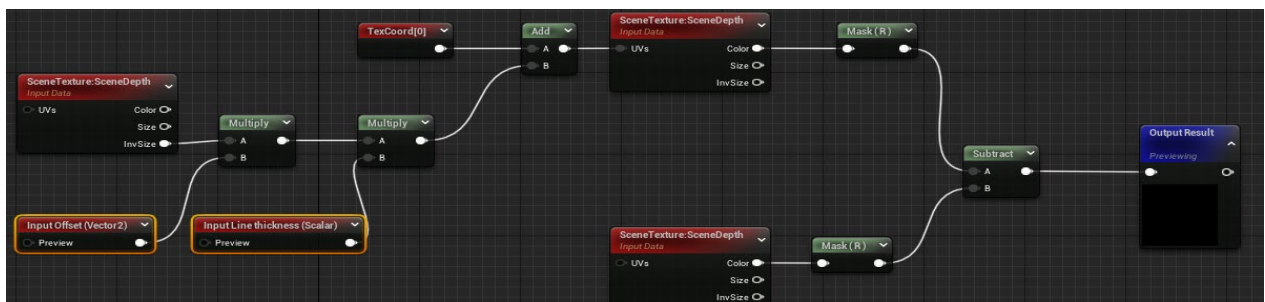


Figure 30: Adding the ability for a master material to pass along information to the function

You'll see the error "SceneTexture expressions cannot be used in opaque materials". This can be dismissed. The SceneTexture nodes of this function will be able to access their information if the function is used in the right material domain, namely post-process. You can call this function by dragging it from your content browser into your material.

You can also enable “Expose to Library” in the material function properties. Then, you will be able to look it up like you would with any other node in the material editor.

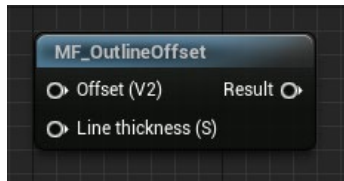


Figure 31: The function in a master material

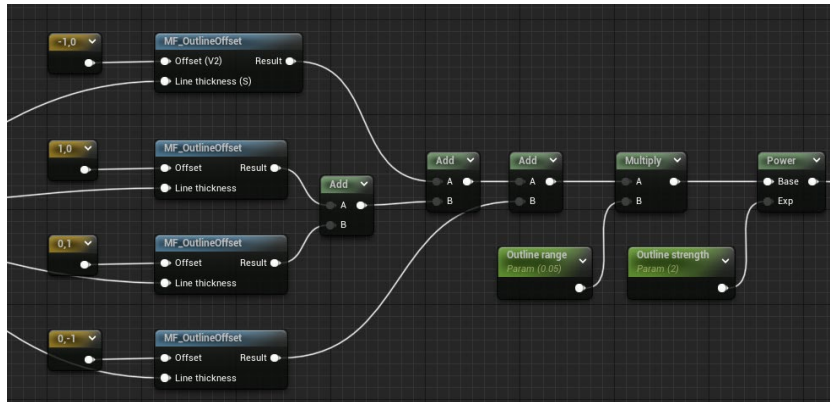


Figure 32: First level of optimization of the Sobel operation

For normal lines, the same can be done. You could even take this a step further by creating a new material function in which you call your outline offset for each direction and add them all together. You can also create a material function in which you paste the entire operation for all directions like we did in the beginning. Using material functions will make your post-process material a lot cleaner, easier to look at and work with.

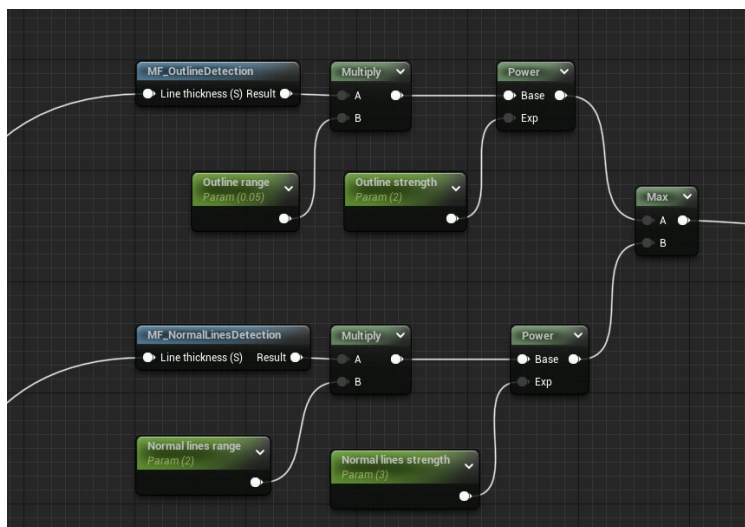


Figure 33: The equivalent of the outlines and normal lines detection using material functions in the post-process material

2.5 CEL SHADER

Another key characteristic of the artwork is that the lighting creates a clear division between lit areas and shadow areas. There are no gradients, which is the case with normal lighting and is the normal way of rendering light (photo-realistic rendering). This means we need to adjust the way our lighting gets rendered. This can be achieved by implementing a cel shader. Cel shading, also known as toon shading, is a non-photorealistic rendering technique designed to make 3D objects appear to be flat by using less shading color. The shading is done in a number of bands of different flat colors. This technique is often used to make 3D look like 2D and is often chosen as a style for games for its aesthetic.

When discussing the key to their art style, the company behind [Guilty Gear](#) mentioned that cel shading is often tricky to pull off well. They said: “In cel shading, everything is either lit or unlit, there’s no in between. This is one of the reasons cel shading is so hard to get right. In a drawing an artist can choose the most convincing distribution of light and darkness but in shaders it’s all math with unforgiving thresholds mercilessly split between light and dark. The slightest difference in the surface normal may end up in a huge blot. This is why precise control is required.”.

There were many elements they implemented to get precise control, one of them being adjusting the normal of a mesh so it would render more aesthetically pleasing.



Figure 34: An example of how Guilty Gear adjusted normals to render better with the cel shader

For the cel shader for this concept a lot of different approaches will be explored, discussed and reviewed based on their procedurality. While for this style it is not needed to go as far as the Guilty Gears approach, it's great to keep in mind in case we run into objects that don't get cel shaded well.

2.5.1 USING A DOT PRODUCT

The most common method to implement cel shading is by using a dot product to compare the normals of a surface with the direction of the light. In essence, we want to know whether the surface normals and the direction of the light are pointing towards each other or not. If they do, it means the surface is lit. The more they point away from each other, the less lit.

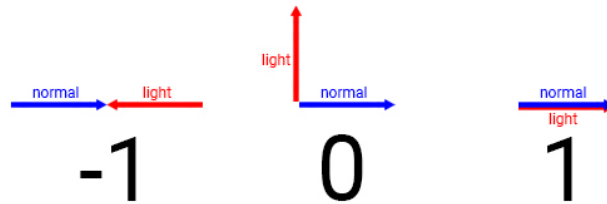


Figure 35: Tommy Tran on Ray Wenderlich – Cel Shading Method

The dot product gives a certain value of the angle between the two and by thresholding the dot product, you can create multiple bands. For example, you can assign a darker color if the dot product is higher than -0.8. If the dot product is lower than -0.8, assign a light color. This will create a two band cel shader ([Tommy Tran, 2018](#)).

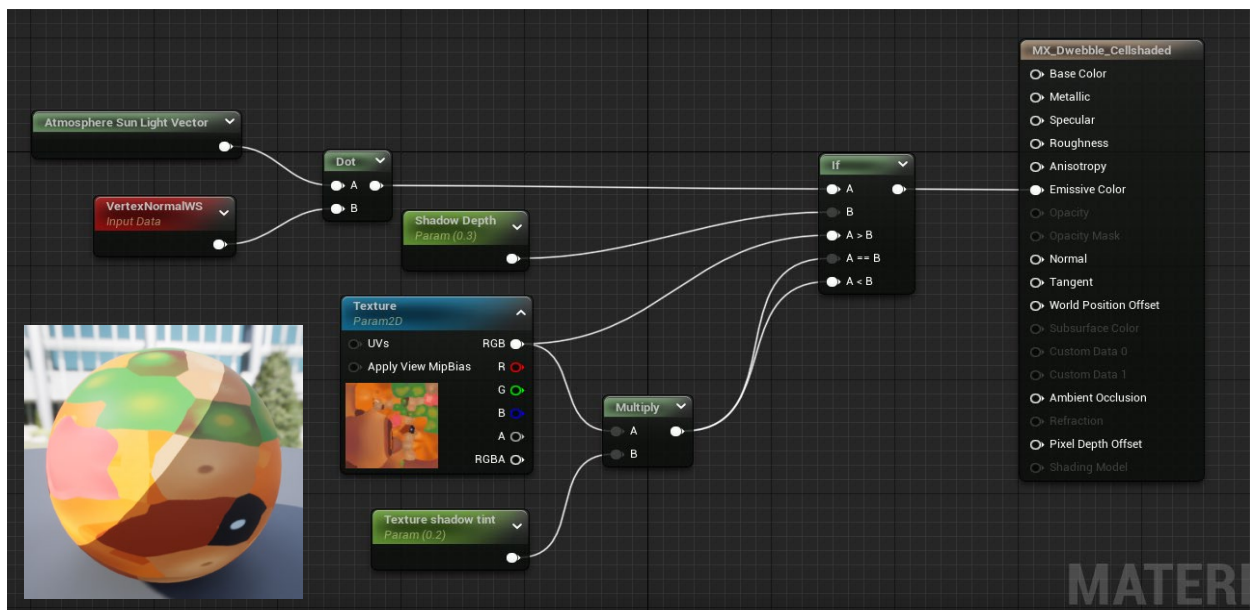


Figure 36: The dot product in a surface material and its result

When a cel shader is to be implemented in a surface material, this seems to be the common way to do it. This method was initially tried to have more control over the way of shading per-object. However, there are some limitations with this method that were also some of the factors that drove me towards a global, post-processing cel shader.

One limitation is that only a directional light can affect the cel shading and only from the directional light can the color be extracted (using the AtmosphericLightColor node). All other lights like spot, point and area lights have no effect or need hard-noding/hard-coding per light to take them into account. Another one is that other objects can't cast shadows on cel shaded objects. This is because, even though another object might be in between the light and the main object, their normals are still pointing at each other and thus the calculations above still count for them.

Because of these limitations and the fact that the scene doesn't require per-object control, I went for a cel shader in a post-processing material. Mind you, with a post-process material it is also possible to create a per object effect by masking objects out from the scene to apply the effect on them. This will be discussed later in the paper. Since post-processing materials gives us access to the G-buffer, we can calculate how lit a surface is instead of using a dot product. This means all light and shadow will be considered and then this value can be used to threshold and create different bands with. Post-processing materials also offer some work-around for implementing light color.

2.5.2 USING AN IF NODE TO DETERMINE A BAND

To implement a cel shader we first need to extract the light information from our unaltered scene. Because of how the G-buffer works, there is no render target that has this information for us, so we need to calculate it ourselves with the render targets we have. In essence, we will divide our black and white unaltered scene, which includes the lighting, by a black and white diffuse color of the scene, which has no light. This will output a new black and white render containing all the light information ([PrismaticaDev, 2020](#)). You could put the light calculation into a material function as well, since this will be reused. In this phase, it's extra important that your post-process material is set to "Before Tonemapping" so you can get the original light values from the scene and not the remapped ones.



Figure 37: Calculating the light information of the scene

We can then plug this information into an if node to create the first band of the cel shader. An if node compares two values to each other. You can then specify what it needs to do/output if one value is bigger than the other, smaller, or if they're equal to each other. In this case, we want to compare the value of each pixel of the light information we just calculated to a threshold we choose ourselves, for example 0.5. This will be our shadow threshold or shadow depth. If the light value is bigger than the shadow threshold of 0.5, we want it to output the diffuse color of the scene. If it's smaller or equal, we want it to output a darker version of the diffuse color, representing the shadows. We can achieve a darker version of the diffuse by multiplying it with a scalar. This will influence the tint of the scene. If you want to have some control over the color of the shadow, you can also multiply the diffuse with a color.

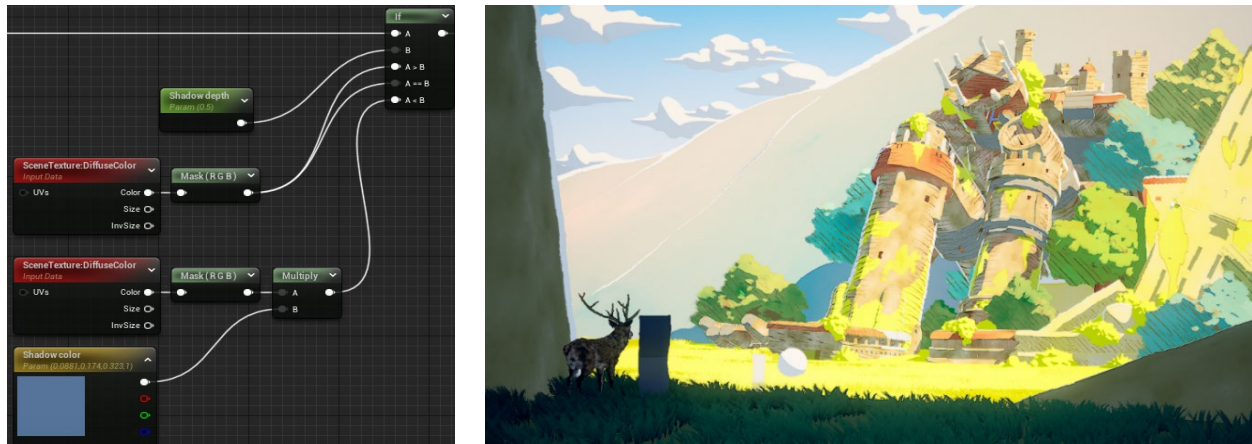


Figure 38: Creating a band for the cel shader

2.5.3 ADDING EXTRA BANDS

To create an extra band, an extra if node is needed. We need to do another value check with the light information, but this time with a lower threshold, for example 0.3. If the value extracted from the light is bigger than 0.3, it can render the band we created with our previous if node. If it's smaller or equal to 0.3, we can output a newly multiplied version of our diffuse scene. If you want to create another band for a highlight, then the light information needs to be checked with a way higher threshold. That's the general workflow for adding bands. This gives you control over every band's depth and color but also has the downside of being hard coded and thus limited to how many bands you code in. This shader is completely dynamic and will thus change along when you adjust your lights.

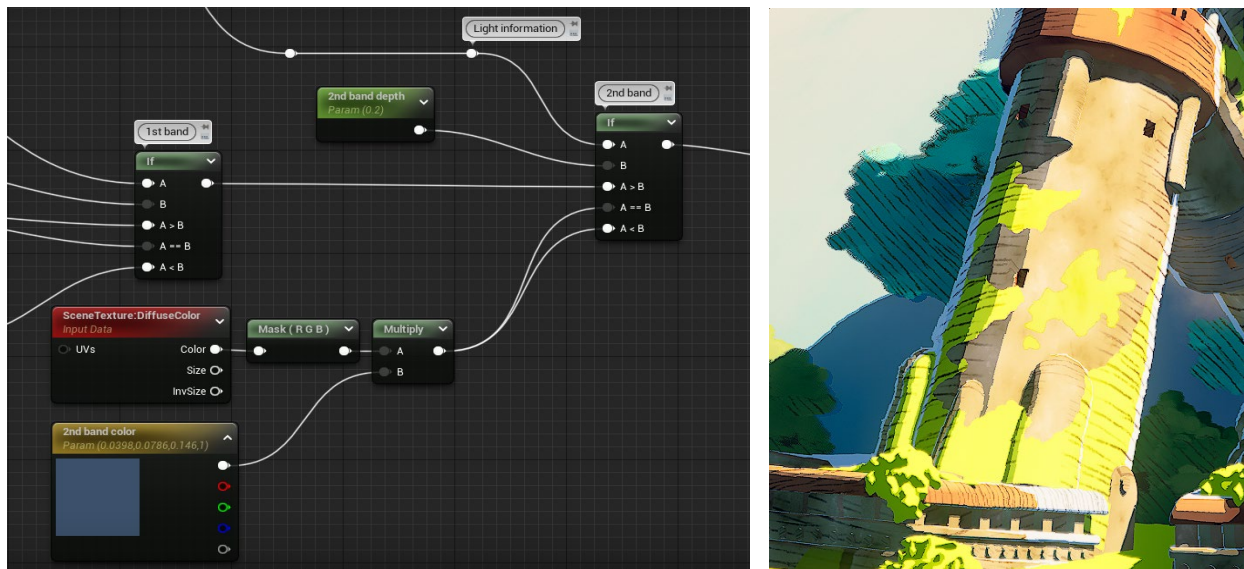


Figure 39: Adding another band to the cel shader

2.5.4 CEL SHADER WITH PROCEDURAL BANDS

If you want a system where the user can specify how many bands they want, there are multiple ways to approach it depending on how much you want to customize each band. Next up is a version in which, in the material instance, you can specify how many bands you want. For this, the lighting information needs to be multiplied by a scalar for

the amount of cels desired. This is reduced to a whole number, posterizing the light information based on the scalar. This is then divided back to the 0-1 range to get normal light and shadow values ([Martin Widdowson, 2020](#)).

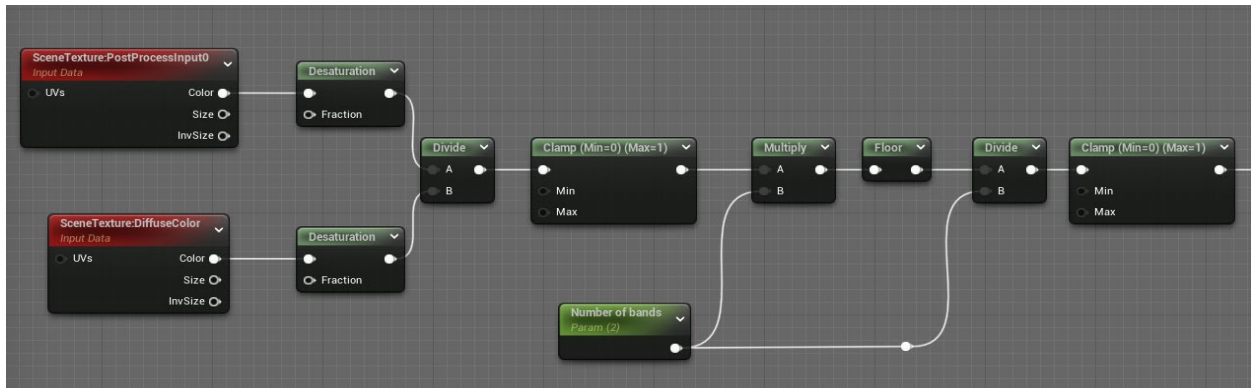


Figure 40: Alternative way for cel shading

If you want further customization of each band, it can be an option to implement a “custom” node. A custom node allows you to write any lines of HLSL code you want your material to perform without having to create and integrate scripts from programming software. In essence, you could write a for-loop that, for the amount of bands that the user wants, checks the light value, tells the system how to render the bands and gives different outputs so the user can have control over each bands depth and color. However, this method was not put to the test since this would definitely increase shader complexity.

2.5.5 MAKING A CEL SHADER USING CURVES AND LUTs

Another way to approach the cel shader is using curves to determine the bands. This provides full control over the amount of bands, as well as the width and color of a band ([Tommy Tran, 2018](#)).

Curves are data containers that hold a set of values interpolated over a range. Curves can be vectors, floats, and colors. To create a curve, go to Add New>Miscellaneous>Curve and choose CurveLinearColor, since we will be using colors for our cel shader. By default, it will give you a grayscale gradient curve to start with. A Curve Linear Color asset can store interpolated RGBA colors. If you double click on the created curve, the curve editor shows up. Since the curve only used the B channel, and since it is interpolated between 0 (black) and 1 (white), it outputs a grayscale gradient.

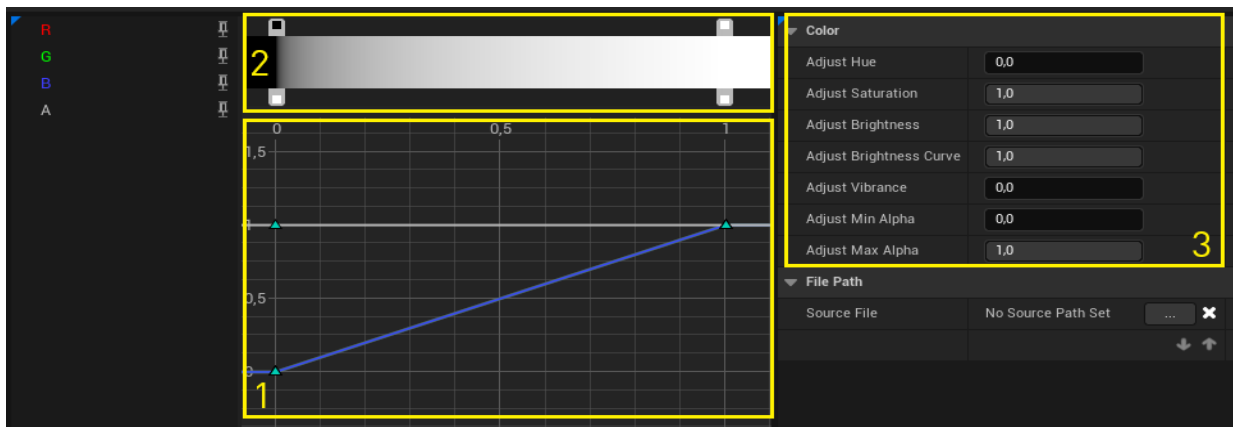


Figure 41: The curve in its asset editor

The editor holds 3 main windows:

1. **The curve graph** in which you can add, subtract and adjust any keys for the RGBA channels (similar to the timeline in blueprints, if you're familiar with that).
2. **The curve gradient result** of the keys from the curve graph. You can also only work with this part of the graph. Then the color keys you add here will automatically create the RGBA keys in the curve graph.
3. **The color panel**, which gives you some post-processing options that affect all the keys.

We can't just use curves like these in the material editor. Only Curve Atlases can do that and so we need to store our curves in there. Then, we can call that atlas and specify which curve it needs to use for our cel shader. To create a Curve Atlas, go to Add New>Miscellaneous>Curve Atlas. Open the Curve Atlas and in the details panel under "Gradient Curves" press the + button to add your created curve into it.

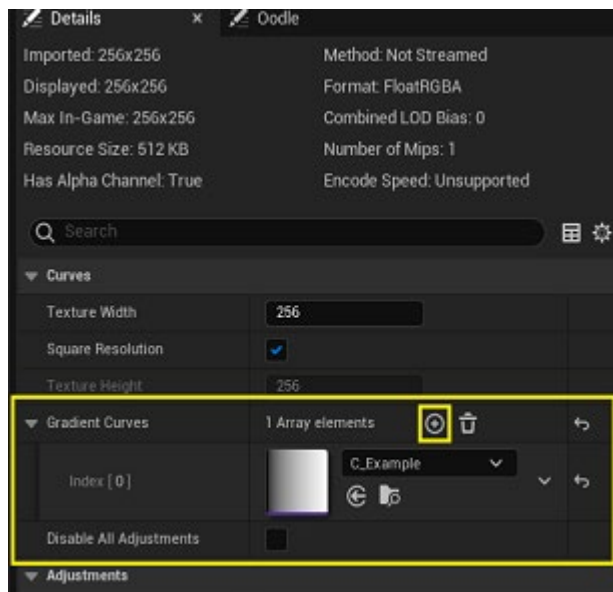


Figure 42: How to add curves to an atlas

We want our curve to remap the values from the extracted light to the values of the curve. So, if a value from the extracted light is underneath a certain value of a key in the curve, it will be mapped to that. If it's above a certain value of a key in the curve, it will be mapped to the key above that. Because of the remapping, it's very important that the extracted light info is clamped from 0 to 1. In the post-processing material, a "CurveAtlasRowParameter" node needs to be added in. This allows us to call our Atlas and thus use curves in materials. In the node, make sure you choose the atlas we just created and choose our example curve as the default curve. Since this node is a parameter, you can always choose a different curve in instances (as long as you make sure the curve is part of the atlas).

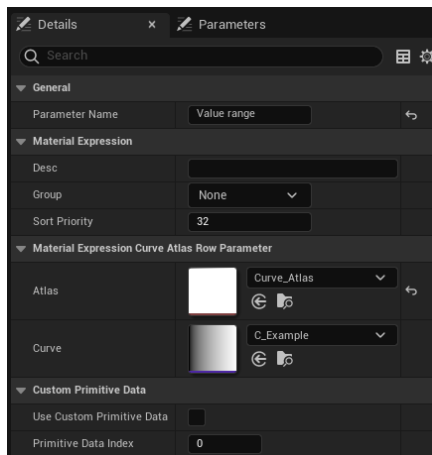


Figure 43: Giving default values to the CurveAtlasRowParameter

All we need to do to remap the values from the extracted light, is plug that info into this parameter and the node will take care of the rest. If you output this result, you will see the scene as if it is photo realistically rendered. That is because our curve is a gradient and thus the values of the light are remapped to this gradient. For a cel shader we don't want it to interpolate between this many values, we want fewer and clear divisions between values.

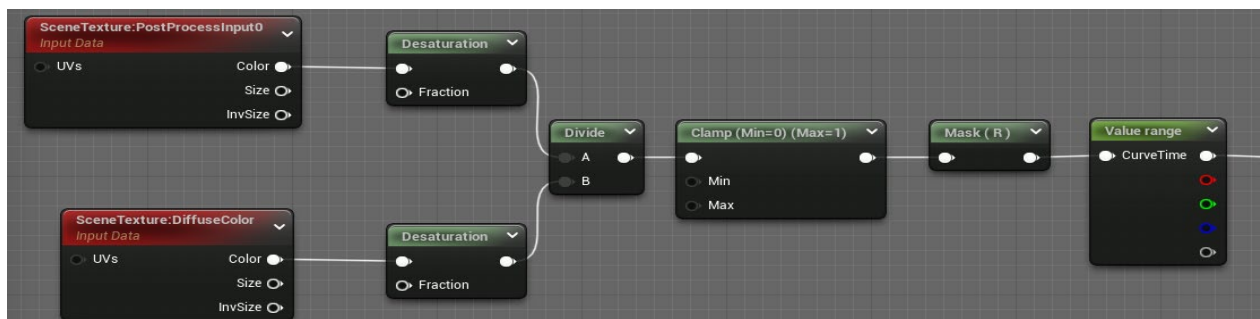


Figure 44: Remapping the light value to the values of the curve

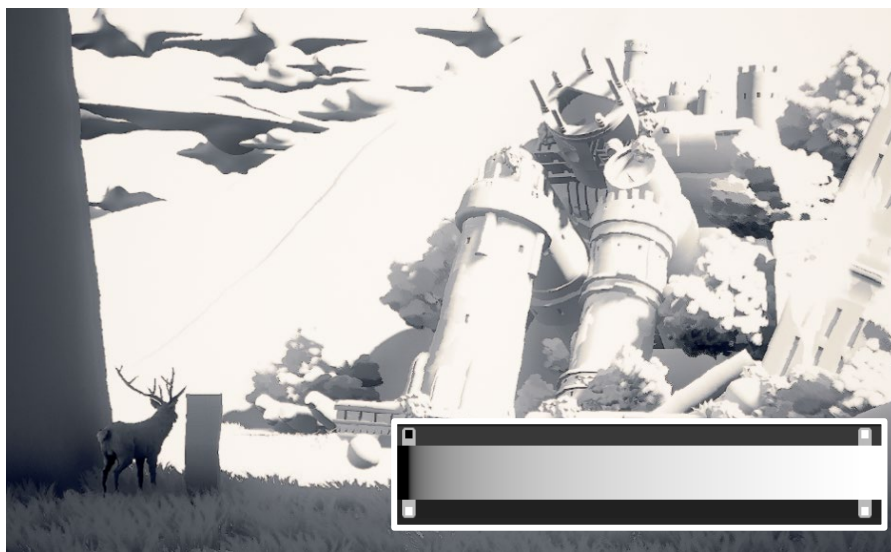


Figure 45: Remapping the light values with a gradient curve

Let's create another curve to make the cel shader. It needs to be added to the atlas. Then, in the instance of the material, you can choose this new curve to be used. In this curve, we will mainly work with the color result graph. If you double click above the graph, you can add in a color key. If you double click underneath, you can add in an opacity key, though we won't be using this. Add a few more versions of gray colors in the color graph. For example, for light value 0 to 0,2 we want it to render completely black. From 0,2 to 0,8 we want it to render one version of gray and from 0,8 onwards we want it to render white. This will create a 2 band cel shader. You can make more bands by adding in more color keys. The closer or further apart you put these values, the smaller or broader a band.



Figure 46: Remapping the light values using keys in a curve to create bands

We can then use these remapped values to multiply on top of our base color and thus add the cel shader on top of our scene.

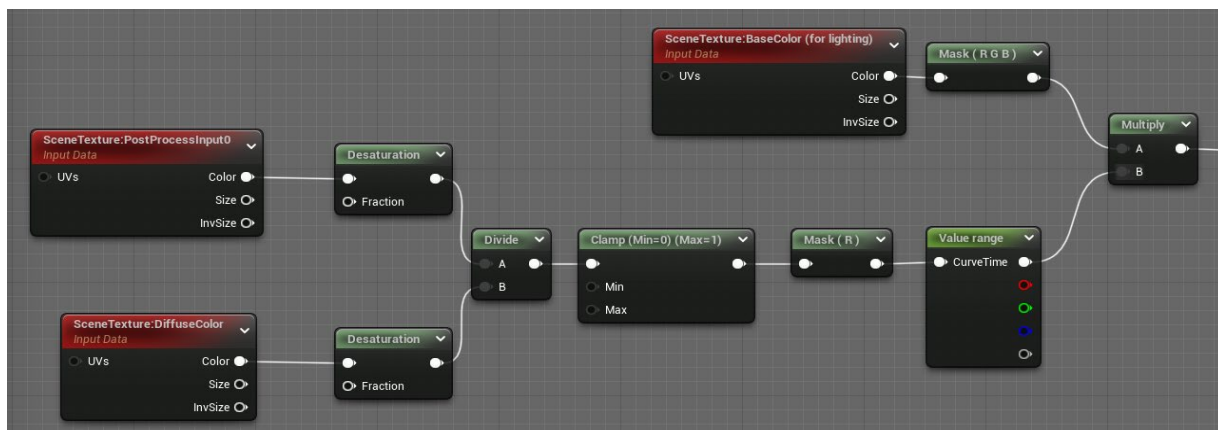


Figure 47: The cel shader with curves in nodes



Figure 48: The intermediate result of this cel shader

You're not limited to black and white. You can use any kind of color for the cel shader. If you want different values of shadows, you just have to make sure the values of the colors fit. In the material instance of the post-processing material we are making, you can double click on the curve being used. You can then directly adjust this curve and immediately see the changes in the viewport. Let's add some color to the curve.

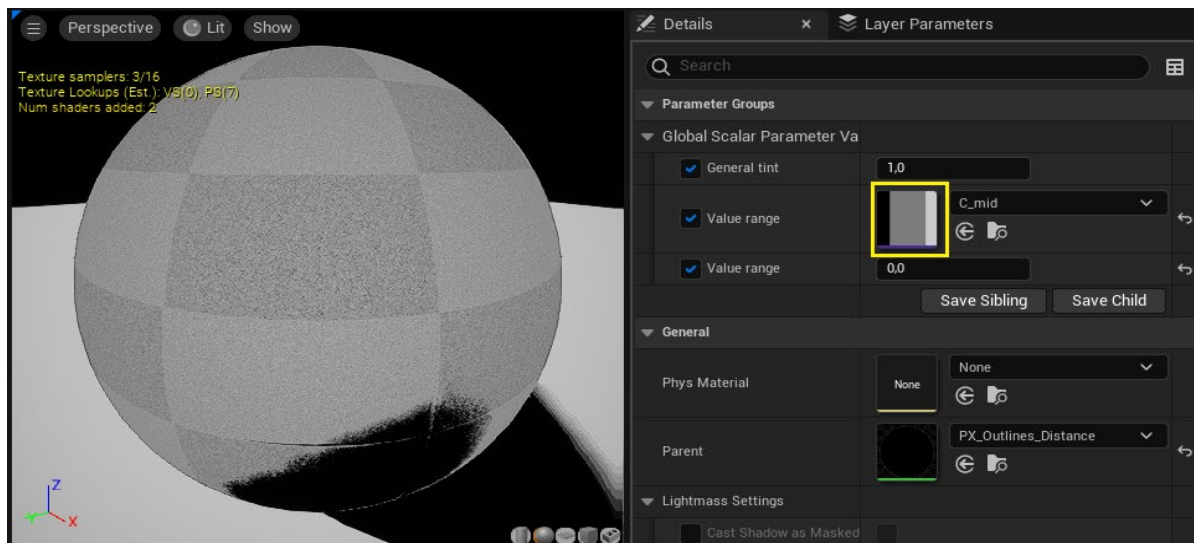


Figure 49: The instance of this cel shader through which curves can be adjusted

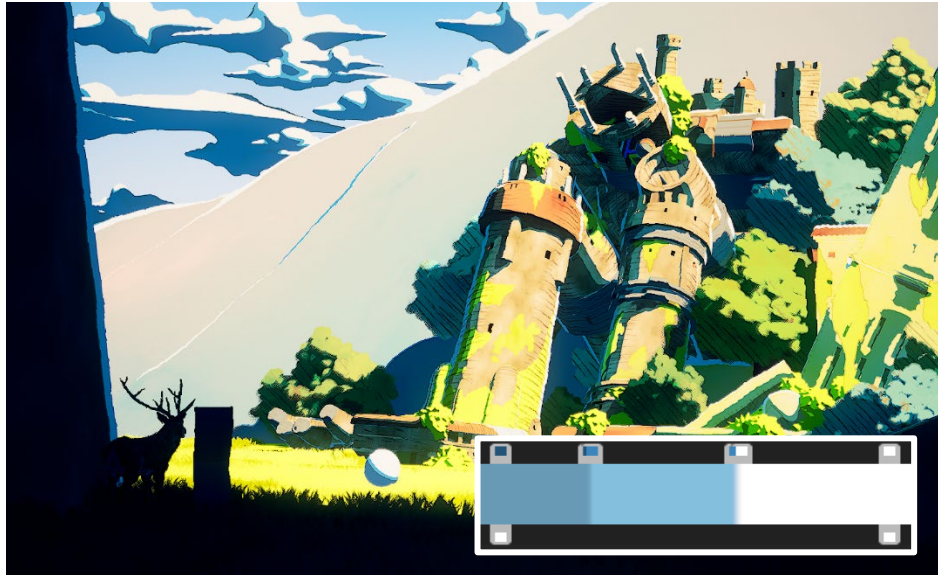


Figure 50: Adding colors to the curve

If you find it a bit too dark, you can either use lighter color values for the curve, adjust the brightness for all keys in the color panel of the curve or in the material multiply the node by a scalar so the user has control over it in the material instance.

2.5.6 ADDING IN AMBIENT OCCLUSION

To add in the post-processing ambient occlusion the cel shader needs to be multiplied with the SceneTexture:AmbientOcclusion. That's really all there is to it. Then, the AO settings within the post-processing volume will work again ([PrismaticaDev, 2020](#)).

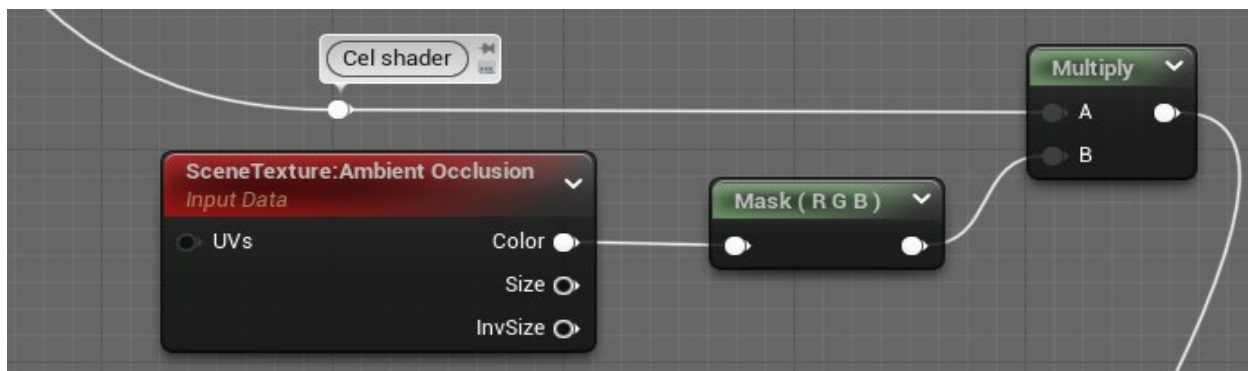


Figure 51: Adding AO to the shader

2.5.7 ADDING IN LIGHT COLOR

As mentioned earlier, extracting the color of lights used in the scene can be a tricky for cel shading and is often where a basic cel shader falls short. If you don't want to deep dive into scripting, then the following method can offer a global node-based workaround for this. The method is not perfect and is best to be implemented early in a project so all texture colors can be coordinated with this effect since it does alter colors a bit. Because of this reason, light

color has been left out for the final scene. Another reason is that this method then discards the color of the different bands created in previous versions because, to take light color into account, it will use the scene color instead.

To add light color to the scene, we take our final unaltered render from the scene and condense it to just its color information. The reason we can't use diffuse or base color is because this is an unlit version of the scene with only the color information of each mesh. However, by neutralizing the complete unaltered render, which holds all lighting information, including the light color, we can create a somewhat similar diffuse version of the scene. This is done by normalizing the SceneTexture:PostProcessingInput0. We can then correct this normalized version a bit by multiplying it by a certain number, like 1. You could make this a scalar if you want to influence it in instances. To then get our previous cel shader back on top of this diffuse color, we need to desaturate the result of the cel shader and multiply it by the diffuse of the scene. This is the inverse version of our light extraction we've been using (dividing the unaltered scene with the diffuse color): adding the light information (now cel shaded) back on top of the diffuse version ([PrismaticaDev, 2020](#)).

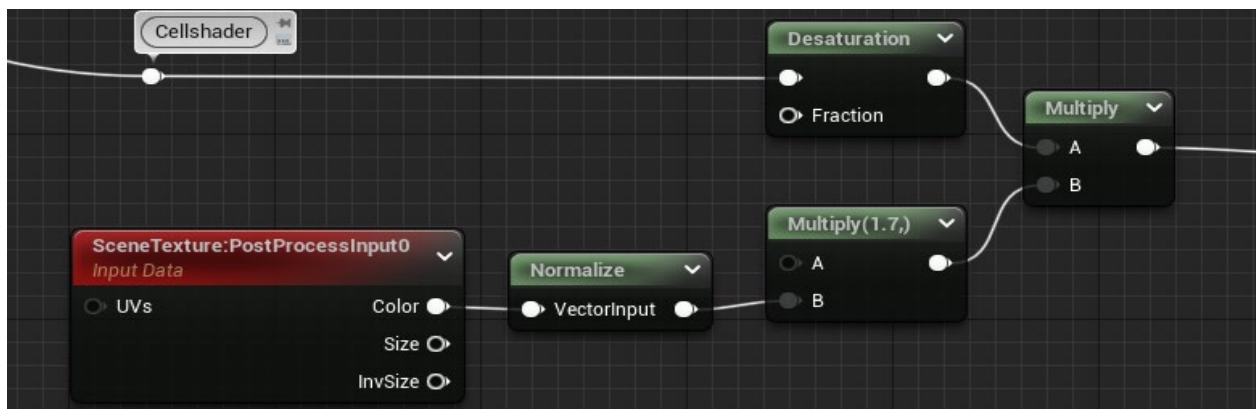


Figure 52: Adding scene color to support light color to the cel shader

2.5.8 BLENDING IT INTO OUR EXISTING NODES

To blend our cel shader with our previously created outlines, we will go back to the lerp that blended our lines on top of our unaltered scene. Instead of blending it with our unaltered scene, we now want it to blend with our cel shader. I've added a Boolean (StaticSwitchParameter) in between so the user can choose whether they want the lines on the unaltered scene or the cel shaded one.

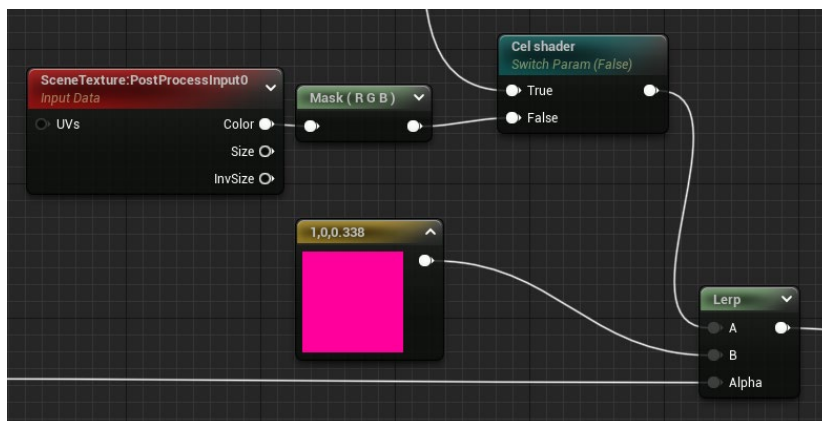


Figure 53: Blending in the cel shader with the previous nodes

I also implemented an option that allows the user to choose between blending the cel shader on top of the diffuse or blending it on top of a flat background color. Since my shader uses the version with the if nodes, we need to adjust it a bit to do that. Between the ComponentMask of the SceneTexture:DiffuseColor and the multiply, we need to insert another Boolean (StaticSwitchParameter). This one will decide whether to output the diffuse color of the scene or a flat background color on which the bands get multiplied.

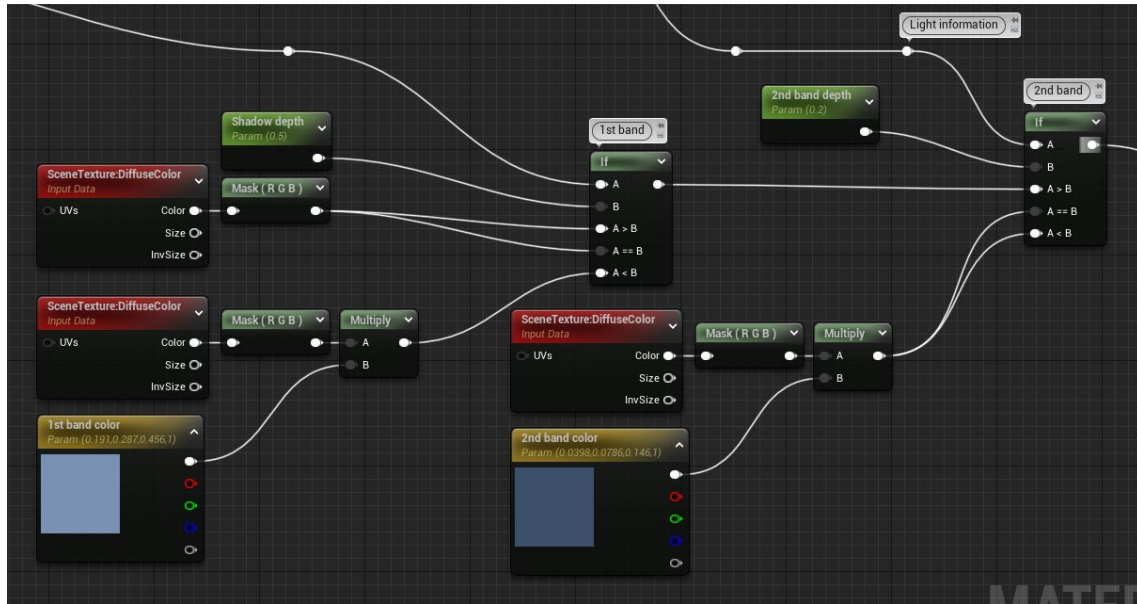


Figure 54: Previous cel shader

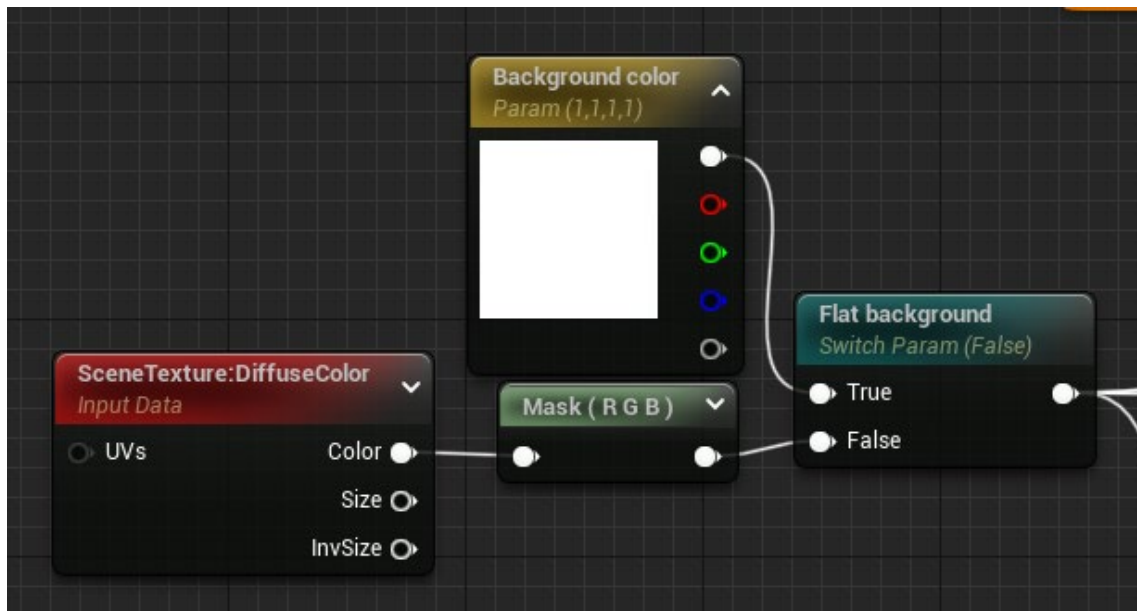


Figure 55: Adding in the switch between the scene texture and the multiplies

2.6 HATCHING SHADER

The next key characteristic of our style is that it uses hatching as a part of its shading. For this project, we will use both a surface material based hatching shader (for texturing) and a post-processing material based hatching shader. In this part I will discuss how a global post-processing one was made, how it can hold up in movement by aligning it to the world and how to mask out an object with custom depth and custom stencils so the effect only applies to that object. This global post-processing shader isn't made to match the art style, it's made to add to the variety of 2D styles you can achieve with the shader. I will discuss the one matching the art style in the chapter on the layered materials.

The logic of the hatching shader is almost the same as the logic of the cel shader. Where the cel shader has different bands of shadows, the hatching shader has different bands of hatching. They're essentially made in the same way: using the light information against thresholds of if nodes or curves. Since most of this logic has been explained in the past chapter, this won't discuss this in depth.

Later in the project, I will discuss how you can make your own hatching textures and how you can adjust them procedurally in-engine to your liking should you want to. For now I am using the hatching textures by [Kamil Hepner](#).

2.6.1 USING AN IF NODE FOR THE BANDS

The first way to achieve this, is with an if node. This time we will create our shader and bands as if they're masks. In the end, we want to output a black and white version of the hatching shader: white for the lines that need to stay, black for the elements that need to be masked out. This will then be put into a lerp node which will interpolate from our scene or chosen background color to the color of the hatching. You don't have to create the shader as a mask. You could just as well return your scene instead of returning a black pixel and lerp your hatching bands with the cel shader bands. I chose to go for the mask approach to avoid a chaotic node structure and because I don't need any further per hatch-band control.

The calculation to extract the light information is needed again. Then a scalar is added for the first threshold, for example, 0.7. To create the mask, the following logic goes: If a light value is bigger than 0,7, we want it to render the pixel black so it can be used as a mask. If the value is smaller than 0,7, we want it to render our black and white hatching texture. If your texture has a white background and black lines, you'll need to invert it first so the lines stay white and won't be masked out. You can do that with a One Minus node. If your texture has a black background and white lines, you don't need to do this. You could build in a static switch parameter if you want to have the option to invert textures. I added a scalar for tiling and a scalar to adjust the rotation of the texture. If you output this result, you will see the mask we created.



Figure 57: The hatching shader mask

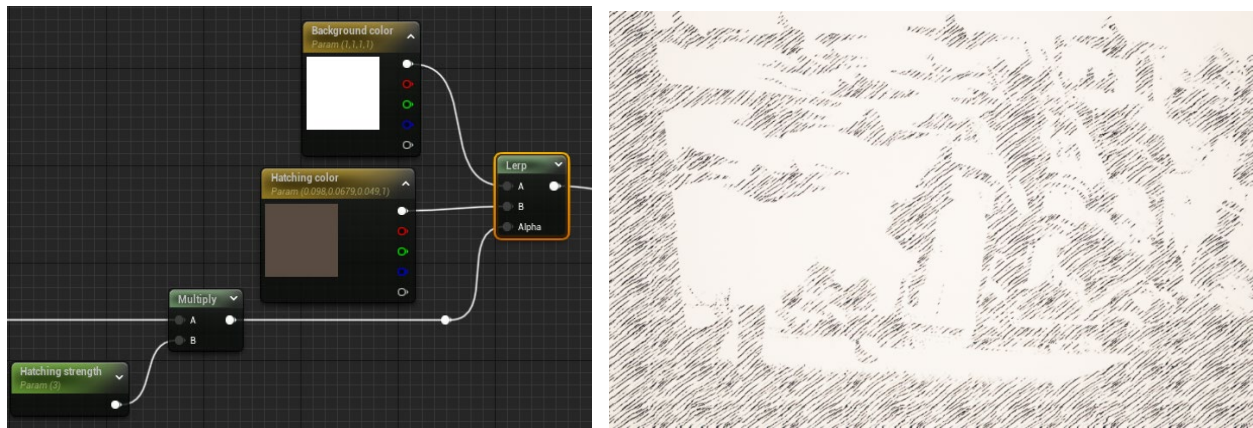


Figure 58: Blending the hatching on top of a flat color

Adding in another band requires the same workflow, but with every band a new threshold and hatching texture is needed.

A downside to using an if node to make the different bands is that the divisions are quite clear and harsh and thus the lines can stop abruptly instead of fading out a bit. Making a band softer is also harder to do since the mask created isn't a texture but values per pixel. It's harder to blur these. This can be fixed by using the curves method. However, with this method you would need a curve for every new band you want in the hatching shader. This method was discovered from [Kamil Hepner](#) and I would like to refer to his tutorials if you are interested in this workflow or a more in depth explanation on a hatching shader.

2.6.2 CAMERA ALIGNED VS WORLD ALIGNED

When panning, rotating or moving around in the viewport, it's very clear that the texture is static and doesn't move along. It's as if it's a texture on top of the camera lens. For a single 2D image, this could work and could be an aesthetic to go for. However, in a game in which you move about in the world, this could break the 2D look for the player and on top of that, feel very hard on the eyes.

There are two different methods that could fix this.

- We could triplanar map the texture onto the camera with a TriplanarCameraVector node. Then, the texture will seem as if it extends when you pan and rotate around in the scene. This will go a long way to fix the issue. However, it is still not completely there as objects up close will keep the same tiling as objects in the distance.

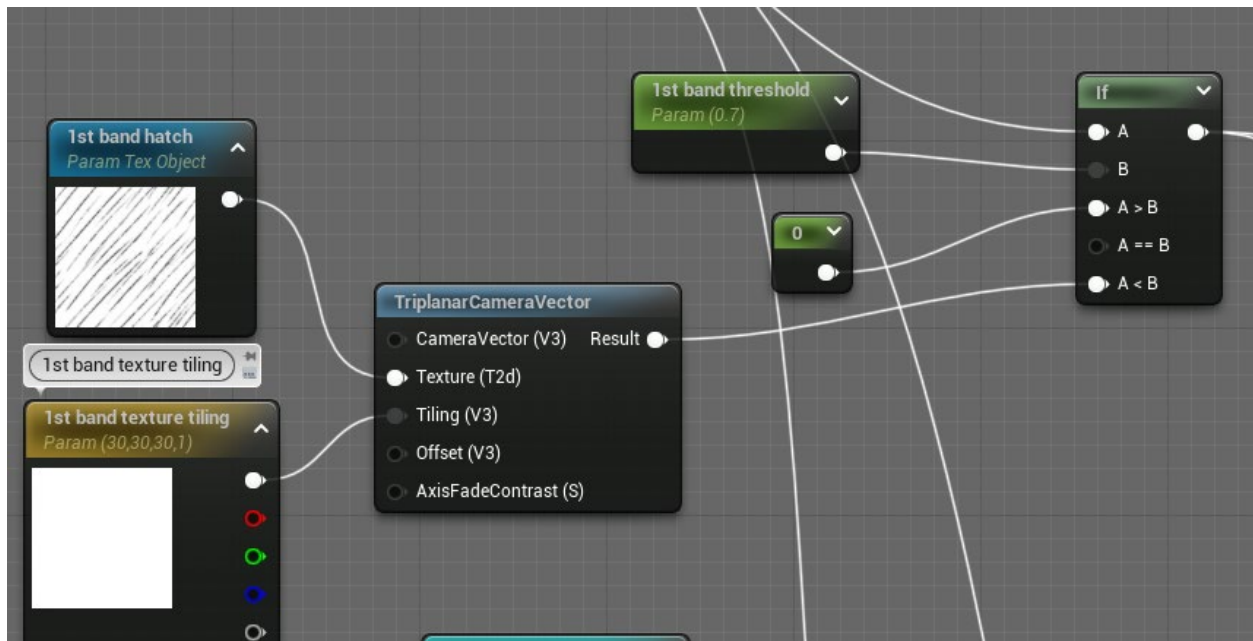


Figure 59: Camera aligned hatching

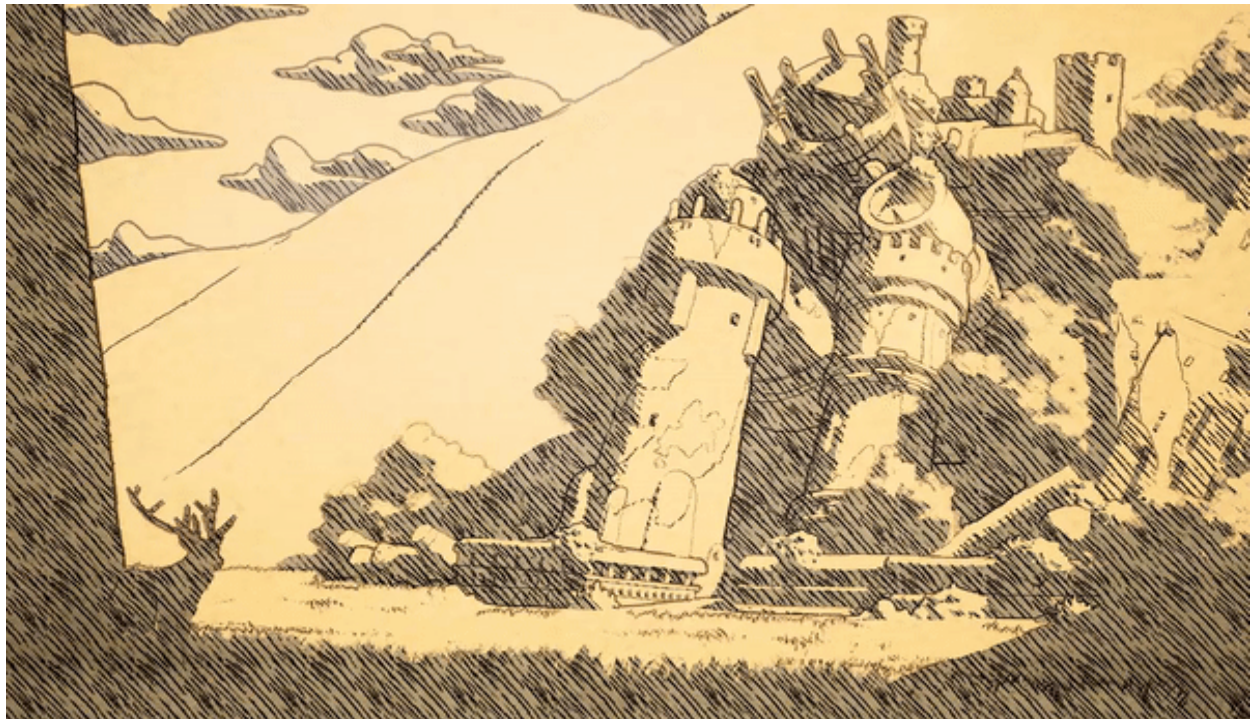


Figure 60: Camera aligned hatching in action

- Another option is to map the texture onto the world. This will wrap the texture around all the objects. Then it will also feel as if the hatching is used to accentuate the volumes of an object. For this, we use the WorldAlignedTexture node. For the final scene we're using a combination of both to get it to look like the concept.

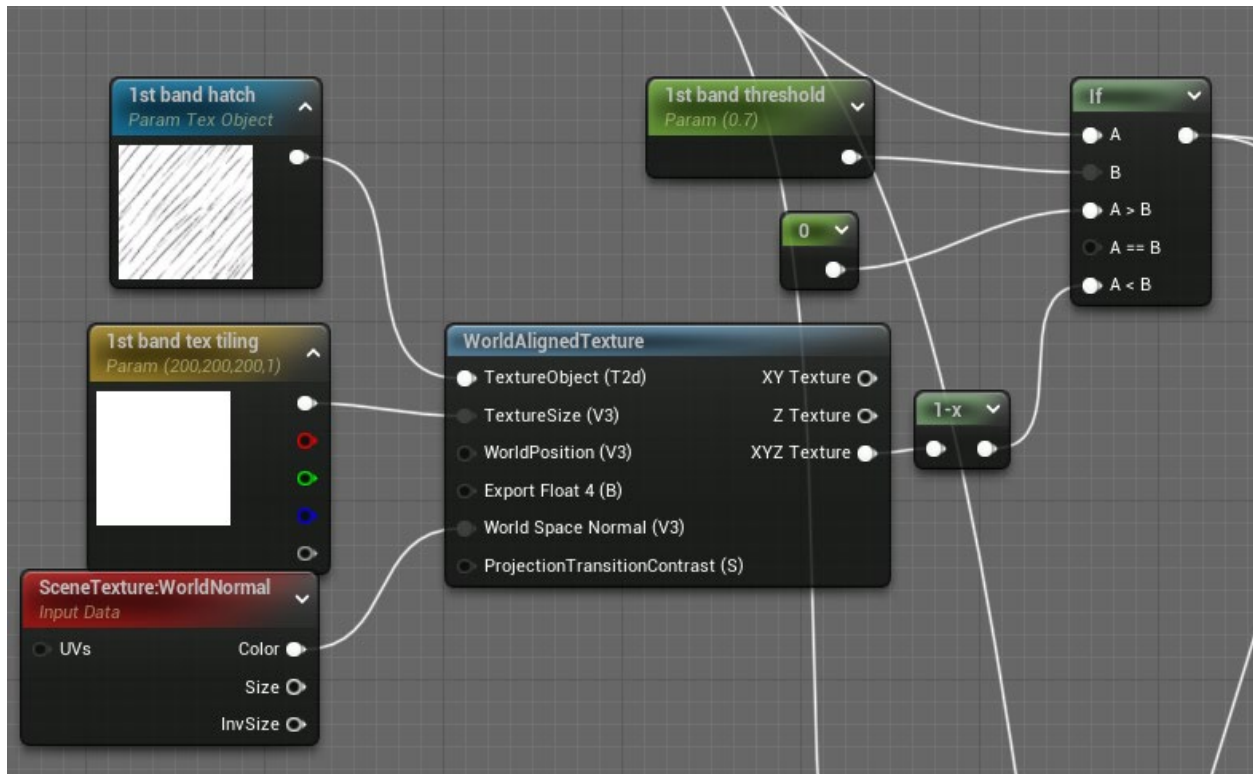


Figure 61: World aligned hatching

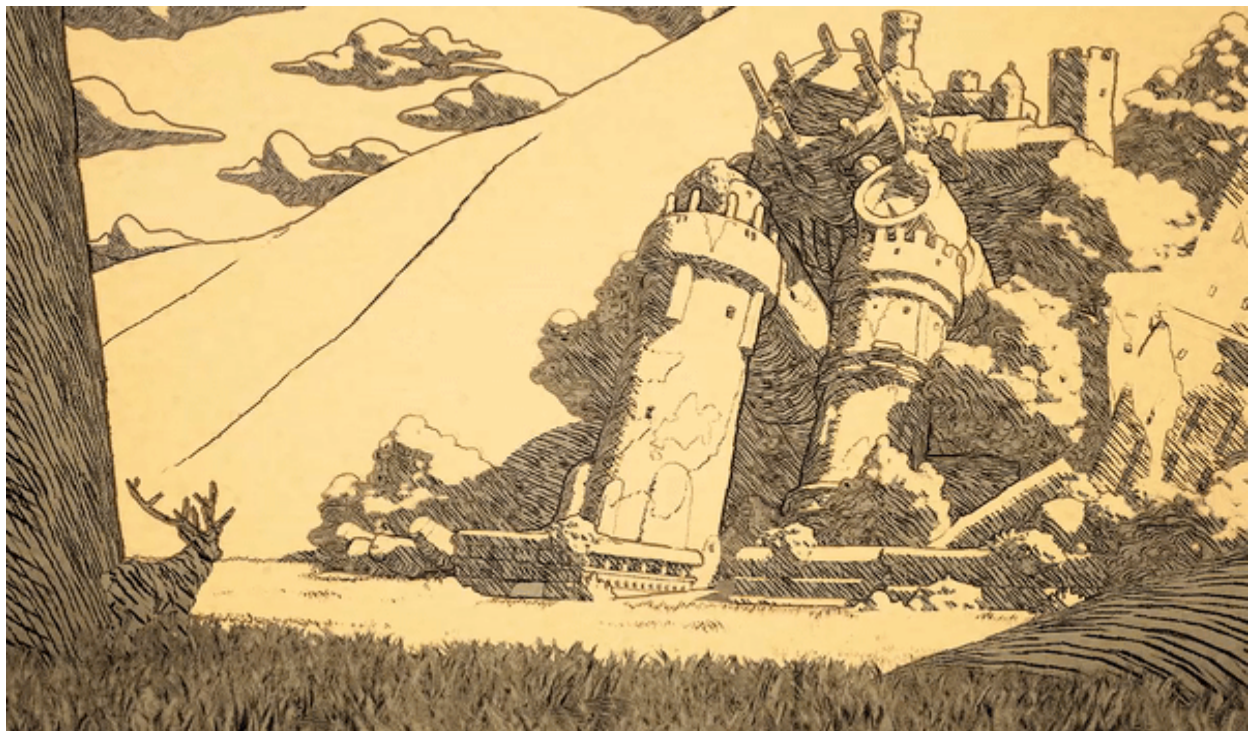


Figure 61: World aligned hatching in action

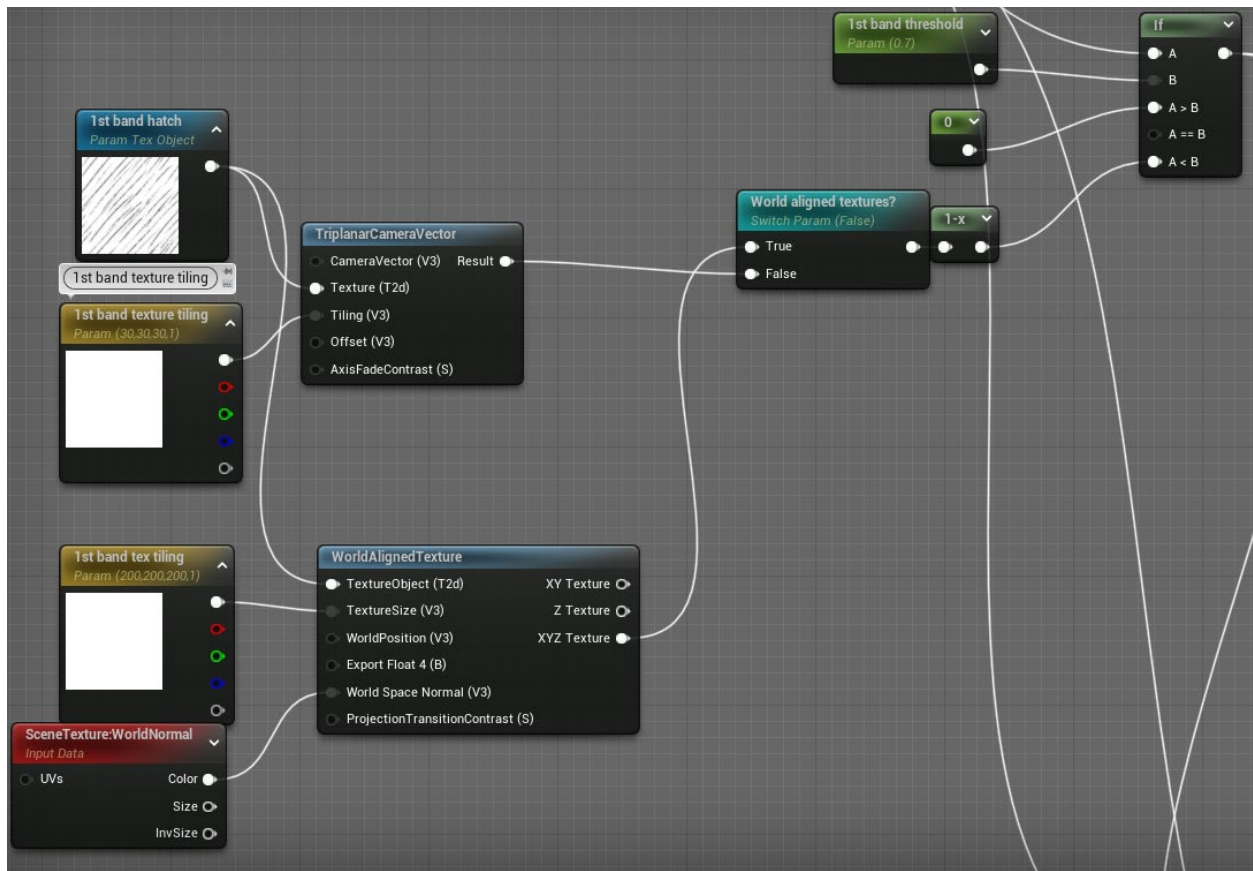


Figure 63: Giving the user the option for camera vs world aligned hatching

I decided to have both options available for the user. With a Boolean the user can then choose which one they want.

2.7 FOG AND DISTANCE FADE

Because the chosen cel shader for the scene didn't have the scene color in it (primarily used to implement light color), it doesn't work well anymore with any kind of fog volume. Because fog can be important to have for a user, I added in the possibility for custom fog through the shader. For this, a tutorial for depth fog by [Tom Looman](#) was followed. The shader takes the scene depth. By adding a scalar to the Scene Depth, we can exaggerate its effect and make objects closer appear further away. This can then be used as mask to lerp the final scene with a color for the fog.

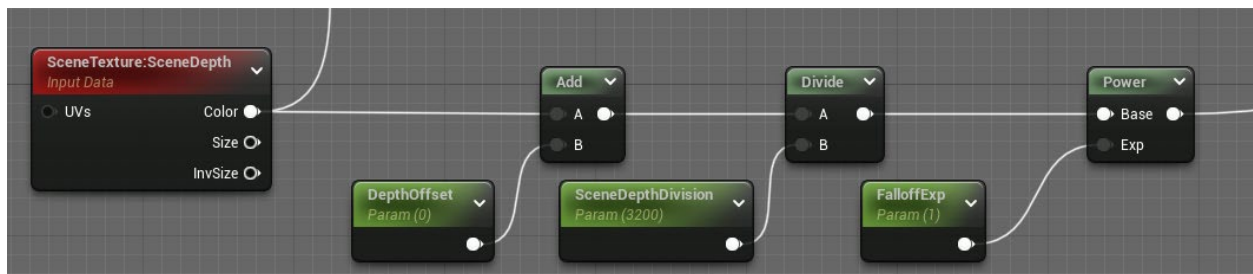


Figure 64: Custom fog

2.8 PAPER OVERLAY

The paper overlay was the last step in the shader and was done through multiplying the last node of the previous calculation with a paper texture in a TriplanarCameraVector node.

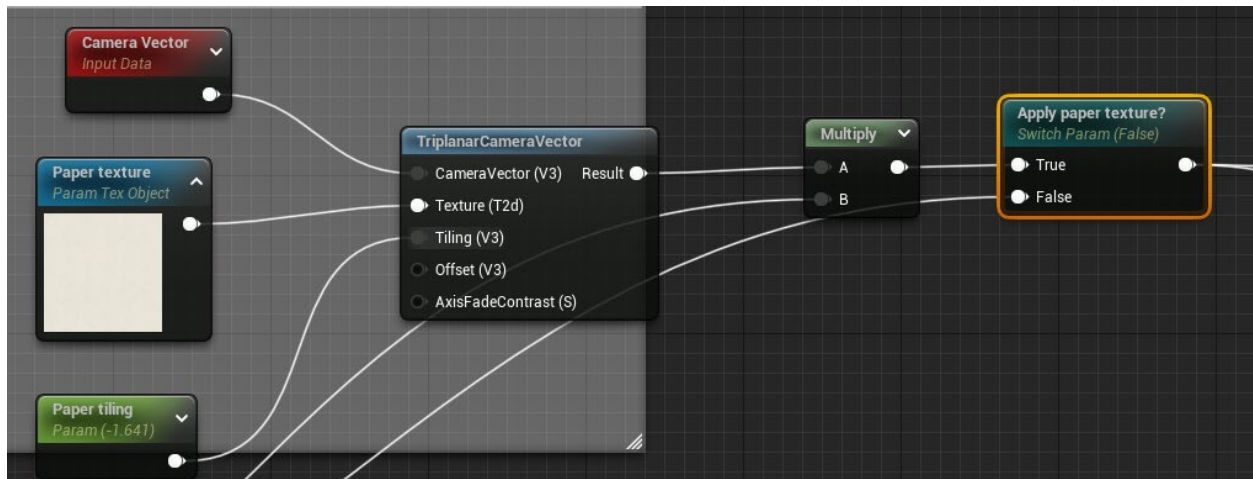


Figure 65: Paper overlay

2.9 MASKING OBJECTS WITH CUSTOM DEPTH AND CUSTOM STENCILS

While post-processing materials apply their effect globally, it's also possible to have their effect applied locally. This can be done through the usage of the [custom depth](#) and [custom stencils](#), which can be enabled in the details tab of each mesh in the world outliner.

The custom depth works in a similar way the scene depth does, except that, when enabled in an object, it renders out the object in a separate pass so it can be masked out from the scene. Because it can do this, separate effects and calculations can be done with it. Custom stencils can then be used to assign different post-processing materials to meshes that have custom depth enabled. The limit to the amount of custom stencils is 255 (including 0, which is the default for every mesh). This means that you can have 255 separate post-processing materials to assign to objects. The custom depth is often used to give an outline to a character and have that outline still be visible when the character is behind, for example, a building.

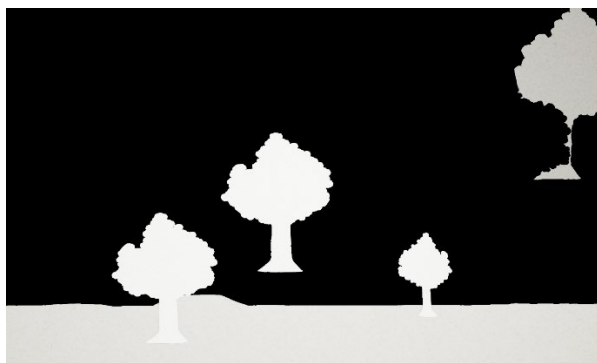


Figure 66: An example of the custom depth

To make a local post-processing effect, an if node can be used on the SceneTexture:CustomStencil node. This will return objects that have custom depth enabled and have a custom stencil value higher than 0. All values used are then returned and displayed. With an if node we can, for example, check: if one of the custom stencils returned has a value of 4, apply this post-processing effect. If it has a value higher or lower than the number we specify (the number we want this post-processing effect to have), it can return the scene as is.

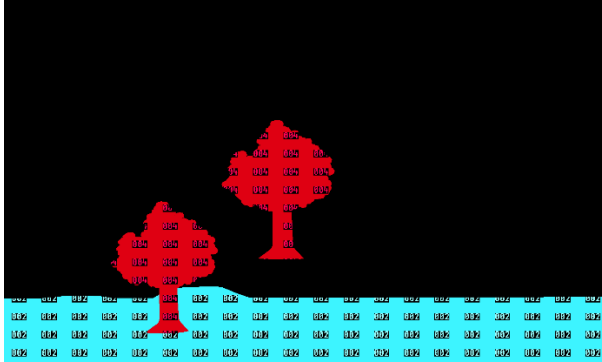


Figure 67: An example of the different custom stencils used in the scene.

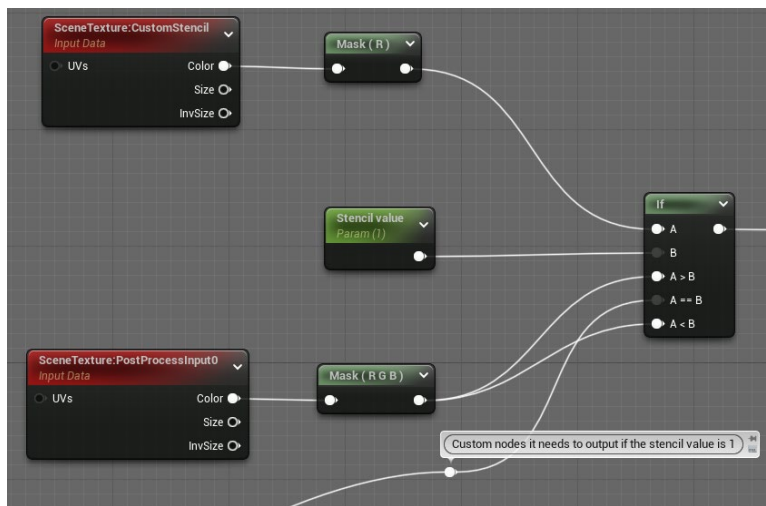


Figure 68: How a custom stencil value can be assigned to a post-processing material

While this is a great way to mask out objects and have effects applied locally, it does have some flaws. If the post-processing is using outlines, then the custom depth and stencils will mask these out. Adding in extra pixels to these masks is necessary to be able to show the lines on a masked out object. Another one is that, when having multiple objects with custom depth enabled, these get put together in the same mask. When these objects overlap, the scene depth won't be able to differentiate the two and thus won't draw a line where they overlap. That is a flaw that is clearly visible in my scene. This might be fixed by adding another value check on the scene depth but as this proved to be tricky, fixing this issue was abandoned to focus on the elements still needed to be implemented.

For a while I explored recreating the concept as a singular 2D image and then using the post-processing materials to texture the whole scene in a flat way as if it was on paper. However, this method was abandoned as it would've resulted in a huge amount of stencils, each added separately in the Post-Processing materials stack. It would then take a lot longer to render each frame which is especially problematic in a game setting. Apart from that, because of the amount of stencils, it would've been a pain for the user to work with.

It's important to note that I do still work with custom depth for my trees and grass so I can only render an outline on the silhouette and not on the inner "leaves". This is done by subtracting the outline detection with the custom stencils.

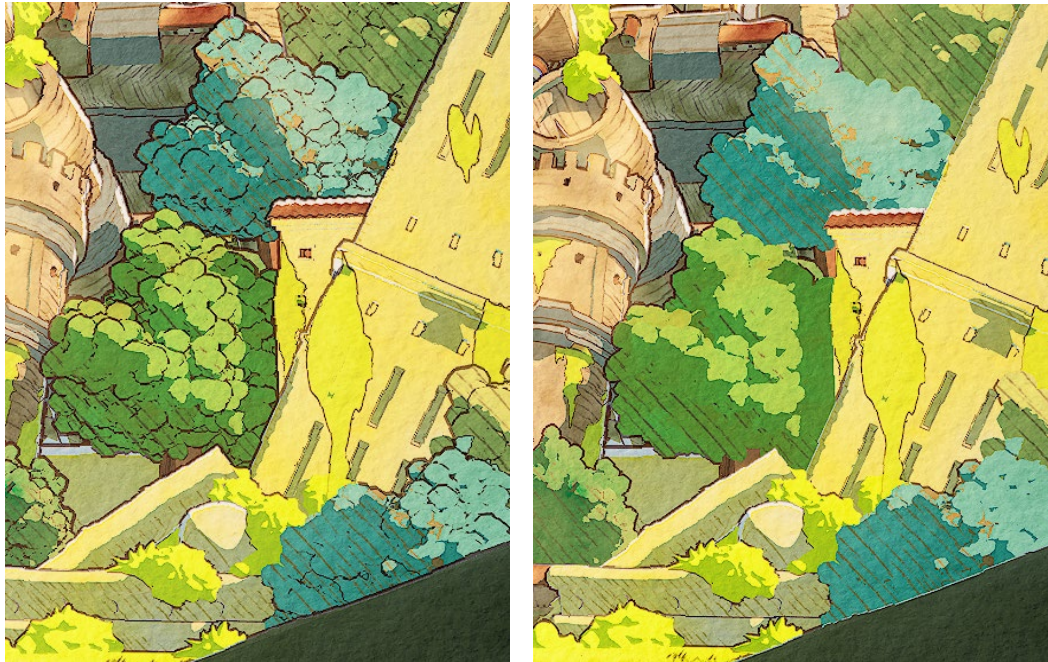


Figure 69: Trees with and without inner lines

2. AUTOMATING TEXTURING USING LAYERED MATERIALS

2.1 CREATING A LAYERED MATERIAL

The creation of a master layered material is done through creating a normal material and enabling "Use Material Attributes" in its material properties. Then, a "MaterialAttributeLayers" node needs to be made. This node will tell the material to create and use the stack interface. The nodes "GetMaterialAttributes" and "SetMaterialAttributes" will get all the material properties from the layers used in the stack and set it as the properties of the instances of this base material. When creating an instance of this material, the UI "Layer Parameters" will then be visible and ready to use.

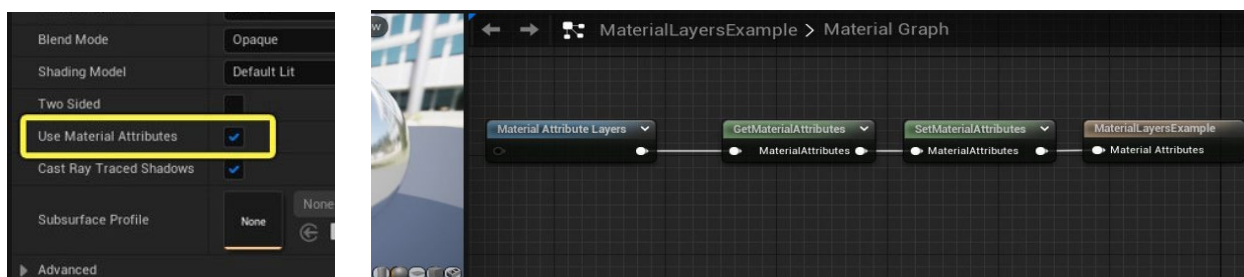


Figure 70: The setting that needs to be enabled to create a layered material system + the node structure to create a layered material

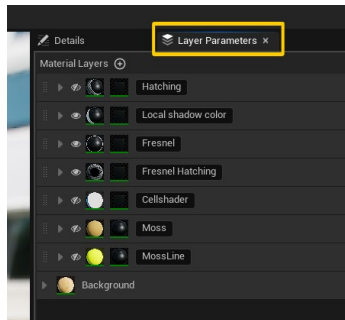


Figure 71: The UI in instances of the material

2.2 CREATING MATERIAL LAYERS AND BLENDS

The creation of a Material Layer Asset and Material Layer Blends asset can be done through Add New > Materials > Material Layer (Blend).

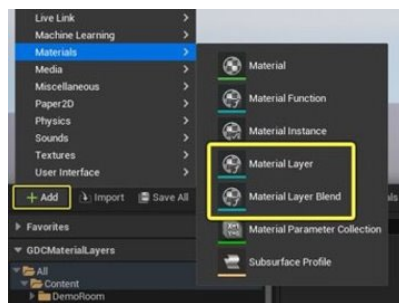


Figure 72: How to create material layers and material blends

Once in a Material Layer, a “MakeMaterialAttributes” needs to be added to get access to the different material properties. Then, you’d set up your textures and parameters like you would in a normal base material.

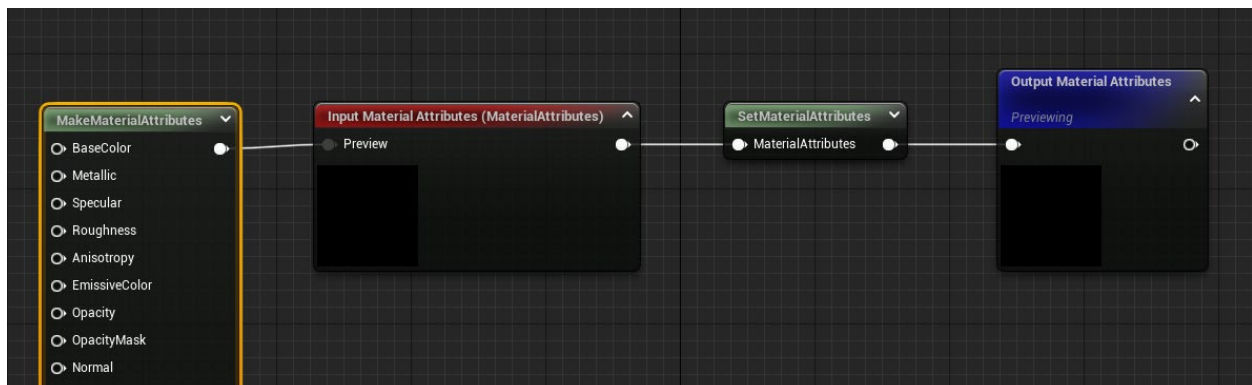


Figure 73: A typical material layer

A Material Layer Blend asset is then used to blend the current layer on top of the layer underneath. This is typically done with an opacity scalar or a mask but you can also create a custom blend between the two layers. A blend asset can be as complex and intricate as you want it to be. It’s also possible to blend layers on top of each other using different blend modes like the ones in Photoshop. These can be accessed by typing “Blend” in the function library.

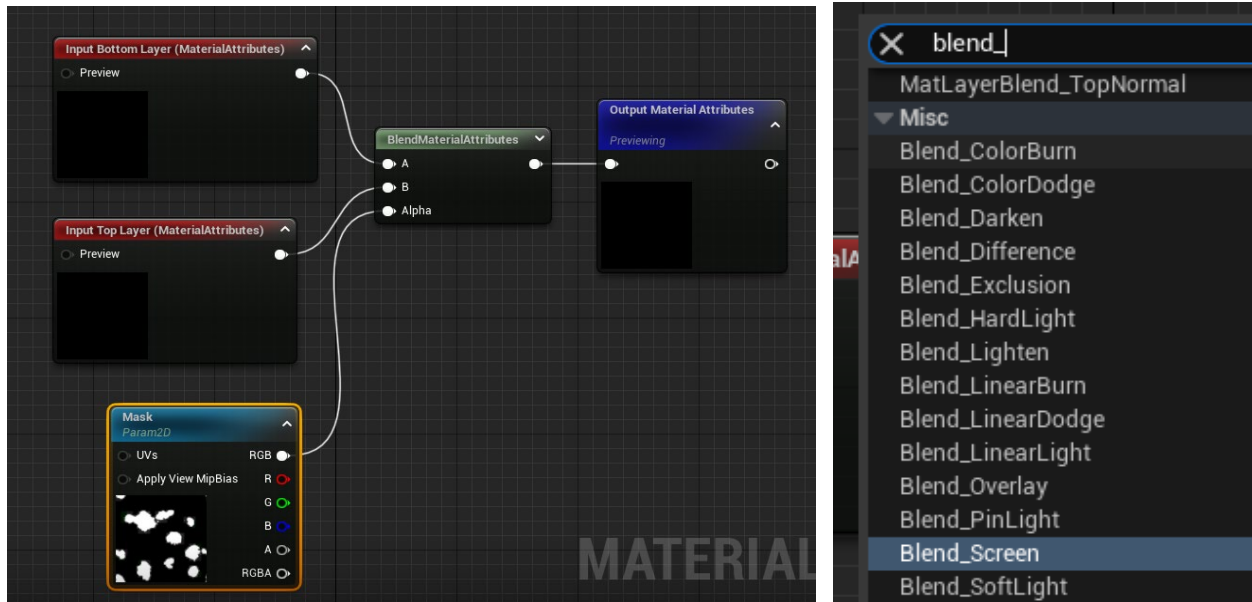


Figure 74: A typical blend asset – the blend modes of Photoshop

2.3 SCENE LAYERS AND BLENDS

2.3.1 WATERCOLOR BASE

This material layer will be the base for every objects material instance. Since it's the base, it will be the background layer in the Layered Material System on which all other layers will be blended. we think in terms of the The Cycle, this base material would consist of the clean base textures of an asset that came out of Substance Painter. The wear and grime is then stacked and blended on top of this base. Because this is the base, this doesn't need to be blended with a blend asset.

This watercolor base is made so that the user can insert any watercolor mask to lerp between two chosen main colors. The mask is tiled across the worlds surface. Because of that, objects don't need to be unwrapped. The specular is set to 0 and the roughness to 1 to nullify light data on textures and flatten out the general look of the object to enhance the 2D feeling.

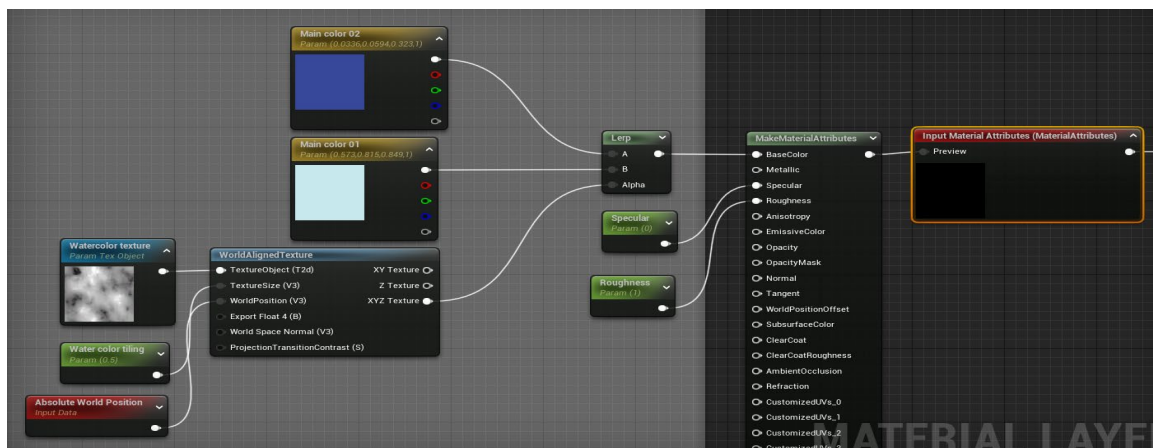


Figure 75: The watercolor material layer which serves as the background layer in every material instance

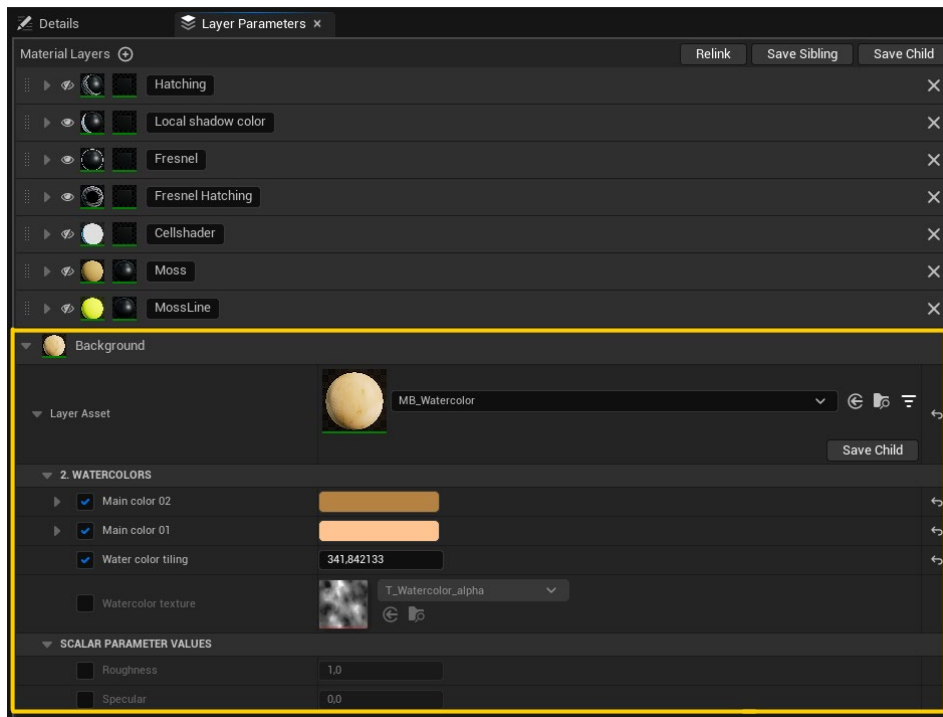


Figure 76: The background of a layered material

2.3.2 FRESNEL RIMLIGHT + FRESNEL HATCHING

To create the white rimlight from the concept I used the Fresnel node to emulate the effect. Because a Fresnel node cannot be used in a blend asset, I made the material layer as if it would output a mask through the base color. It's worth noting that in almost all the layers I only work with the base-color since any other properties weren't needed for the concept.

Then, in a new blend asset I implemented a custom way of blending: it takes the base color of the current layer as the mask to lerp between the layer underneath and a color for the white areas of the mask. To access the material properties of the bottom or top layer in a blend asset, a "BreakMaterialAttributes" node can be used. After the lerp, the current layer needs to be set again with a "SetMaterialAttributes". In here I return the specular and roughness of the bottom layer so they can be used as the properties of the current layer. If you don't return any properties, it will simply use the properties of the top layer. This way, the blend will keep the specularity of 0 and roughness of 1 from the bottom layer.

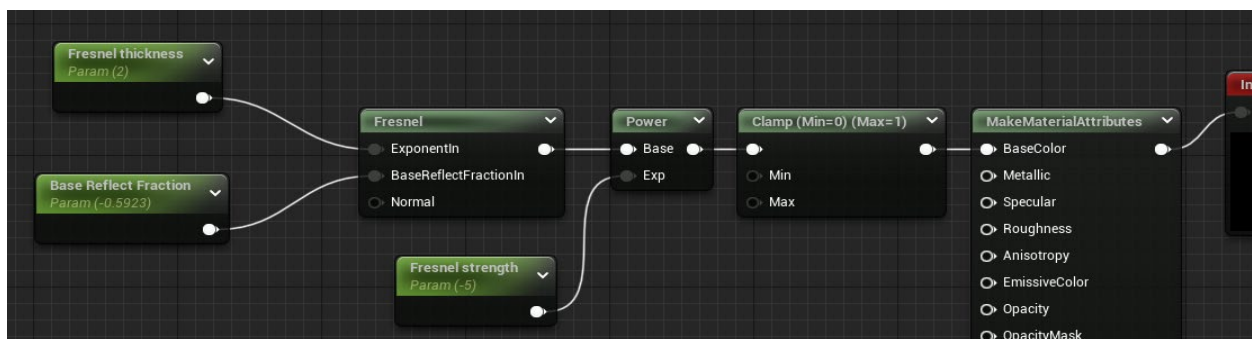


Figure 77: The Fresnel Layer – ML_Fresnel

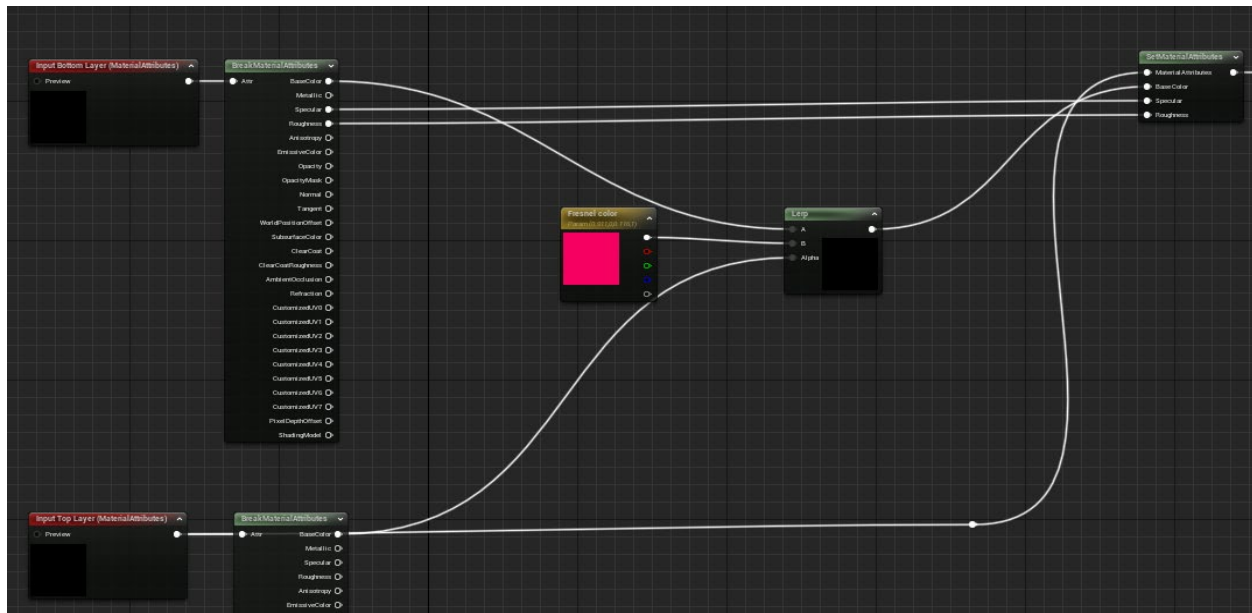


Figure 78: The Fresnel Blend – MB_Fresnel

To create the hatching on the edges of the legs, I used a Fresnel to mask out everything but the edges of the object the texture is projected on. I added in some tiling and rotating parameters for the texture. This one is not mapped to the world, meaning it relies on UVs. A version that is mapped to the world can also be created. The MB_Fresnel

asset that was just created can be reused to blend this layer on top of the previous one. In the concept, the hatching is always underneath the rimlight. We can swap around the order of the layers to get this right.

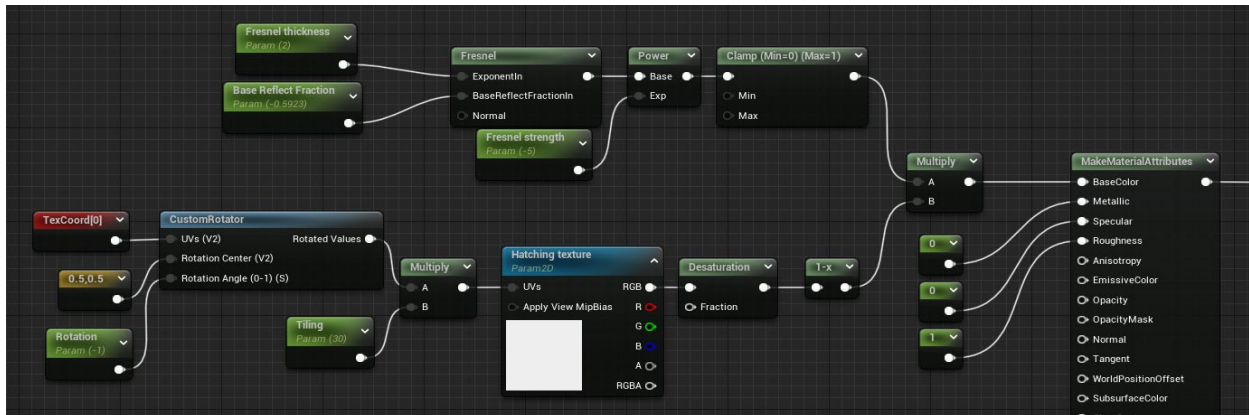


Figure 79: ML_FresnelHatching

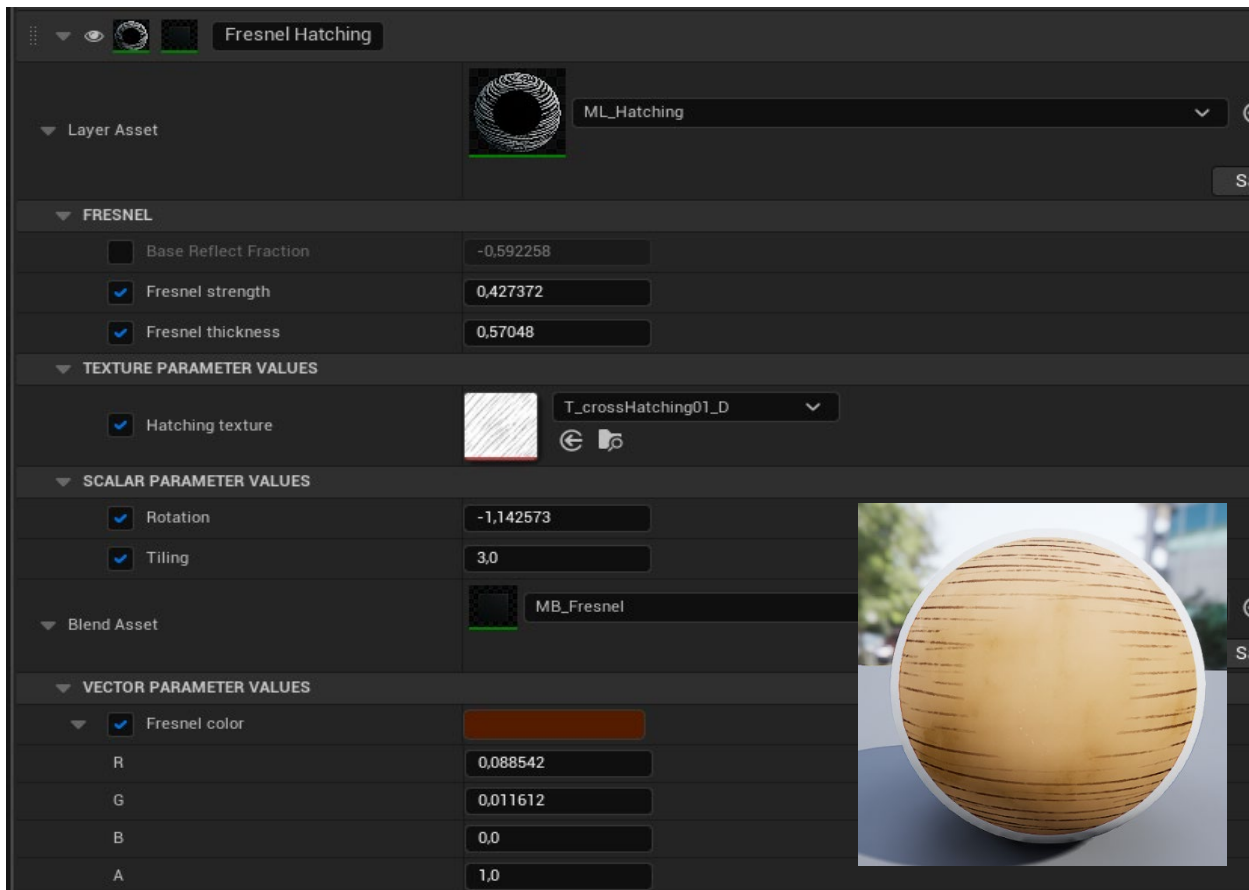


Figure 80: The parameters available with the ML_FresnelHatching

2.3.3 CEL HATCHING

Since some objects have hatching in their shadow areas, I made another Layer that uses the cel shader bands as a mask to only display hatching in those areas. This is done with the dot-product. While I initially mentioned this

method has flaws, and while these are visible, since the effect is used very locally it's not a big problem. I added in a Boolean to invert the band since there are also objects who have their hatching in the lit areas. I added in a Boolean to map it to the world or the camera. I expanded the layer so it could be used for the hatching of the trees as well. It has another cel band for the dividing line between hatching and lit area of the trees in the concept. Since this outputs a mask through the base color, the Fresnel blend can be used once again.

Since these hatching textures are mapped to the world with a WorldAlignedTexture, it uses texture objects instead of texture samplers. Along the way I wanted to rotate the hatching texture. Since this is not possible with a texture object and since we're putting that object in a WorldAlignedTexture/TriplanarCameraVector node, we can adjust those nodes. Those nodes are actually material functions you can go into. So, between the component masks and the texture samplers, a customRotator was inserted. A FunctionInput was added that gives the possibility to pass along values for the angle from the master material to the function. That value is divided by 360 because the CustomRotator wants the angle mapped between 0 and 1.

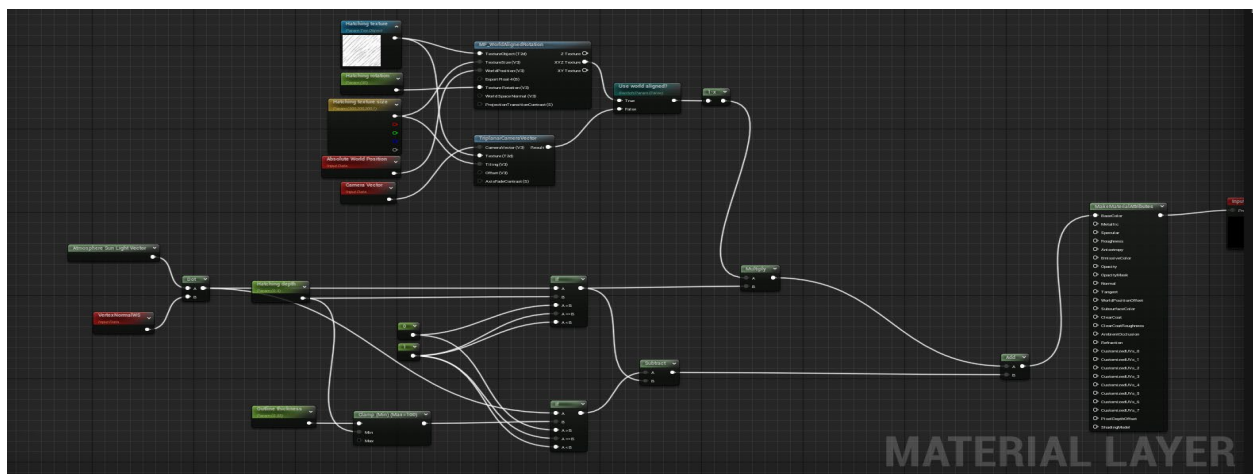


Figure 81: ML_Hatching

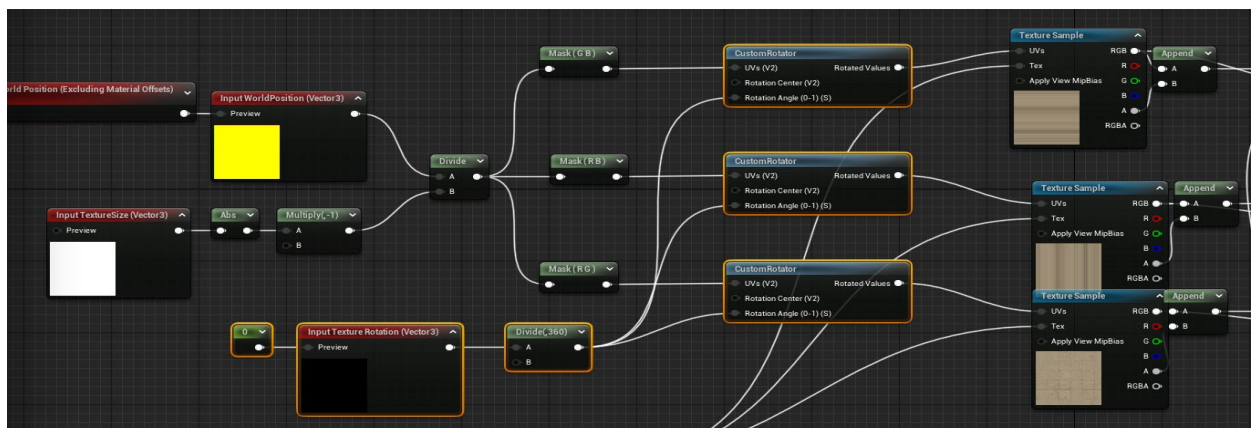


Figure 82: Adding rotation to the WorldAlignedTexture function

2.3.4 LOCAL CEL SHADER

The scene has a global cel shader through the post-processing volume and thus sets the shadow color for the whole scene. But because this is global, it falls short in certain areas. Especially on the robot's skin. The shadow areas here are a bit colder and that doesn't come through with the post-processing shader. While this can be adjusted in that shader, I still wanted the current shader for the rest of the scene. Since I didn't want to dive into custom stencils for its outlines issue, I decided to create a cel shader material layer that would then alter the diffuse color of the robot in its shadow areas. This was done through the dot product.

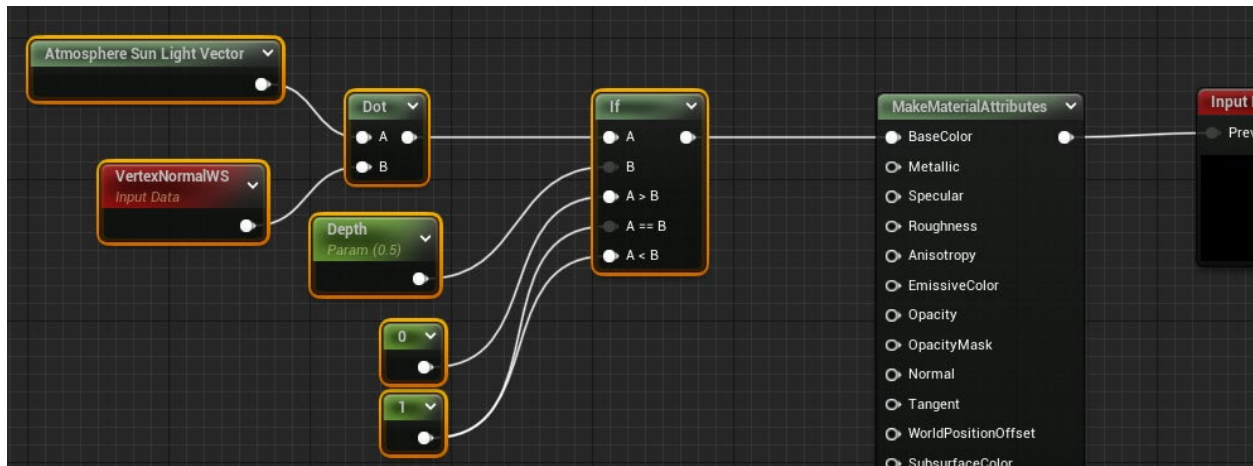


Figure 83: ML_LocalCel

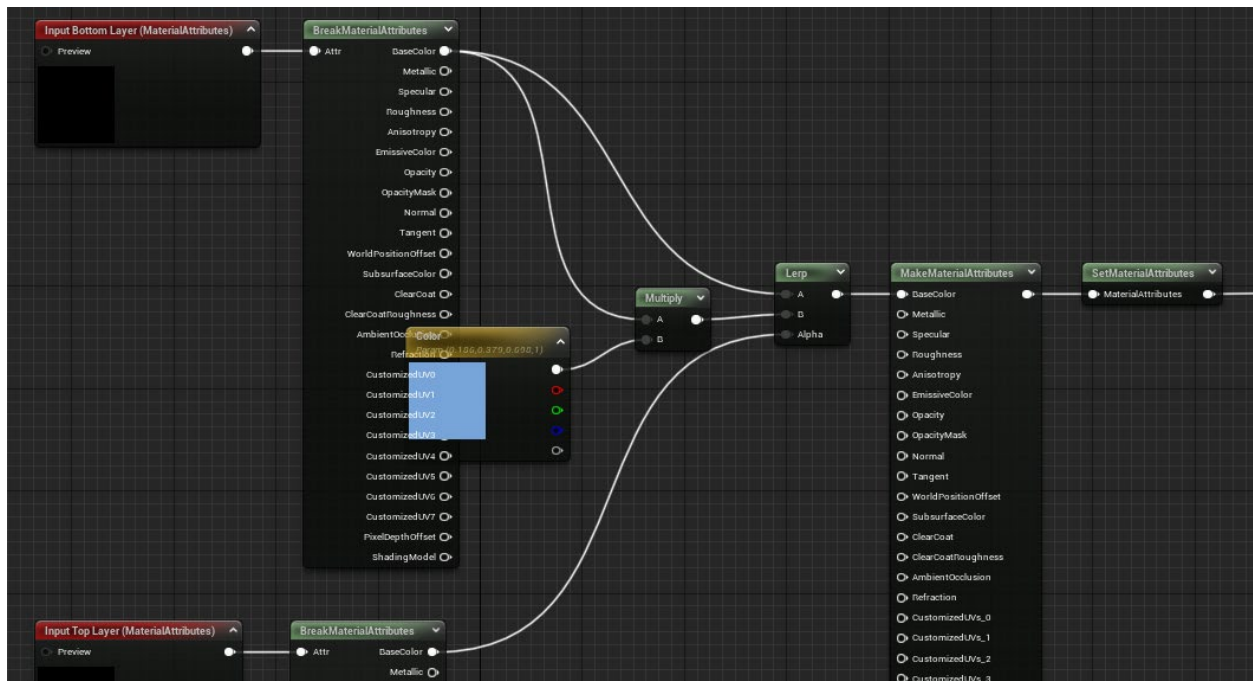


Figure 84: MB_LocalCel

2.3.5 MOSS PATCHES IN BASE COLOR

The Layer for the moss patches was just a simple color node in the base color, specular and metalness set to 0 and roughness to 1. I created a new blend asset that uses a texture as a mask to blend the two layers. The user has the possibility to graypack textures and then tell the blend asset which channel of the texture to use. I used two layers to make the patches: a layer for the moss patches, which was packed in the R channel of a texture, and a layer for their outlines, which was packed in the G channel of the same texture. Since these patches are only rendered in the base color of the robot, there's no way for the post-processing volume to detect outlines. That's why I decided to fake the outlines in a texture. The making of this texture will be discussed in part 4 of this paper. However, it's important to note that post-processing volumes can detect outlines on decals and meshes that have opacity, which are also elements I will create and use for the moss.

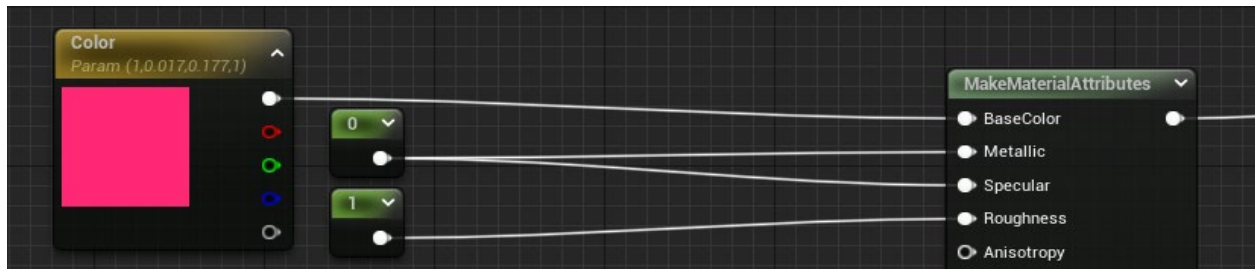


Figure 85: ML_Basic

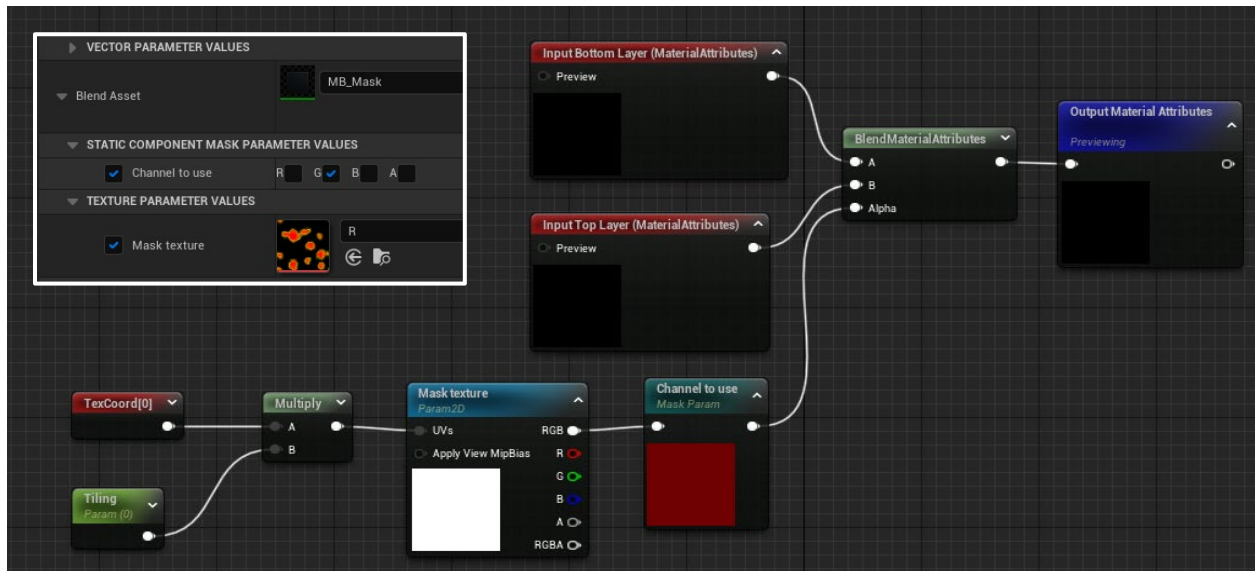


Figure 86: MB_Mask

2.3.6 VERTEX PAINT

While vertex painting was not needed for the concept, I wanted the option to be there in the material library. Any layer can be used to vertex paint with as long as it uses the vertex paint blend asset.

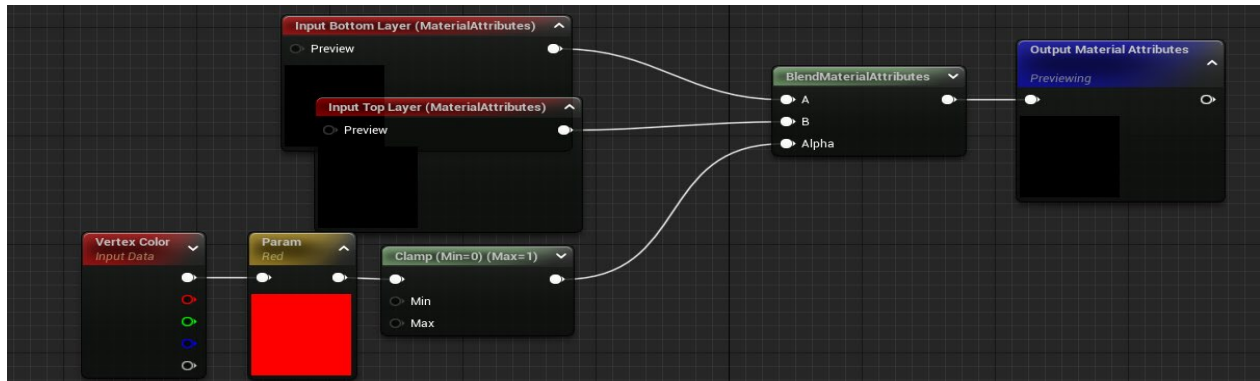


Figure 87: MB_VertexPaint

2.3.7 GRADIENT BLEND

A gradient way of blending was added to give more blending options in the material library.

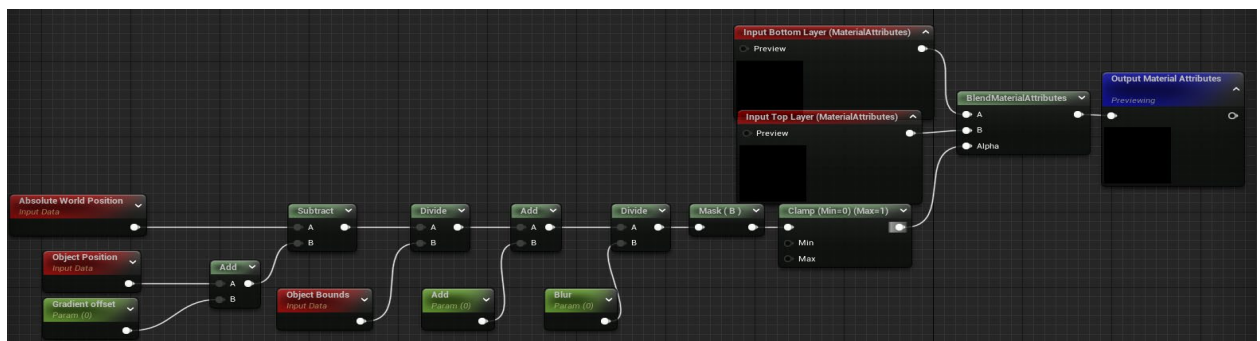


Figure 88: MB_Gradient

2.3.8 MOSS PATCHES AS DECALS

As mentioned before, the layered material system can be used for decals as well. This way, you can use from the same material library as your assets. A new layered master material needs to be made but with material domain now set to Deferred Decal. Because decal blend mode is gone in UE5, I only could get my decals to work by using the ApplyDbuffer nodes. When this material is set up, you can now use it like the previous layered materials.

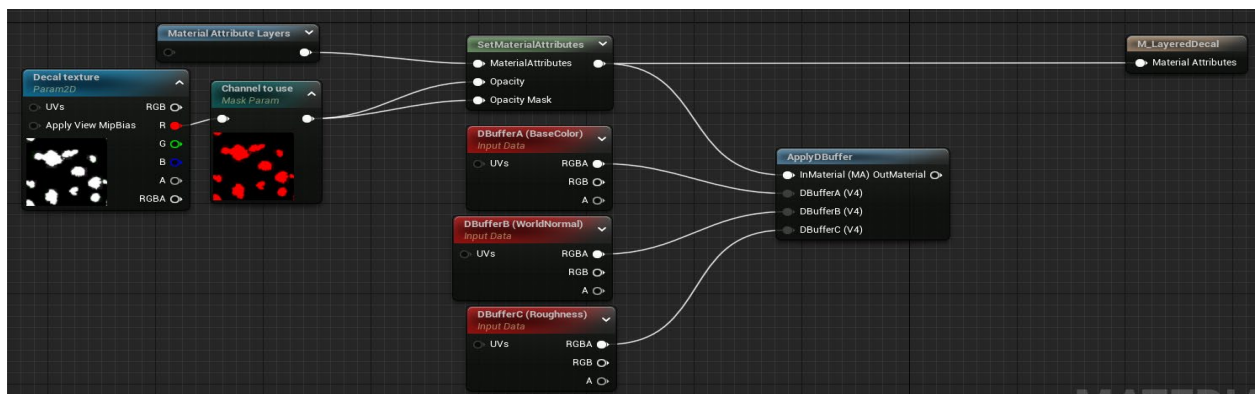


Figure 89: Moss decals – master material (layered)

3. CREATING PROCEDURAL TEXTURES IN SUBSTANCE DESIGNER

The process of constantly adjusting, exporting and importing textures from alternative software into Unreal Engine to see if they would fit the scene became tedious. Because of that, I wanted to implement another automated system in this part of the pipeline for the textures I had to make for the scene. I wanted to find a way for the user to adjust some textures procedurally in-engine and that led me to Substance Designer.

Substance Designer is a node-based, non-destructive application for creating materials. The software is industry standard. One of its key features is that it allows users to create procedural and thus customizable material files (.sbsar), which can be imported in game engines and then be adjusted in real-time.

Being able to immediately test a texture and adjust it where necessary in-engine speeds up the workflow by a lot. It also allows for a variety of textures created by single material.

Underneath, I will discuss the Substance Designer materials created for the scene and how adjustable they are in-engine. While these materials are more on the simple side, there is so much more that can be done with Substance Designer. However, since time is limited, research has been kept on the materials needed to recreate the concept.

5.1 HATCHING TEXTURE

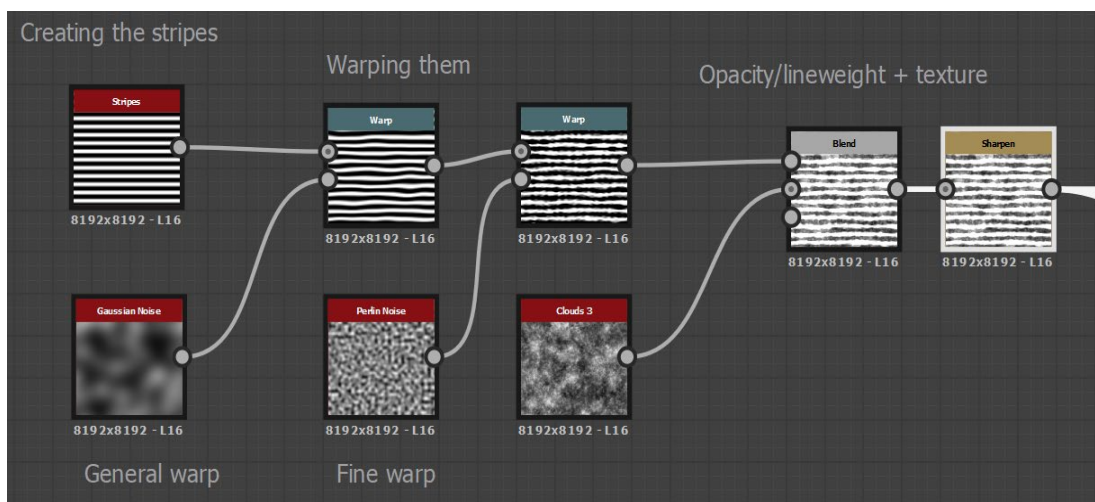


Figure 90: Hatching texture nodes in Substance Designer

The workflow for this procedural hatching pattern is as follows: a Stripes node was used to create the lines. These were then warped with a Gaussian Noise to put a general waviness into them, making them feel as if they were hand drawn. Next, a Perlin Noise was added to have the lines bleed out a little. They were then blended with a Clouds 3 to put a pencil-like texture into them. The result was sharpened to enhance the pencil texture.

The Stripes node offers the following parameters that can be exposed and adjusted in-engine: the amount of stripes and the width of them. The scale of the Gaussian Noise was exposed as well to give the user control over the waviness. Same goes for the Perlin Noise, the Clouds 3 and the Sharpen node.

To create crosshatching, a Switch Grayscale node was added to overlay a copy of the same lines on top of the existing lines. Both can be adjusted individually. Since there is no for-loop, the system is limited to how many times were coded in.

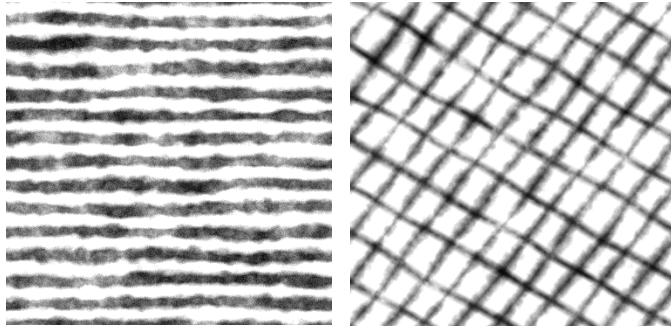


Figure 91: Textures created in Substance Designer using the Stripes node

While this way more lines can be added on top of the existing ones, it's also possible to layer lines on top of each other with the layered material system. However, overlaying lines on top of each other with the substance graph and outputting that texture to use in the layered material system is more performant than adding lines on top of each other with the layered material system. Each layer you save gives you performance back.

An alternative method was tried with the Tile Sampler and the Tile Generator, since both nodes offer a lot of customizability, like X and Y amount of lines, size, scale, rotation, randomness,... The Tile Sampler takes longer to process an adjustment (between 50-60ms for 1k texture) than the Tile Generator (1-2ms for 1k texture) while creating the same result. The Tile Generator also wins from the Stripes node (20-40ms). Because of that, it's best to use the Tile Generator in the .sbsar.

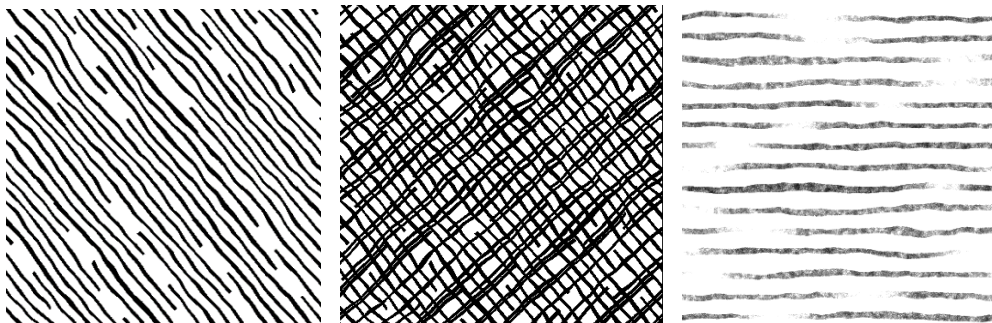


Figure 92: Textures created in Substance Designer using the Tile Generator

To import and use .sbsar materials in Unreal Engine 5, the Adobe Substance plugin needs to be installed. When the material is then imported, "Output" in the graph needs to be enabled. This will make textures of the different outputs (made in Substance Designer) of the material. These textures can then be used like any other texture in the layered material system. Then, when any of the parameters are adjusted, the material will override the textures with a new, adjusted version, automatically updating everywhere it's being used.

In the end, I settled for the textures from Kamil Hepner to texture the final scene.

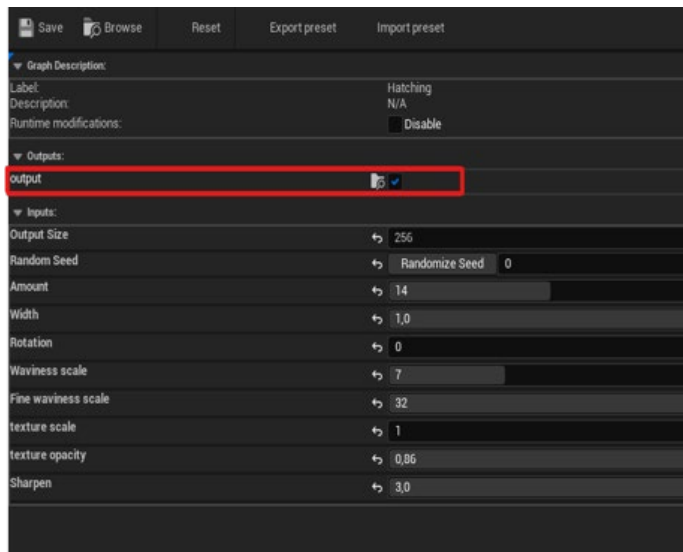


Figure 93: The setting that needs to be enabled to export a texture from a substance graph

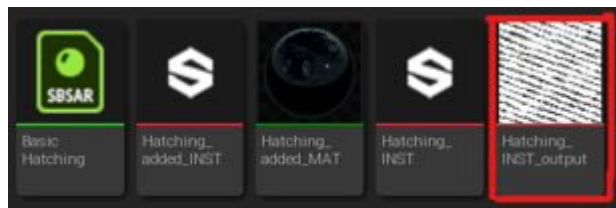


Figure 94: The texture that gets exported

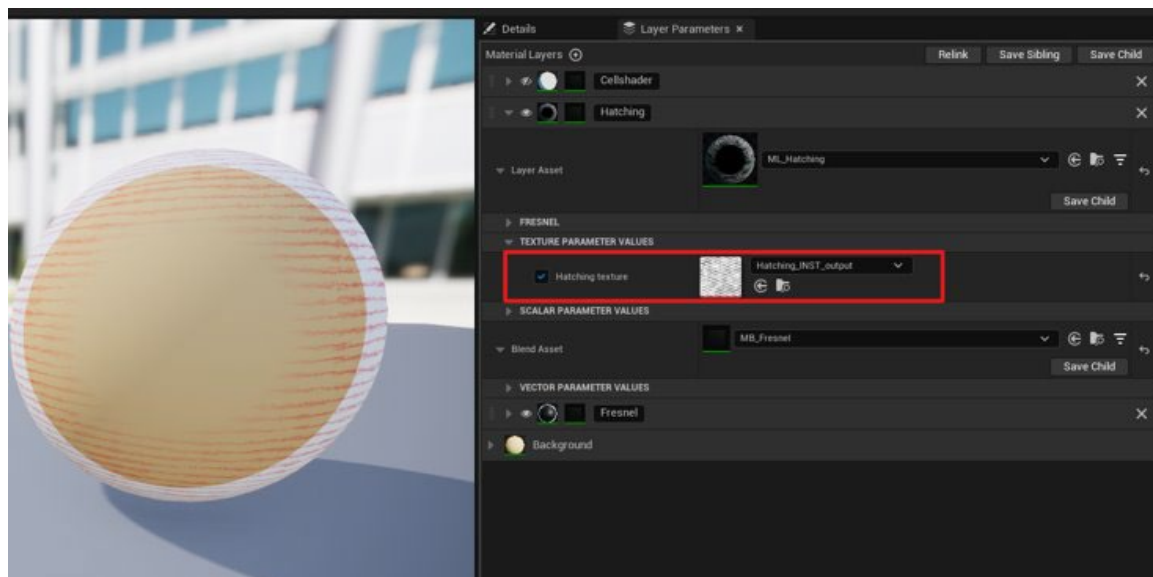


Figure 87: Putting the texture inside the hatching shader of the layered materials

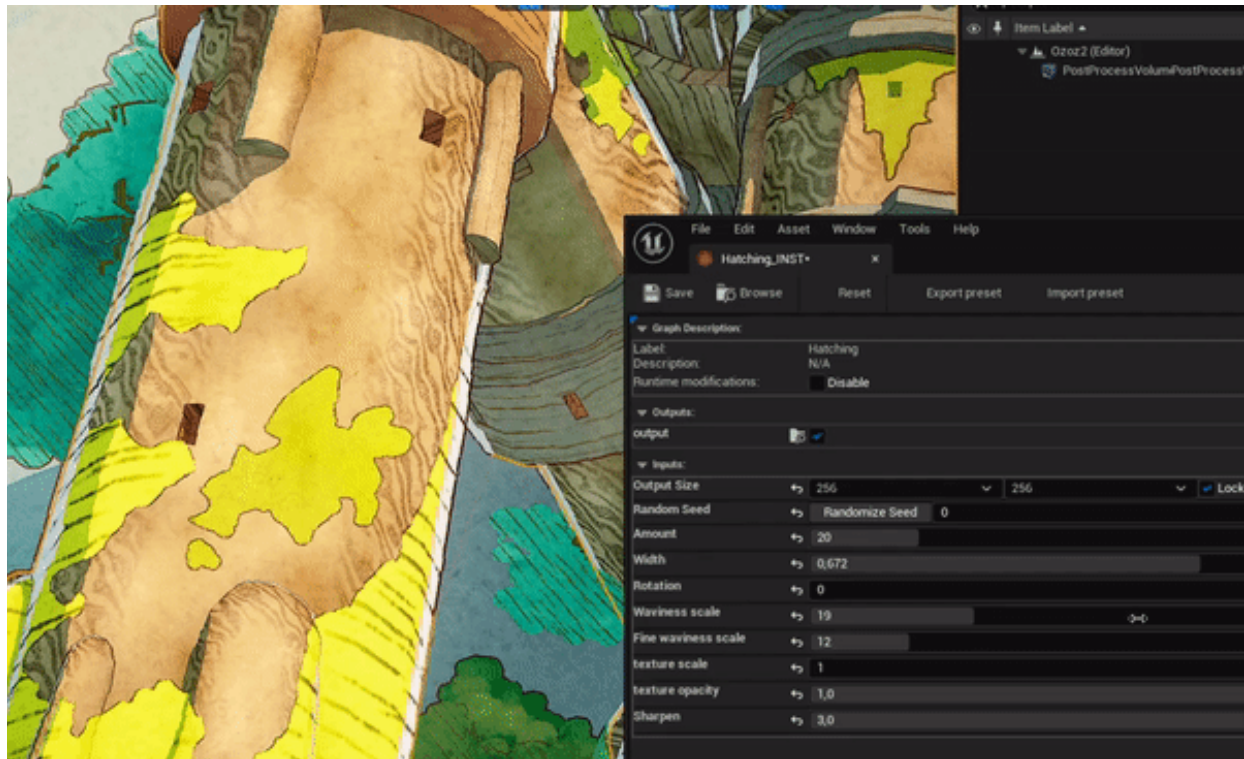


Figure 95: The .sbsar material in action

5.2 WATERCOLOR TEXTURE

To mimic watercolors with software, there's a lot of traits that need to be considered and mimicked to get your material across as real watercolors. While it is important to know how watercolors work to be able to recreate them, and while I did research this, I won't discuss this theory in the paper. The materials made were tailored to the concept, which could mean some characteristics of watercolors weren't needed.

The key traits we can observe from the concept are a slight "spottiness", which could also be due the paper being used, areas bleeding into each other and fluctuation of pigment intensity.

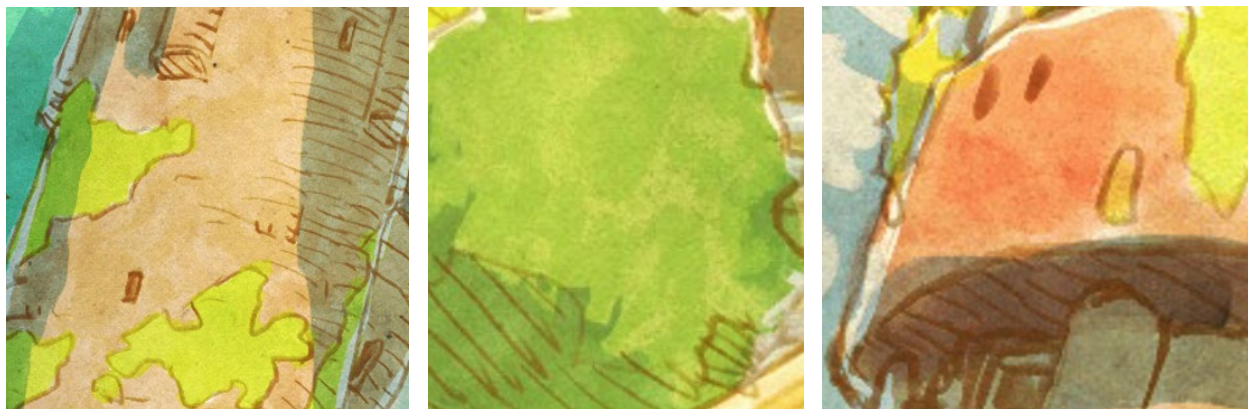


Figure 96: Some examples of the watercolors in the concept

All of the following attempts used elements from [Jimmy Malachier](#)'s tutorial for the creation of hand-painted/Ghibli-style textures in Substance Designer.

In the 1st attempt an EZ splatter was used to distribute bitmaps of watercolor brushes (by [Kirsten Harri](#)). The result was then used to lerp between 2 colors. Using alphas of brushes that are designed to mimic watercolors can help the process tremendously. However, they can get quite repetitive and require extra work to avoid that. Even though the brushes helped with watercolor traits, they didn't mimic the characteristics of the concept.

In the 2nd attempt I slope blurred a grunge map and then used it to lerp between 2 colors. This result was then sharpened. While this texture fits the concept a lot more, there is not a lot of adjustability to it besides the two colors and the amount of blur. In the end, I didn't feel the need to do adjustments in-engine.

In the 3rd attempt I used separate nodes for each characteristic. The base spottiness was achieved the same way the previous attempts were. With levels I added more contrast to create pigment intensity. To create bleeding, I slope blurred the result with a different dero nodes (by Jimmy Malachier). This was then blended multiple times with Gaussian Spots to mimic paint settling itself on paper. While each of these traits are adjustable in-engine, I didn't feel the need to do so. Because of that, I decided to let the material editor in Unreal Engine take care of basic adjustments: a black-and-white version of the texture lerping between 2 colors. Depending on the value of the colors used, spots can become more or less visible. In the end, I settled for the 2nd attempt for the brighter spots.

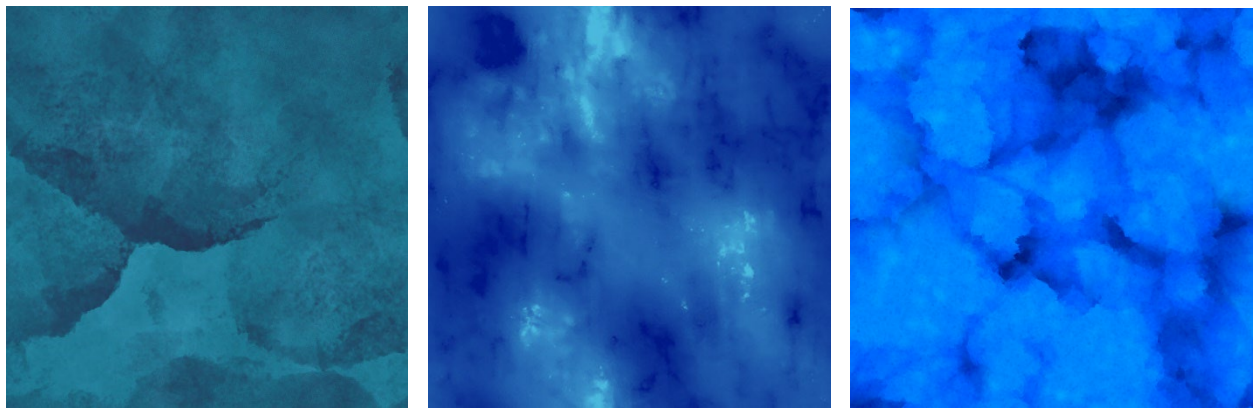


Figure 97: Substance Designer watercolor textures – 1st, 2nd and 3rd attempt

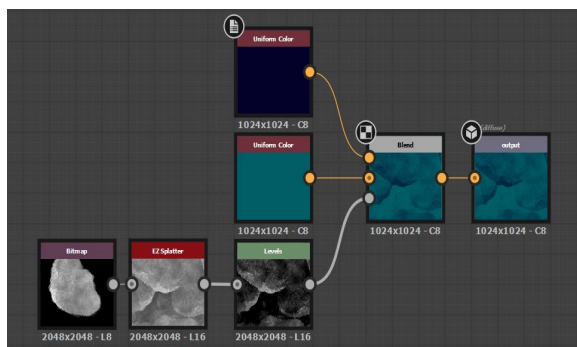


Figure 98: Nodes of 1st attempt

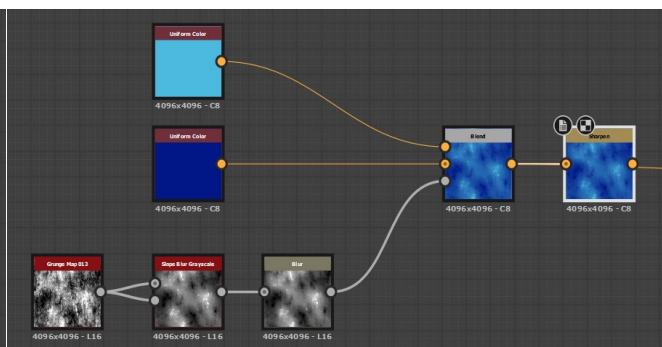


Figure 99: Nodes of 2nd attempt

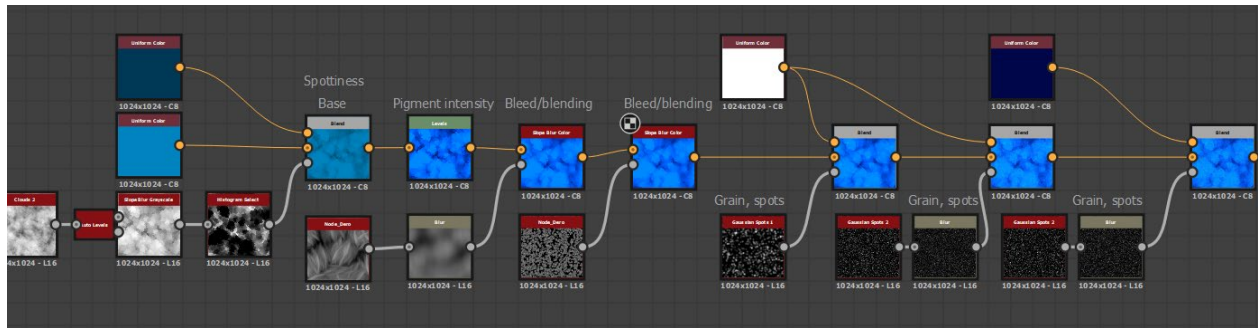


Figure 100: Nodes of 3rd attempt



Figure 101: Some examples of the watercolors in the scene – fog cards were turned off for this

5.3 MOSS PATCHES DECAL

What we can observe in the concept is that the moss patches are quite wavy and have an outline. Since I was planning to add them into the base color of the robot, I needed a way to fake the outlines.

To create the moss patches on the legs of the robot procedurally, the following workflow was used: An ellipse was distributed with the Tile Generator. These were then warped with a Gaussian and Perlin noise to get irregularity in the shape. An edge detect was then used to create the outlines. The shapes were put in the R slot of an RGBA merge and the outlines in the G slot. Because these are separate, the user can choose later whether they want both or not.

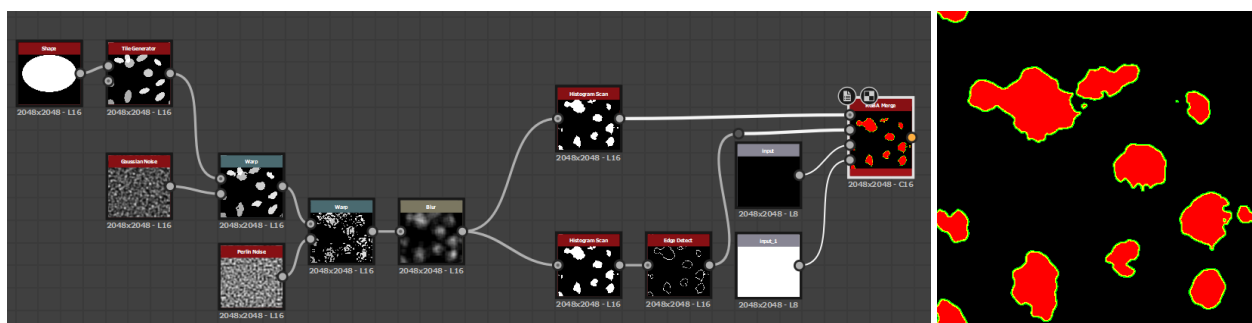


Figure 102: Nodes of the moss patches

When using this in the engine, the distribution of this texture on the legs felt too random. Because of that, I settled for making the textures in Photoshop by hand so they would fit the concept. These were then applied as decals to the robot. So, while the option is there to procedurally adjust and distribute moss patches, there's also room for an artist to make intentional choices and insert handmade textures where wanted. The procedural moss patches were instead used to create the patches on the ground.

4. MODELING AND SCENE COMPOSITION

Modeling

The trees in the concept don't have separate leaves. This means that normal 3D trees, trees with planes that have an opacity map in the form of a leaf, are not useable for this scene and require a different workflow to fit in. The trees made for the scene had to be made entirely out of different spheres merged together. Once the spheres were shaded, this would achieve the flat shadows on them like in the concept. I had to modify the outline shader to the point that it would only draw an outline on the silhouette of the whole tree and not on any inner spheres. The same approach was used for the clouds.

Dynamic elements

To add liveliness to the scene, some dynamic elements were implemented:

- Moving clouds
- Swaying grass with a texture panning on top of it to emulate clouds casting shadows on them
- Moving trees and bushes

Scene adjustments

To add softness to the scene, a few fog cards were made and set to additive. These were used extensively in different places on the scene, especially on the right side of the robot where the main light hits (as seen in the concept). The fog cards were created because it became difficult to have localized bloom. This is because a lot of color values are 1 and the scene technically became an unlit version because of the post-processing material.

Adding sharpness to the scene was very important for selling the look. This made the lines feel as if they were done by a sharp pencil. A blueprint was made that sets the sharpen tonemapper automatically.

Another blueprint was created to set the LOD of a mesh so that when it hits the fog zone, it would become lower res. This is to make it feel as if objects that are further away in the drawing, would be drawn more simple and less detailed so the focus stays on the main scene.

Underneath is the final result of the recreation of the concept in 3D through the use of the created shader. Then, there's how the UI of the shader looks like. The shader works in such a way that the user can first define which characteristics they want on the scene. Then, by enabling any of the settings, which can all work individually from each other, they can further customize them.

RESULTS AND EVALUATION

RESULTS

Underneath is the final result of the recreation of the concept in 3D through the use of the created shader. Then, there's how the UI of the shader looks like. The shader works in such a way that the user can first define which characteristics they want on the scene. Then, by enabling any of the settings, which can all work individually from each other, they can further customize them to their liking. Any kind of textures used in the shader can all be swapped out with the user's own textures.



Figure 103: The result of the recreation in 3D

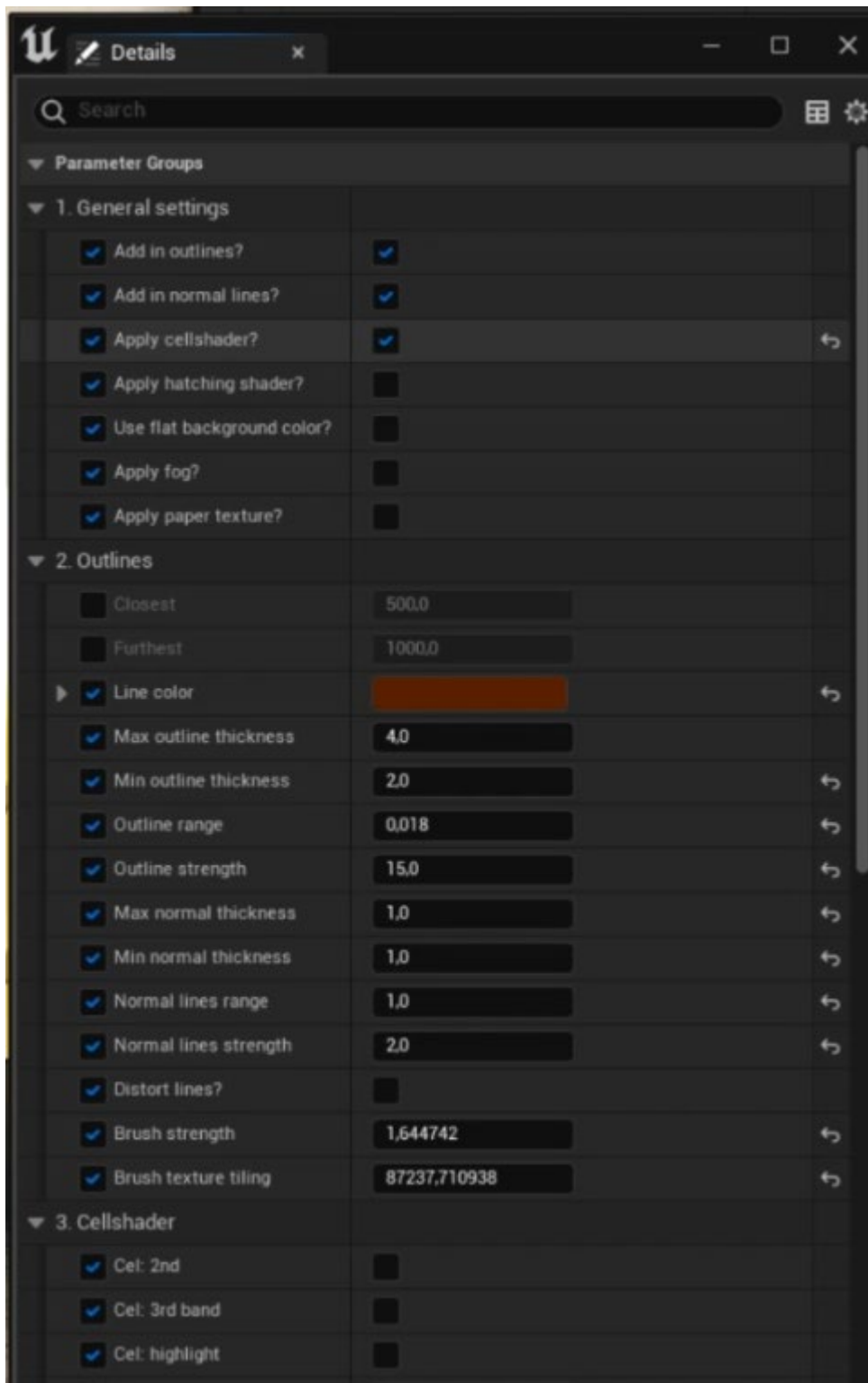
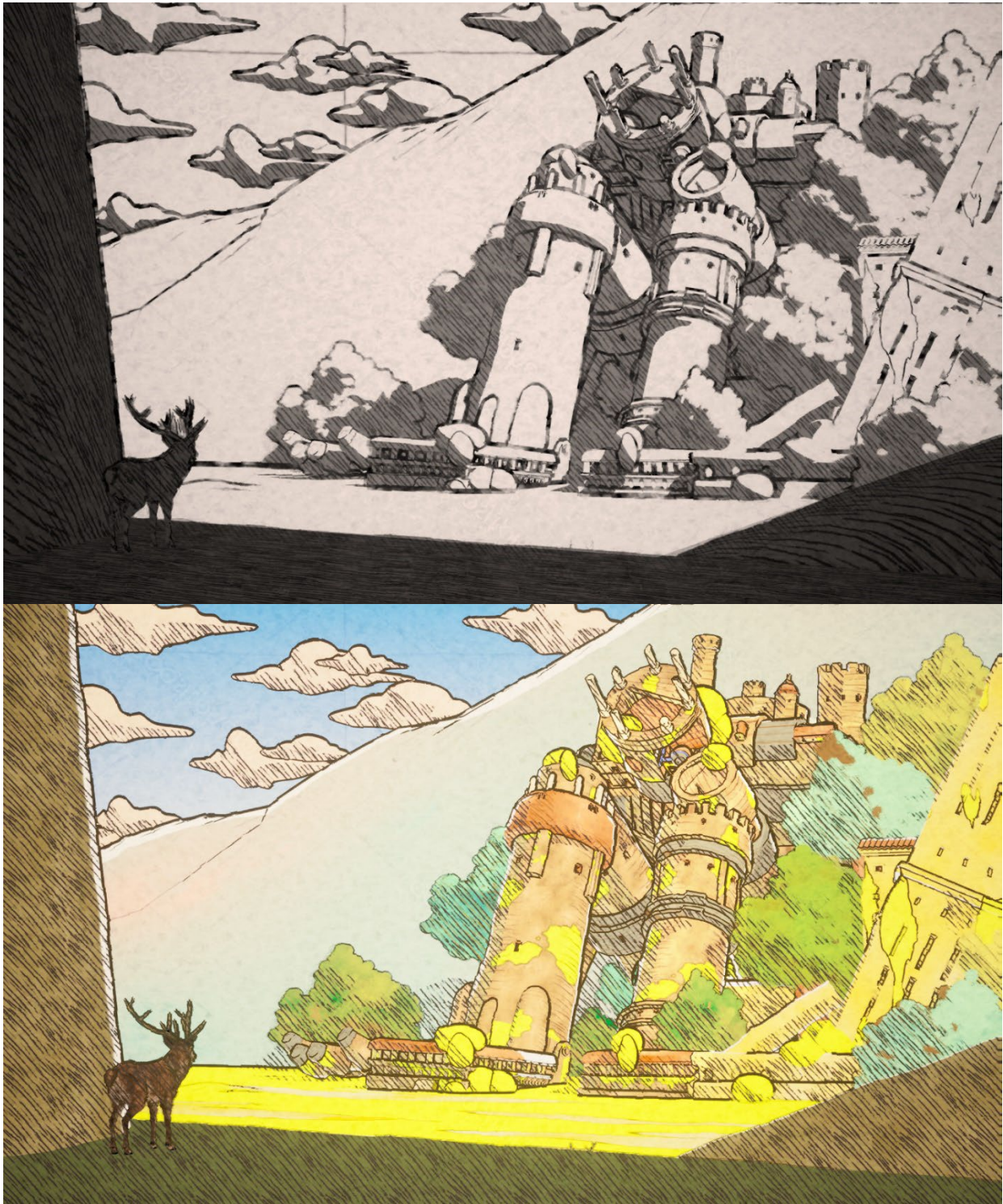
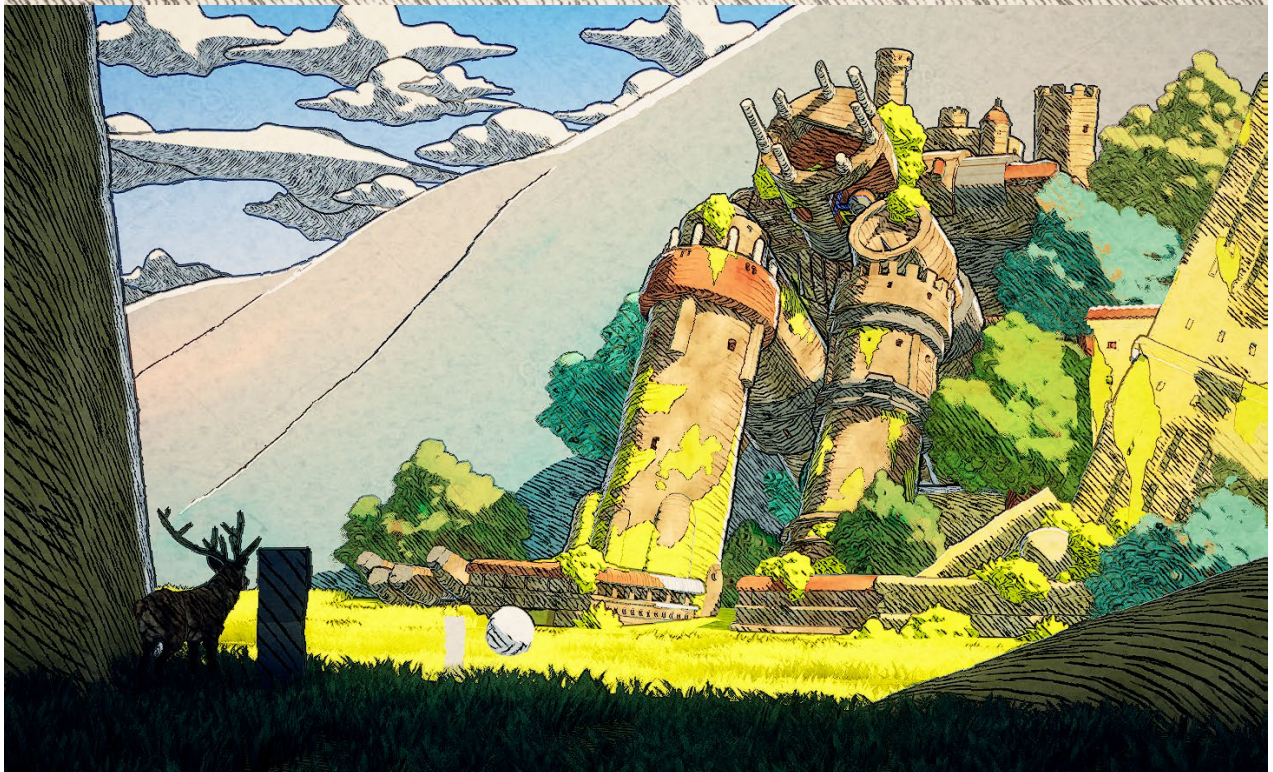
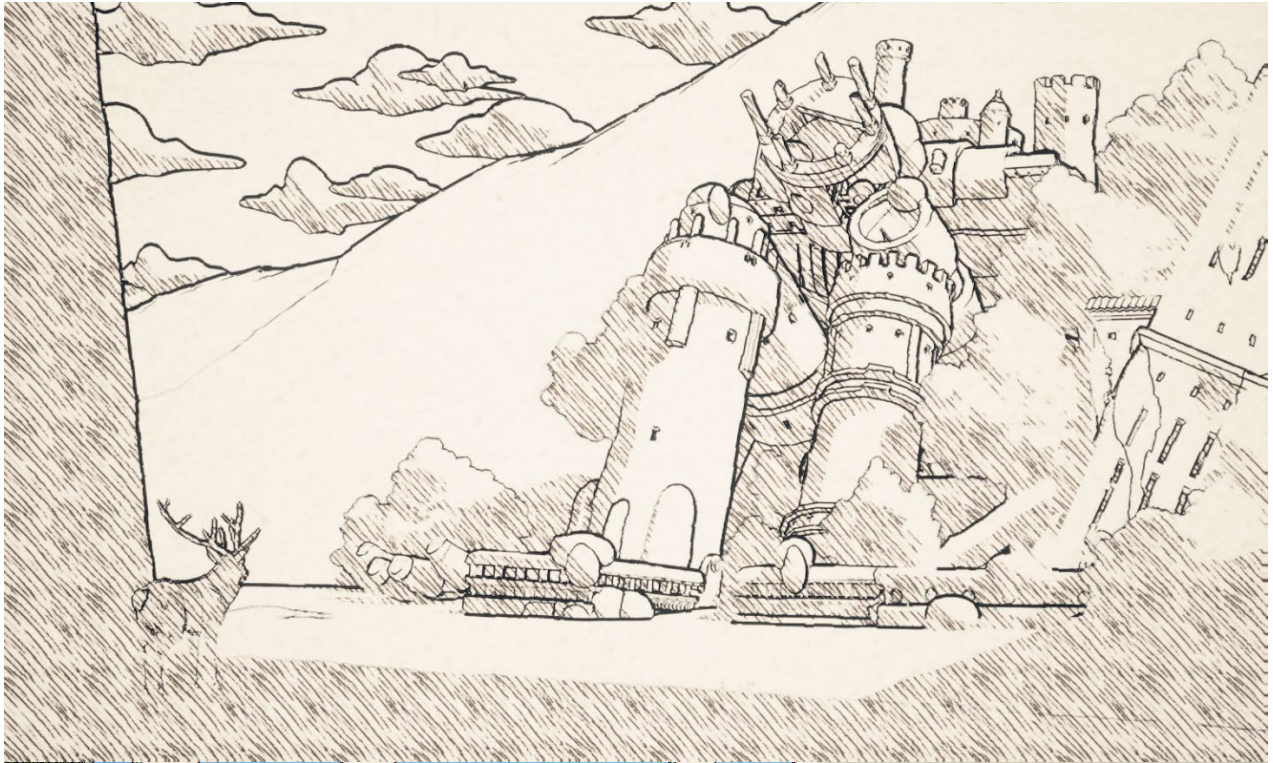
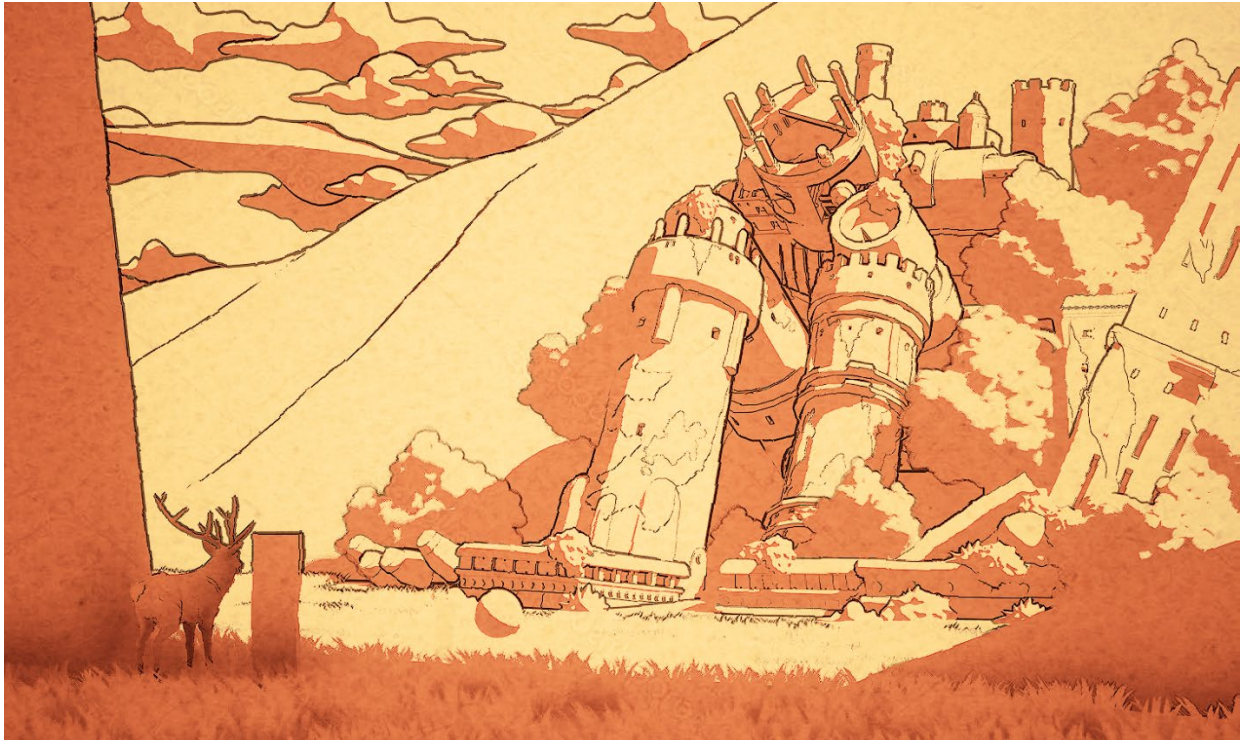


Figure 104: The UI of the created shader

The following variants of the scene were all created with the same shader and goes to show that the shader is capable of creating a variety of 2D looks.









The shader was then tested on a different, stylized PBR scene and proofs that it is applicable to any kind of scene.







EVALUATION

IMPLEMENTING POST-PROCESSING MATERIALS TO CREATE A HIGHLY ILLUSTRATIVE STYLE

Effectiveness and user-friendliness for an artist

Using post-processing materials to alternate the look of a game seems almost inevitable. Especially when going for non-photorealistic rendering, which is almost necessary to achieve a 2D style.

While post-processing materials offer a great node-based way for an artist to alter the look of a game without having to script into the rendering pipeline, it takes some time to understand how it works until it can be used to their advantage. However, since this topic is relatively popular, information and resources can be found online, which helps the learning process.

It's worth mentioning that, even though this can be used without having to program/script anything, this also means that you are limited by the possibilities within the node editor. Being able to create all my key characteristics with just nodes was a huge benefit for me. Though there were occasions when I felt like I could've benefitted from using code:

When working with multiple post-process materials, some texture ids can become overridden which can make blending effects on top of each other hard or require actual programming.

There was an occasion when I had overridden the final scene through adjustments in a post-process material. In the next post-process material I couldn't access the "clean" scene anymore and had to implement the "custom" node (allows you to code into the editor) to retrieve this from a different pass.

This happened a lot in the early stages of the projects. As I understood post-processing materials better, I didn't need any custom nodes at all anymore. Because of that, I think this was more due to my inexperience with post-process materials. This goes to show that it does take some time to understand and work with properly. If you plan on creating a new way of shading, almost a custom way of "rendering" like I did, I learned that generally it's best practice to compile all your shaders into one instead of having a lot of separate passes. This way, the previous pass stays the clean pass on which you can build your material on. Any smaller effect, that functions more as a filter instead of a whole new way of shading, can then be a separate pass. Like the pixel and the distortion shader. There are a few more reasons why compiling your shaders into one is important. This will be discussed in the performance section.

However, I found that, when working in a node-based editor, it is best to not implement code through the "custom" node. This increases shader compilation times significantly and makes the shader more complex.

Whether post-processing material speeds up the workflow in achieving a 2D style is a more difficult question. To achieve a hatching style, it does. But all other aspects are so vital in flattening out a 3D render that they don't replace any traditional workflow.

While the math within the shaders is always correct, you can't rely on this alone to achieve a 2D style. Adding imperfections in the shaders was needed to sell the look more, like the distortion for the outlines. However adding imperfections in post-process materials is not always easy and often miss subtlety and intent because the effect is global. Imperfections often needs to be added in a different part of the pipeline: like adjusting the geometry of the mesh or adjusting the textures.

Reusability

- The shader can recreate a variety of 2D looks with its current functionality and can be applied to any kind of scene. The shader has a lot of options for the user to insert their own textures.
- The variety is however limited to what has been added in. Any specific style, especially niche styles, need their own specific node network.
- The shader works well with decals and meshes that contain opacity.
- The shader relies on the G-Buffer from the deferred renderer and can only be used with this renderer. The forward renderer renders in a different way and has no G-Buffer. Using the shader with this renderer will break the shader.
- The shader doesn't work well with metalness. Because of the way the cel shading works in the shader, objects that have a metalness other than 0 will not render out properly.

Performance in-game

While optimization was not part of the research, I want to look at how the current systems run in-game and see where some of the flaws are and how these could be fixed. This will give an idea about the potential for the shader in a game setting.

Computer specs:

Lenovo IdeaPad Y50-70-01516

- **CPU:** Intel (R) Core (TM) i7-4720HQ CPU @ 2.60GHz
- **RAM:** 16.0 GB
- **GPU:** NVIDIA GeForce GTX 960M

For a more just evaluation of the shaders, I had to disable my trees. The trees have an extremely high poly count, were not optimized and thus had a lot of draw calls. Because of that, they are slowing down the scene and all the render passes by a lot. Especially with the dynamic light shadows and the custom depth passes. Since this is an issue inherent to the mesh and not the shader, they were disabled.

It's worth noting that a lot of the following mentions could be improvement if run on better hardware.

With outlines, normal lines, cel shader and paper texture enabled in my shader, I am able to reach 45-50 FPS in the main static camera view of the scene. This stays somewhat consistent when flying and moving around in the scene.

The final scene takes about 20,15 ms to render each frame. As expected, most of the GPU cost goes into post-processing: 12,12 ms. When disabling the post-processing materials, the duration gets almost halved: 7,75 ms. To get the scene optimized for 60 FPS, we need the scene to cost around 15 ms. The shader is taking up a lot of this bandwidth.

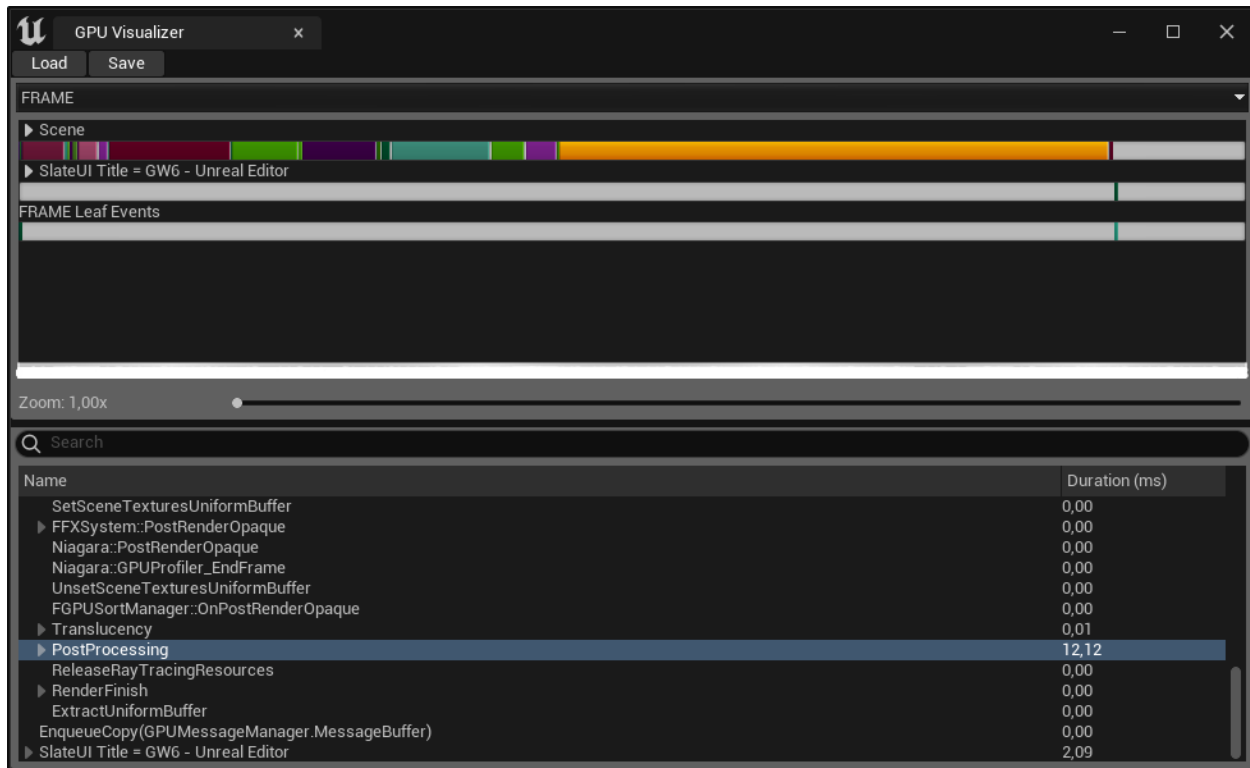


Figure 105: The profileGPU of the scene and the cost of the post-processing volume

The cost of a post-processing material is affected by:

- **The rendering resolution**
- **The number and quality of post-processing features**
My volume takes care of exposure, color adjustments of shadows and midtones and bloom. This is set to infinite extent.
- **The number and complexity of blendables (post-processing materials)**
Combining all my shaders into one results in 2 materials in the post-processing stack: the shader and another distortion filter.

I wanted to see how different these numbers would be if I split up my main post-processing material and added each part as a separate material to the stack. I observed it also takes about 12 ms to render each frame then. While the cost stays the same, an extra pass is added per material to compose the final image. Since it's best to limit the amount of passes, I combined all my shaders into one.

I checked how heavy and costly each part of my shader was. Enabling them one by one and checking the GPU profiler always gave me around 11ms per frame which didn't add up with the fact that, when everything is enabled the scene takes around 12 ms. When taking a look in renderdoc, the most "expensive" shader is the hatching shader (1200 μ s)

What I can conclude is that the use of custom depth on meshes with a high polycount can quickly ramp up the time needed to render each frame and that this also effects other passes than just the post-processing pass.

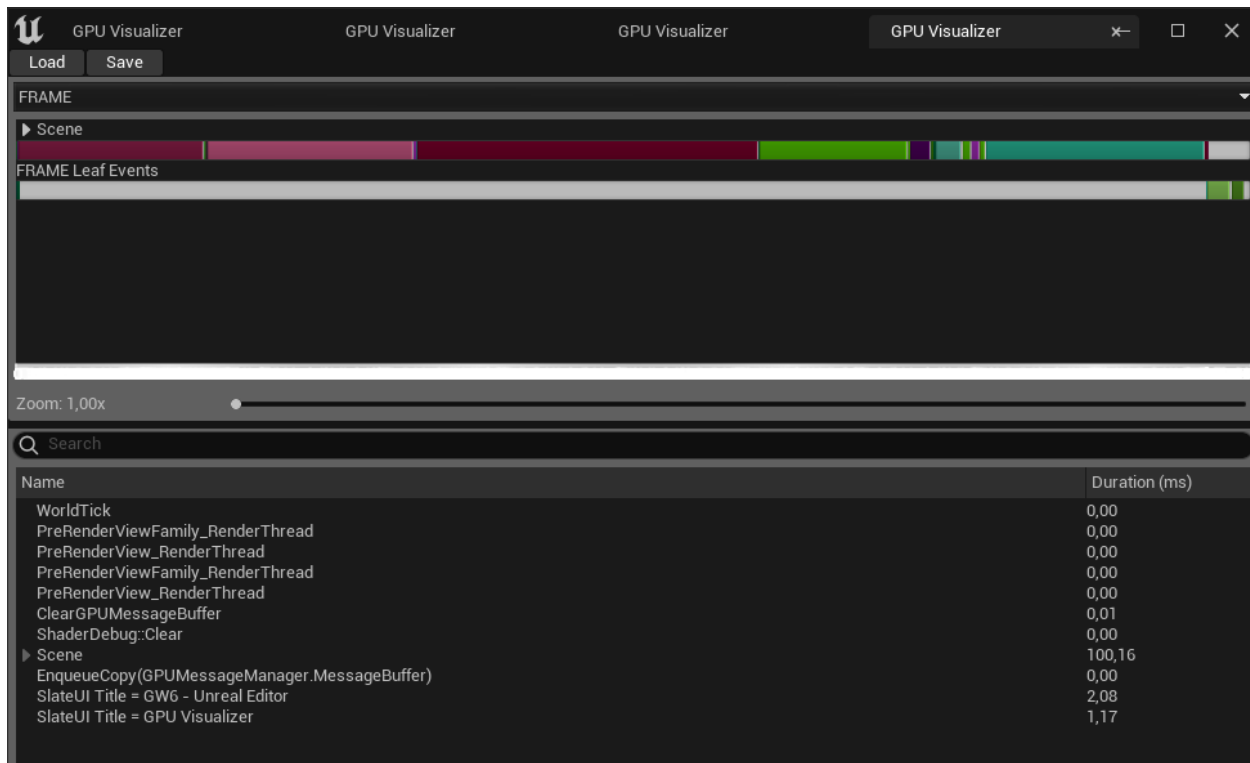


Figure 106: The cost of the scene with custom depth on the dense trees

What could make or break this look

Apart from breaking the CGI look of shaders like these, another challenge these shaders face is how they hold up in movement, how they hold up while a user is going through the level. To get as close to the concept as possible I used a combination of textures mapped to the world as well as to the camera. However, the textures mapped to the camera can break the look for a player because their tiling stays the same in movement. Mapping to the camera for a game would therefore not be an option. Then, it's best to either textures with the UVs of a mesh or by mapping the textures to the world. It's a very important aspect to keep in mind when making shaders like these as look dev for a game.

USING THE LAYERED MATERIAL SYSTEM FOR A HIGHLY ILLUSTRATIVE ART STYLE

Since the post-processing materials were the most important part of this article, the following evaluation of the layered material system will be kept short. For any further experiments and thoughts on it, I would like to refer to [Silke Van Der Smissen](#) who has extensively researched the system and shared her workflow.

Effectiveness and user-friendliness for an artist

Because all the elements in the concept were quite *layered*, I saw it as an opportunity to try-out the layered material system to texture the assets completely in-engine for an illustrative art style. I can say this was a pleasant experience and I can definitely see myself using this for any other scene and in another style as well. The use of the layered material system felt very intuitive. Almost like working with photoshop. The fact that material graphs don't become as complex anymore definitely was a great advantage for me, as well as being able to disable a layer without having to go into a node network.

However, as the material gets more complex, swapping around layers or deleting them can result in long shader compilation times. Since I didn't recreate the traditional workflow due to time constraints, I couldn't compare the difference between the two.

Reusability

- + With a few basic material layers and blends, you can go a long way.
- + The system can be used for decals as well. Because of that, decals can share the same material library as the assets, which can create a coherent look for a scene.
- Something specific needs to be build-in (though if build well, an asset can become very reusable).

Performance in-game

- When a stack is a bit more complex, swapping around or deleting layers can lead to long shader compilation times.
- The amount of layers ramps up shader complexity relatively quickly.

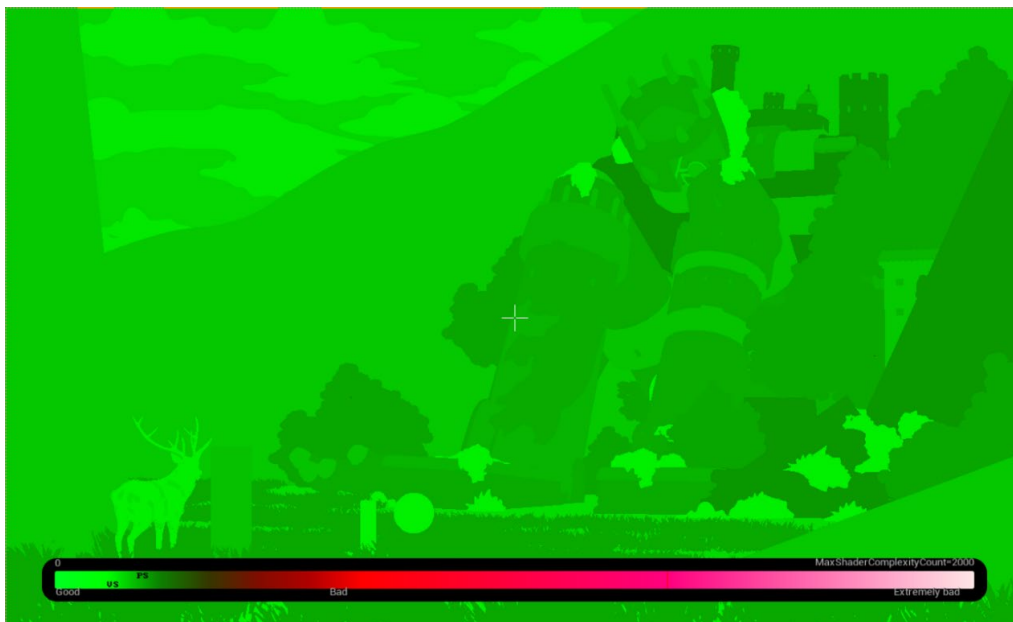
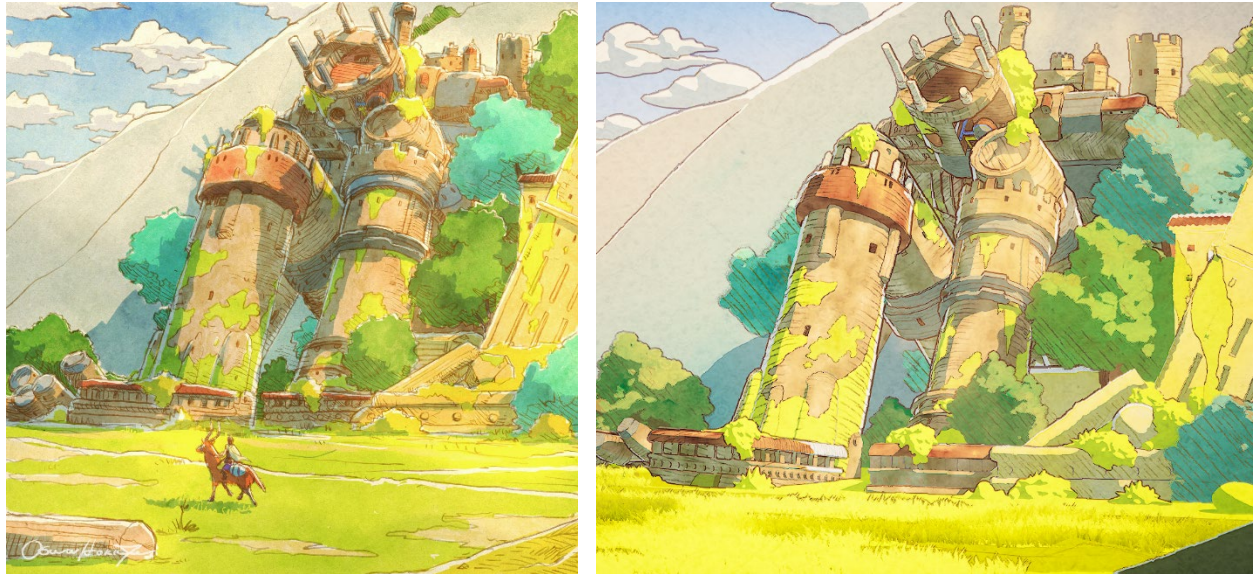


Figure 107: Showcasing the shader complexity of the layered material system

THE CONCEPT VS THE FINAL RESULT

In this last section I will compare the final piece with its concept and discuss my experience in recreating its key characteristics.



One of the most challenging parts was making everything appear as organic as the concept. I definitely felt the need for further adjustments of my calculations to make them appear more organic, especially for the outlines. Without all the distortions added in, they looked very CGI. However, with the distortions and opacity, they got more of a hand drawn look and I'm quite satisfied with how they turned out. While there is a general distortion applied to the whole scene, some elements could have been pushed more. For example, distortion on the shadows, so they feel more hand painted and less sharp. The robot could also have had another push in its geometry to get rid of straight lines. Putting more of that intentionality in there.

Another tricky aspect related to the outlines is that often small details didn't register and because of that some lines weren't drawn. Objects that were overlapping with an object that has custom depth enabled, also wouldn't draw their outline. It was a hard issue to fix and in the end I had to abandon it to focus on other important aspects. As the concept is very linework heavy another pass on this could push the work even more.

Another challenging part to recreate were the less straightforward and more abstract elements about the piece. For example, the scene has a sort of softness to it. Especially on areas where the light hits. I wanted to approach this softness by adjusting my bloom but as mentioned before, because of the way the cel shader works, it became hard to use bloom well. While the amount of sharpness applied was necessary to achieve that pencil feeling, it also made the piece look harsher. Lines had the tendency to become pixelated because of it. Scattering around fog cards softened everything up but too many fog cards would work against the pencil feeling of the outlines and would make the colors fade out and become a lot lighter. It was definitely an interesting balance to deal with.

While there is still room for improvement, I'm happy with how close I got to the concept purely using computer generated elements. It was a very interesting experience recreating a 2D art style in 3D and one that I would very much like to repeat. I really had a lot of fun creating the shader and tackling the challenge of making it feel organic. I'm also very happy with its capabilities. It's safe to say that, while I've probably only scratched the surface, making shaders is no longer a scary topic.

CONCLUSION

With this project, I researched how a 2D style can be achieved in 3D through implementing several automated/procedural systems:

- The use of post-processing materials to alternate the general look of a game.
- The use of the layered material system to speed up texturing a 2D style.

Next to that I wanted to evaluate how effective, user-friendly and reusable these systems could be and what could make or break this auto-generated 2D look for a player in-game. Now, I will give my final thoughts on the matter.

The use of post-processing materials seems almost like a necessary and fundamental step in the process of recreating a 2D art style in 3D. While they offer a great node based way for an artist to alter the shading of a scene, it does take some time to get used to. However, since in the field they are often used for NPR scenes, a lot of information and resources can be found online. Whether a very complex post-processing material can run well in a game is however a different research question to be answered.

The layered material system provides the power of material functions but without the complexity of the traditional workflow. In that sense, it offers a much more user friendly workflow for artists, especially since it's more visual. However, shader complexity ramps up the more layers are used, which needs to be kept in mind. Swapping or adjusting layers in a more full stack, can result in long shader compilation times. Apart from this, using it for a highly illustrative art style proved to be beneficial, especially for styles consisting of different *layers* which are reused a lot. Then it proves very beneficial making a material library with this workflow. Since the system is relatively new in Unreal Engine, it will likely see more updates and optimization.

The recreation of experimental 2D art styles in 3D is definitely a challenging exercise. The look often fails or succeeds based on its shading but most importantly its imperfections. It is clear that, while implementing any automated system or computer generated element is necessary, adjustments need to be added to break the CGI look. While automated systems can speed up the workflow, they should not replace the artist's intentionality, since that is what can really make or break a 2D look in 3D. Another challenge these shaders face is how they hold up in movement. When used for a game, this is an important factor that needs extra care to make a shader like this succeed in its objective.

Personally, I would like to see more NPR in the industry, especially the paths less traveled: the more experimental art styles. Like the developers from Guilty Gear said: a lot of research still needs to be done in this field and so many styles are still untouched. In that sense, a lot of potential lies in it and anyone who would tackle this, could possibly pioneer in it.

BIBLIOGRAPHY

- Arc System Works RED. (2015, May 21). *GuiltyGearXrd's Art Style : The X Factor Between 2D and 3D* - YouTube. <https://www.youtube.com/watch?v=yhGjCzxJV3E>
- Brady Matthew, & Unreal Engine. (2019, May 20). *Layered Materials for Environments | Unreal Fest Europe 2019 | Unreal Engine* - YouTube. <https://www.youtube.com/watch?v=dl5fe-DAXRI>
- Cloward, B. (2019, December 12). *Distortion Shader - UE4 Materials 101 - Episode 4* - YouTube. <https://www.youtube.com/watch?v=gwx2NOZJ5CE>
- Dermott, B. (2022, May 7). *ArtStation - Stylized Smart Shader, Dermott Burns*. <https://www.artstation.com/artwork/vJBezX>
- Hepner, K. (2021, December 5). *Hatching shader tutorial* - YouTube. <https://www.youtube.com/watch?v=18U50KPdD2A>
- Hepner, K. (2022, March 2). *Stylized lines and outlines - Unreal Material* - YouTube. <https://www.youtube.com/watch?v=pJ42yIVyDpc&list=WL&index=211>
- Kaufman, K. (2022). *Stylize Renders with Post Process Materials*. <https://dev.epicgames.com/community/learning/courses/qyl/stylize-renders-with-post-process-materials/G6l/introduction-to-stylized-rendering>
- Lightning Boy Studio. (2020, August 18). *How to Create Ghibli Trees in 3D - Blender Tutorial* - YouTube. <https://www.youtube.com/watch?v=DEgzuMmJtu8&t=1153s>
- Looman, T. (2018, April 9). *Distance Fog Post-Process Material*. <https://www.tomlooman.com/unreal-engine-distance-fog-material/>
- Malachier, J. (2020). *Artstation - Handpaint/Ghibli style Texture with Substance Designer 2020.1.2*. <https://www.artstation.com/misuuu/store>
- Harri, K. (2021). *Artstation - Procedural Watercolour Material | Substance Designer & Painter*. <https://www.artstation.com/kirskii/store>
- Penty, C., & Wong, I. (2017). Gears of war 4: Creating a layered material system for 60fps. *ACM SIGGRAPH 2017 Talks, SIGGRAPH 2017*. <https://doi.org/10.1145/3084363.3085026>
- PrismaticaDev. (2020, December 12). *Advanced Cel-Shader in UE4 & UE5 [Prismati-Cel Part 1]* - YouTube. <https://www.youtube.com/watch?v=RkFwe7JI8R8&t=243s>
- Świerad, O., & Tech Art Aid. (2019). *Introduction - Unreal Art Optimization*. <https://unrealartoptimization.github.io/book/>
- Tran, T. (2018, February 27). *Unreal Engine 4 Cel Shading Tutorial | raywenderlich.com*. <https://www.raywenderlich.com/146-unreal-engine-4-cel-shading-tutorial>
- Tran, T. (2018, May 1). *Unreal Engine 4 Paint Filter Tutorial | raywenderlich.com*. <https://www.raywenderlich.com/100-unreal-engine-4-paint-filter-tutorial>
- Unity, & Natalie Burke. (2019, August 30). *Learn how to do stylized shading with Shader Graph - SIGGRAPH 2019* - YouTube. <https://www.youtube.com/watch?v=4XxfiwNqU4I>

Unreal Engine. (2018, January 4). *Post Processing in UE4: Cel-Shading | Live Training | Unreal Engine Livestream - YouTube*. <https://www.youtube.com/watch?v=cQw1CL0xYBE>

Unreal Engine. (2020, August 19). *How to Create Masks With the Custom Stencil Buffer | Tips & Tricks | Unreal Engine - YouTube*. https://www.youtube.com/watch?v=PiQ_JLJKi0M&t=3s

van der Smissen, S. (2022, January). *ArtStation - Lost City*. <https://www.artstation.com/artwork/VyLgon>

The people that inspired me to do this:

- Nick Carver: <https://www.artstation.com/artwork/689JQr>
- Nick Carver: <https://www.artstation.com/artwork/IR9O6J>
- Lotte de Brabander: <https://www.youtube.com/watch?v=KPFep7SFIps&t=2s>
- The game "GRIS" by Nomada studios: <https://www.youtube.com/watch?v=Y7SUJljekc4>
- The game "Old Mans Journey" by Broken Rules <https://www.youtube.com/watch?v=tJ29Ql3xDhY&t=9s>
- The game "Dordogne" by UN JE NE SAIS QUOI <https://www.youtube.com/watch?v=DhXimnpPalg>
- Laure Dhooge "Charcoal and Rinus van de Velde" <https://www.artstation.com/artwork/oOYANw>
- Chau Nguyen Thi Hoai "The First Journey" <https://www.artstation.com/artwork/gJQOWm>